Developing Qt Quick and QML Applications with Python and C++

A practitioner's guide for experienced developers.

Table of Contents

Part I: QML & Qt Quick Foundations

- Chapter 1: The QML Ecosystem: Architecture and Design Philosophy
- Chapter 2: QML Language Deep Dive
- Chapter 3: Qt Quick Core Primitives

Part II: Qt Quick Controls & Layouts

- Chapter 4: Qt Quick Controls 2: Internals and Customization
- Chapter 5: Layouts, Positioners, and Responsive Design
- Chapter 6: Models, Views, and Delegates
- Chapter 7: Navigation Patterns and Application Shell
- Chapter 8: Animation and State Machines

Part III: Python Integration

- Chapter 9: Qt for Python (PySide6): Architecture and Tooling
- Chapter 10: Exposing Python Objects to QML
- Chapter 11: Data Models in Python
- Chapter 12: Python Backend Patterns
- Chapter 13: Packaging and Deployment

Part IV: C++ Integration

- Chapter 14: Qt Quick from C++: Project Structure and Build Systems
- Chapter 15: Exposing C++ Types to QML
- Chapter 16: C++ Models and Data Pipelines
- Chapter 17: Extending the Scene Graph
- Chapter 18: C++/QML Interop Patterns and Performance

Part V: Production-Grade Applications

- Chapter 19: Internationalization, Accessibility, and Theming
- Chapter 20: Testing Qt Quick Applications
- Chapter 21: Architecture Patterns for Large Applications

Chapter 1: The QML Ecosystem: Architecture and Design Philosophy

The Declarative Paradigm vs. Imperative UI

Qt Quick applications are built around a declarative model: you describe *what* the UI looks like in a given state, not *how* to get there. This is a significant departure from traditional widget-based toolkits where the developer drives widget construction, layout recalculation, and repaint calls explicitly.

In a widget-based system, a developer might write:

```
// Imperative: tell the system every step
QLabel *label = new QLabel(parent);
label->setText("Hello");
label->setGeometry(10, 10, 200, 30);
label->setVisible(true);
```

In QML, the same thought is expressed as a self-contained description:

```
Text {
    x: 10; y: 10
    width: 200; height: 30
    text: "Hello"
}
```

The difference becomes profound when state changes. In the imperative model, the developer writes transition code explicitly — detecting the state change and issuing updates. In QML, *bindings* propagate changes automatically through the dependency graph. When a property that another property depends on changes, the engine re-evaluates and updates all downstream bindings without explicit wiring.

This reactive model is the foundation of everything Qt Quick does. Understanding it early prevents a common mistake: fighting the binding engine

by manually assigning property values in JavaScript when a binding would serve better.

When Declarative Breaks Down

Declarative UI excels at describing structure and steady-state appearance. It is less natural for expressing sequences of imperative operations: network calls, file I/O, complex algorithmic transformations. QML accommodates these through JavaScript handlers and by delegating to backend objects written in Python or C++. A well-architected Qt Quick application keeps QML as the pure presentation layer and pushes logic into the backend.

Qt's Rendering Pipeline: Scene Graph and Graphics Backends

Qt Quick's rendering model is fundamentally different from Qt Widgets. Widgets use the QPainter abstraction, which ultimately issues 2D paint commands to a raster or platform surface. Qt Quick uses a *retained scene graph* — a tree of nodes that represents the visual output — rendered on a dedicated render thread using a GPU-accelerated backend.

The Scene Graph

When QML items are instantiated, the Qt Quick runtime builds a scene graph: a tree of `QSGNode` subclasses that carry geometry, materials, transforms, and clipping information. This graph is traversed each frame to produce GPU draw calls.

Key node types:

Node type	Purpose
`QSGGeometryNode`	Carries vertex data and a material, represents drawable geometry
`QSGTransformNode`	Applies a matrix transform to its subtree
`QSGClipNode`	Clips its subtree to a stencil region
`QSGOpacityNode`	Applies opacity to its subtree
`QSGRootNode`	The root of the graph, owned by the `QQuickWindow`

The scene graph is not synchronized with the main thread on every property change. Instead, Qt Quick batches changes and synchronizes the scene graph with the UI state at the beginning of each frame, during the *sync phase*. This is why Qt Quick can maintain smooth rendering even when the main thread is busy.

The Render Thread

By default, Qt Quick renders on a dedicated render thread, keeping rendering decoupled from main-thread activity. The pipeline per frame is:

1. **Main thread**: Advance animations, evaluate bindings, run JavaScript
2. **Sync phase** (main thread blocked briefly): Copy UI state into scene graph nodes
3. **Render thread**: Traverse the scene graph, issue GPU commands
4. **Swap**: Present the rendered frame

This threading model means you must never touch scene graph nodes from the main thread outside the sync phase. Custom `QQuickItem` subclasses in C++ implement `updatePaintNode()` to safely write scene graph data during the sync phase.

Graphics Backends

Qt 6 supports multiple graphics backends through the *Qt Rendering Hardware Interface (RHI)*:

Backend	Platforms
Vulkan	Linux, Android, Windows
Metal	macOS, iOS
Direct3D 11/12	Windows
OpenGL / OpenGL ES	All platforms (compatibility)
Software rasterizer	Fallback, no GPU required

RHI abstracts the differences between these APIs so that most Qt Quick code is backend-agnostic. You can select the backend at runtime via the `QSG_RHI_BACKEND` environment variable or programmatically via `QQuickWindow::setGraphicsApi()`. This is relevant when integrating external rendering code or when targeting platforms with constrained GPU support.

QML Engine Internals, the JS Runtime, and Object Lifecycle

The QML Engine

The `QQmlEngine` is the root of all QML activity. It owns:

- The **type registry**: the mapping of QML type names to C++ or QML-defined types
- The **import system**: resolution of `import` statements to directories or plugins
- The **JavaScript engine**: a V4 engine (SpiderMonkey-derived) embedded in Qt
- The **component cache**: parsed and compiled QML components for reuse

- The **network access manager**: used for remote QML and image loading

An application typically has one engine. Multiple engines are possible but they share nothing — types registered in one are not visible in another, and objects created by one cannot be parented to objects in another.

```
// C++ main.cpp
QQmlApplicationEngine engine;
engine.loadFromModule("MyApp", "Main");
```

```
# Python main.py
from PySide6.QtQml import QQmlApplicationEngine
engine = QQmlApplicationEngine()
engine.loadFromModule("MyApp", "Main")
```

Components and Instantiation

A QML *component* is a compiled description of an object tree. Every `.qml` file defines a component. Components are instantiated to produce object trees. The `Component` type in QML gives you explicit control over this:

```
Component {
    id: dialogComponent
    MyDialog { }
}

// Instantiate on demand
onClicked: {
    let obj = dialogComponent.createObject(root, { title: "Alert" })
}
```

`createObject()` creates a new instance with the given parent and initial property values. This is the primary mechanism for dynamic UI creation.

Object Ownership and Memory

QML objects participate in two ownership systems that interact in non-obvious ways:

QML engine ownership: Objects created by the QML engine — via component instantiation, `createObject()`, or `Qt.createQmlObject()` — are owned by the engine's garbage collector. When the last reference (in QML or JS) to an object is dropped, the GC is free to collect it.

C++ / Qt ownership: Objects that have a QObject parent are kept alive by their parent. When the parent is destroyed, all children are destroyed with it. If you pass a C++ object to QML and that object is parented, the parent's lifetime governs it.

The conflict: If a C++ object with no parent is returned to QML, the engine may take ownership and destroy it. To prevent this, either parent the object to a long-lived C++ owner, or set `QQmlEngine::setObjectOwnership(obj, QQmlEngine::CppOwnership)` explicitly.

A common mistake:

```
// Bug: engine may garbage-collect this
QObject* MyBackend::createThing() {
    return new Thing(); // No parent, no explicit ownership set
}
```

Fix:

```
QObject* MyBackend::createThing() {
    Thing *t = new Thing(this); // Parented — C++ owns it
    return t;
}
```

The V4 JavaScript Engine

QML's JavaScript runtime is V4, a JIT-compiling engine embedded in Qt. It executes JavaScript written in QML signal handlers, property binding

expressions, and `.js` files imported by QML.

V4 integrates with the QObject meta-object system through an automatic bridge: accessing a QObject property from JS triggers the meta-object `read()` accessor; assigning triggers `write()`; connecting a signal passes through the meta-object signal machinery.

This bridge has a cost. Each property access from JS that crosses the C++/JS boundary involves a lookup and type conversion. Tight inner loops in JS that repeatedly access C++ properties are a known performance hazard. The mitigation is to cache values in local JS variables:

```
// Slow: meta-object lookup on each iteration
for (let i = 0; i < model.count; i++) { ... }

// Faster: cache the value
const count = model.count
for (let i = 0; i < count; i++) { ... }
```

For the most performance-critical logic, move the computation to C++ and expose only the result to QML.

Summary

Qt Quick's architecture is purpose-built for GPU-accelerated, reactive UI at scale. The key ideas to carry forward:

- The binding engine propagates state automatically; work *with* it rather than around it.
- The scene graph decouples rendering from main-thread activity, enabling smooth frame rates.
- RHI makes Qt Quick portable across Vulkan, Metal, Direct3D, and OpenGL without backend-specific code.
- The QML engine owns the JS runtime and object lifecycle; understand ownership rules before mixing C++/Python and QML objects.

These foundations underpin every other concept in the book.

Chapter 2: QML Language Deep Dive

QML Elements and the Object Hierarchy

The Object Hierarchy: A Structured Approach to UI

QML applications are built around an **object hierarchy** — a tree of elements nested within one another, forming a parent-child structure. This hierarchy is both:

- **Visual**: it defines what you see on screen — how elements are positioned relative to one another and which elements contain which others.
- **Logical**: it defines the ownership and lifetime of objects — when a parent is destroyed, its children are automatically destroyed; when you interact with an element, you navigate up or down the tree to access related elements.

At the top of the hierarchy is typically an `ApplicationWindow` or a `Window`. Nested inside are containers (like `Column`, `Row`, `Rectangle`) and interactive elements (like `Button`, `TextField`, `Text`). Each nested level corresponds to one level of the QML file's indentation, making the hierarchy immediately visible in the source code.

This structural approach is one of QML's core strengths: the UI's organization is explicit and easy to reason about. Properties and signals flow through this hierarchy, and the hierarchy itself determines visual stacking order, input delivery, and resource cleanup.

What is a QML Element?

A QML element is an object instantiated from a type and described declaratively. Elements are the building blocks of every Qt Quick application —

they are concrete: when you write `Rectangle { color: "red" }`, you are creating a `Rectangle` element that actually exists at runtime.

Elements form a tree: a parent element can contain child elements, and children are positioned relative to their parent's coordinate system. When a parent is destroyed, its children are automatically destroyed too. This ownership model eliminates manual memory management.

```
ApplicationWindow {
    visible: true
    width: 400; height: 300

    Column {
        anchors.fill: parent
        spacing: 10

        Text {
            text: "Hello, QML"
        }

        Rectangle {
            width: 100; height: 50
            color: "steelblue"
        }
    }
}
```

Here, `ApplicationWindow` contains a `Column`, which contains a `Text` and a `Rectangle`. When the window is destroyed, so are all its descendants.

`Item`: The Root of the Visual Hierarchy

Every visual element in Qt Quick — `Rectangle`, `Text`, `Image`, `Button`, and so on — is either `Item` itself or a subclass of it. `Item` is invisible: it renders nothing. Its role is to:

- **Define a rectangular region** with position (`x`, `y`) and size (`width`, `height`)
- **Provide a coordinate origin** for child elements

- **Carry common visual properties**: `visible`, `opacity`, `z`, `enabled`, `clip`, `scale`, `rotation`
- **Be a container** — any element can have children

```
Item {
    width: 200; height: 200

    // This Rectangle is positioned at (10, 10) in Item's coordinate
space
    Rectangle {
        x: 10; y: 10
        width: 50; height: 50
        color: "red"
    }
}
```

When you need a container that does not draw anything and simply holds other elements, use `Item`. Chapter 3 covers geometry, anchoring, and the coordinate system in depth.

Item's Properties

`Item` provides a rich set of properties that all visual elements inherit:

Geometry and Positioning

Property	Type	Purpose
`x`, `y`	`real`	Position relative to the parent's top-left corner (in logical pixels)
`width`, `height`	`real`	Item dimensions (in logical pixels)
`implicitWidth`, `implicitHeight`	`real`	Suggested size based on the item's content (often set by layout systems)
`anchors`	`Anchors`	Declarative positioning constraints relative to sibling or parent edges

Property	Type	Purpose
`transform`	`list<Transform>`	Custom 2D/3D transformations (rotation, scale, matrix)

Visual Appearance

Property	Type	Purpose
`visible`	`bool`	If `false`, the item and all children are hidden (but still exist and consume memory)
`opacity`	`real`	Transparency: `0.0` (fully transparent) to `1.0` (fully opaque); inherited by children
`z`	`real`	Stacking order among siblings; higher `z` appears on top
`scale`	`real`	Multiplier for item size; `1.0` is normal, `2.0` is twice as large
`rotation`	`real`	Rotation in degrees (clockwise); pivot is the item's center
`clip`	`bool`	If `true`, children and content outside the item's boundaries are not rendered

Interaction and State

Property	Type	Purpose
`enabled`	`bool`	If `false`, the item does not receive input events (mouse, keyboard, touch); usually rendered with reduced opacity by controls
`focus`	`bool`	If `true`, the item has keyboard focus and receives key events; only one item in the window has focus at a time
`activeFocus`	`bool`	Read-only; `true` if this item has focus and is currently active (in focus chain)
`hoverEnabled`	`bool`	Enables the `HoverHandler` behavior; if `true`, the item can track mouse hover

Property	Type	Purpose
		(MouseArea property only)

Hierarchy and Structure

Property	Type	Purpose
`parent`	`Item`	The containing item, or `null` if this is a root item
`children`	`list<Item>`	Array of direct child items
`childrenRect`	`Rect`	Bounding rectangle of all children (useful for auto-sizing containers)

Signals Provided by Item

`Item` also emits several signals:

Signal	Purpose
`parentChanged()`	Emitted when the parent is reassigned
`childrenChanged()`	Emitted when children are added or removed
`visibleChanged()`	Emitted when `visible` is toggled
`focusChanged()`	Emitted when `focus` is gained or lost
`activeFocusChanged()`	Emitted when `activeFocus` changes

Example: responding to visibility changes:

```
Item {
    onVisibleChanged: {
        if (visible) {
            console.log("Item became visible")
        } else {
            console.log("Item became hidden")
        }
    }
}
```

Most of these properties have corresponding *change signals* — for instance, `opacityChanged()` is emitted when `opacity` is modified — allowing you to react to property changes via signal handlers or bindings.

Core Built-in Elements

Qt Quick provides a set of built-in visual elements — all inheriting from `Item` — for common UI tasks:

Element	Module	Purpose
`Rectangle`	`QtQuick`	Filled and/or bordered box with optional rounded corners
`Text`	`QtQuick`	Renders text using the system font engine
`Image`	`QtQuick`	Loads and displays raster (PNG, JPEG) or vector (SVG) images
`MouseArea`	`QtQuick`	Input region for mouse clicks, hover, and drag events
`Flickable`	`QtQuick`	Scrollable container with flick/momentum scrolling
`ListView`	`QtQuick`	Efficient list display, data-driven from a model
`GridView`	`QtQuick`	Grid layout, data-driven
`Repeater`	`QtQuick`	Creates multiple instances of a

Element	Module	Purpose
		delegate for each model item
`Loader`	`QtQuick`	Defers loading and instantiation of a child component
`ApplicationWindow`	`QtQuick.Controls`	Top-level window with menu bar, tool bar, and status bar slots
`Button`, `TextField`, `CheckBox`, `Slider`, …	`QtQuick.Controls`	Styled interactive controls with platform conventions

These form the vocabulary of UI construction. Each element has properties you can bind to, signals you can connect to, and behavior you can configure.

Non-Visual Elements

Not all QML types are visual elements that inherit from `Item`. Qt Quick also provides utility and infrastructure objects that do not appear on screen but support the UI:

Element	Module	Purpose
`Component`	`QtQuick`	Wraps a subtree without instantiating it; acts as a reusable template
`Timer`	`QtQuick`	Fires a signal at regular intervals; useful for animations and periodic tasks
`Connections`	`QtQuick`	Connects to signals from other objects, often used when the target is not the parent
`QtObject`	`QtQml`	Base QObject without visual properties; useful for defining property-only containers

Element	Module	Purpose
`Binding`	`QtQml`	Creates an explicit binding relationship; useful for restoring bindings programmatically

These objects are instantiated like any other element and can own children or emit signals, but they do not participate in the visual tree and have no geometry, visibility, or rendering.

`Component`: Defining Without Instantiating

`Component` wraps a QML subtree without creating it immediately. It acts as a reusable template:

```
Component {
    id: delegateTemplate
    Rectangle {
        width: 100; height: 30
        color: "lightgray"
        Text {
            anchors.centerIn: parent
            text: modelData
        }
    }
}

// Later: instantiate the component
Loader {
    sourceComponent: delegateTemplate
}

ListView {
    delegate: delegateTemplate
}
```

The component is not instantiated when the `Component` is created — only when you ask for it via `Loader`, `ListView.delegate`, `Repeater`, or by calling `createObject()` on the component reference. This allows deferred loading and reuse of the same template.

Best Practices: `.qml` Files vs. Inline `Component`

Use inline `Component` (inside a `.qml` file) for one-off templates:

- `ListView` delegates, `GridView` delegates, `Repeater` children
- Templates you only use in one place
- When you need to defer instantiation of a subtree

Use `.qml` files for reusable custom types:

The `.qml` file itself **is a component** — you don't need to wrap it in `Component`. When you create `MyButton.qml`, the file IS the template:

```
// MyButton.qml (correct)
Rectangle {
    property string label: "Click me"
    signal clicked()
    // ...
}

// Caller — instantiate directly
MyButton {
    label: "Greet"
    onClicked: doSomething()
}
```

Don't make the root of a `.qml` file a `Component`:

```
// MyButton.qml (incorrect — don't do this)
Component {
    Rectangle {
        property string label: "Click me"
        // ...
    }
}
```

This forces callers to write awkward code and defeats the purpose of `.qml` files as reusable types. The file itself is already the template.

Summary: Use `.qml` files for anything you want to reuse in multiple places or as a named type. Use inline `Component` for templates you define and use within a single file.

Inheritance and Type Extension

Every `.qml` file you create defines a new QML type. The type's name is the filename (minus `.qml`). The type extends whatever element appears at the root of the file.

```
// MyButton.qml
Rectangle {
    id: root
    width: 100
    height: 40
    color: "steelblue"
    radius: 6

    property string label: "Click me"
    signal clicked()

    Text {
        anchors.centerIn: parent
        text: root.label
        color: "white"
    }

    MouseArea {
        anchors.fill: parent
        onClicked: root.clicked()
    }
}
```

Now `MyButton` is a reusable type that extends `Rectangle`. Callers can use it like any built-in element:

```
// MainWindow.qml
ApplicationWindow {
    MyButton {
        x: 20; y: 20
        label: "Greet"
        onClicked: console.log("Greeted!")
    }
}
```

The caller can set properties inherited from `Rectangle` (`x`, `y`, `width`, `height`, `color`, `radius`) as well as properties you added (`label`). They can also

connect to signals you declared (`clicked`).

Each `.qml` file has its own **scope**: the `id root` inside `MyButton.qml` refers only to the root `Rectangle` in that file. If the caller has a separate `id root` in `MainWindow.qml` , they are entirely separate — no naming conflicts.

The Element Tree at Runtime

At any time, an element has:

- **`parent`** : a reference to its containing element, or `null` if it has no parent
- **`children`** : an array of all direct child elements
- **`width` , `height`** : the element's size in logical pixels

When you add a child element to an element, the parent takes ownership. When the parent is deleted (destroyed), all its children are deleted too. This automatic cleanup prevents memory leaks and makes reasoning about lifetime straightforward.

The element tree is also the **visual tree**: rendering walks this tree from root to leaves, drawing each element (or delegating to a scene graph node) in order.

Type System, Property Bindings, and the Binding Engine

The QML Type System

QML is a statically-typed language at the property level, even though JavaScript — a dynamically-typed language — is embedded within it. Every property in QML has a declared type. The engine enforces type compatibility at binding evaluation time and, in Qt 6 with `qmlsc` enabled, at compile time.

Primitive property types map directly to C++ equivalents:

QML type	C++ type	Notes
`int`	`int`	
`real`	`double`	Also accepts `float`
`bool`	`bool`	
`string`	`QString`	
`url`	`QUrl`	
`color`	`QColor`	Accepts `"#rrggbb"`, `"name"`, `Qt.rgba()`
`date`	`QDateTime`	
`var`	`QVariant`	Escape hatch — avoid where possible
`list<T>`	`QQmlListProperty<T>`	Typed list of QObject subclass

Value types (structs without QObject identity) such as `point`, `size`, `rect`, `font`, `vector2d`, `matrix4x4`, and `quaternion` are passed by value and do not participate in the QObject ownership model.

Object types — anything that derives from `QObject` — are reference types in QML. Assigning one to a property copies the reference, not the object.

Property Declarations

In QML components you define, you declare properties with `property`:

```
// In MyItem.qml
Item {
    property int count: 0
    property string label: "unnamed"
    property color accent: "#3daee9"
    property var payload   // untyped — use sparingly
    property list<Rectangle> boxes
}
```

Properties can have default values (the expression after the colon). That expression is itself a binding if it references other properties.

Modifiers: `readonly` and `required`

`readonly` **properties** are read-only from outside — they can only be assigned within the component that declares them:

```
Item {
    readonly property int itemCount: model.length
    readonly property bool isValid: title.length > 0 && age >= 0

    // These can be reassigned inside handlers / onCompleted
    Component.onCompleted: {
        // If itemCount were not readonly, this is where you'd re-
assign it
        console.log("Count:", itemCount)
    }
}
```

Use `readonly` when:

- A property is **computed** (derived from other properties) and should not be reassigned from outside.
- You want to **prevent accidental mutation** from parent components or external code.
- A property's value is **managed internally** but should be readable to others (e.g., `isLoading`, `currentIndex`).

The binding on a `readonly` property is active and will update as its dependencies change — the only restriction is that external code cannot assign a new value.

`required` **properties** are the inverse: they **must be supplied** by the caller before the component can be used. The component will not instantiate without them:

```
// ProfileCard.qml
Item {
    required property string userId
    required property string displayName
    required property color highlightColor

    Text {
        text: displayName
        color: highlightColor
    }
}


// Caller — using a Loader
Loader {
    source: "ProfileCard.qml"
    onLoaded: {
        item.userId = currentUser.id
        item.displayName = currentUser.name
        item.highlightColor: "#6c3aec"
    }
}

// Or using setSource() to supply them at load time:
profileLoader.setSource("ProfileCard.qml", {
    userId: currentUser.id,
    displayName: currentUser.name,
    highlightColor: "#6c3aec"
})
```

If a `required property` is not supplied, the engine emits a compile-time or runtime error — making the missing dependency explicit rather than silently using a default value.

Use `required` when:

- A component **cannot function** without certain data from its caller.
- You want to **enforce a contract**: the caller must think about what to pass in.
- You are building a **reusable component** that will be used in multiple contexts.

Common Pattern: `required readonly`

In components meant to display read-only data, combine both modifiers:

```
// ContactCard.qml
Item {
    required property var contact  // must be supplied
    readonly property string displayName: contact.firstName + " " +
contact.lastName
    readonly property url avatarUrl: "file:///avatars/" + contact.id
+ ".png"
}
```

The caller supplies `contact`; the component derives read-only display properties from it.

When to Use Each

Situation	Choose
Component is a generic utility (button, label, input field)	No modifiers — normal mutable properties
Component displays data that should not be changed from outside	`readonly` for the computed properties
Component needs mandatory input to work (profile card, detail view)	`required` for those inputs
Data should be supplied once and not changed	`required` + `readonly`
Property is optional with a sensible default	Normal `property` with a default value
Internal state you track but want to expose read-only	`readonly` binding to internal state

The Binding Engine

A binding is an expression — evaluated by the JavaScript engine — whose result is assigned to a property. What makes bindings special is that the QML engine

tracks which other properties the expression *reads* during evaluation. Those properties become *dependencies* of the binding. When any dependency changes, the engine marks the binding as dirty and schedules re-evaluation.

```
Rectangle {
    width: parent.width * 0.5   // depends on parent.width
    height: width                // depends on this.width
    color: highlighted ? "steelblue" : "transparent"  // depends on
highlighted
}
```

Change `parent.width`, and `width` re-evaluates. Change `width`, and `height` re-evaluates. Change `highlighted`, and `color` re-evaluates. None of this requires explicit wiring.

Binding Breakage

Bindings are broken — permanently — when a property is assigned a plain value in JavaScript:

```
Rectangle {
    width: parent.width * 0.5  // binding active

    MouseArea {
        anchors.fill: parent
        onClicked: parent.width = 200  // BREAKS the binding
    }
}
```

After the click, `width` is `200` and will never again react to `parent.width`. To restore a binding programmatically, use `Qt.binding()`:

```
onClicked: parent.width = Qt.binding(() => parent.width * 0.5)
```

This is a common source of subtle bugs. The rule: in JavaScript handlers, think carefully before assigning to a property that has or should have a binding.

Lazy vs. Eager Evaluation

The binding engine evaluates bindings *lazily* by default: a dirty binding is not re-evaluated until its value is read. If nothing reads the value (because the item is off-screen, or the property is not currently used), no re-evaluation occurs. This makes Qt Quick efficient for large item trees where only visible parts are active.

Signals, Signal Handlers, and the Event Loop

Signals

Every QML property implicitly has a *change signal* named `on<PropertyName>Changed`. You can also declare custom signals:

```
Item {
    property int count: 0
    signal thresholdReached(int value)
    signal resetRequested()
}
```

Signals are emitted with call syntax:

```
onClicked: {
    count++
    if (count >= 10) thresholdReached(count)
}
```

Signal Handlers

A signal handler is a special property whose name is `on<SignalName>` with the signal name capitalized. Its value is a JavaScript function body (or a function reference):

```
Button {
    text: "Reset"
    onClicked: root.resetRequested()
}
```

Handlers can also be attached with `Connections`, which is necessary when connecting to signals from an object that is not the handler's parent:

```
Connections {
    target: someModel
    function onDataChanged() {
        listView.positionViewAtBeginning()
    }
}
```

`Connections` also supports ignoring signals when a condition is false via the `enabled` property — useful for temporarily suspending reactions without disconnecting.

Connecting Signals to Slots Programmatically

QML objects expose a `.connect()` method on each signal, mirroring Qt's signal-slot connect syntax:

```
Component.onCompleted: {
    someObject.someSignal.connect(myHandler)
}

function myHandler(value) {
    console.log("Received:", value)
}
```

Disconnect with `.disconnect()`. Managing these connections carefully is important — connecting the same handler twice creates duplicate invocations, and forgetting to disconnect from destroyed objects causes crashes.

The Event Loop

Qt Quick operates on Qt's event loop (`QCoreApplication::exec()`). All QML JavaScript and signal processing happens on the main thread within the event loop. Qt Quick's animations and timers post events to the event loop, which the scene graph processes to drive frame rendering.

The event loop processes one event at a time. Long-running synchronous JavaScript will block the loop, causing dropped frames and an unresponsive UI. Any operation that may take more than a few milliseconds — network I/O, file parsing, heavy computation — must be pushed to a background thread or handled asynchronously. The `WorkerScript` element provides a lightweight way to run JavaScript on a worker thread:

```
WorkerScript {
    id: worker
    source: "worker.js"

    onMessage: function(msg) {
        // msg.result contains the worker's reply
        processResult(msg.result)
    }
}

onClicked: worker.sendMessage({ action: "compute", input:
largeDataSet })
```

For heavier work, delegate to C++ threads or Python async patterns (covered in later chapters).

Component Instantiation, Dynamic Object Creation, and Ownership Semantics

Static Instantiation

The most common form: a child element declared inside a parent's body is created when the parent is created and destroyed when the parent is destroyed.

```
Window {
    Rectangle {
        id: box
        // Created with Window, destroyed with Window
    }
}
```

Dynamic Instantiation via `Component`

`Component` wraps a subtree that is not immediately instantiated. Call `createObject()` to instantiate it at will:

```
Component {
    id: popupComp
    Popup {
        property string message
        // ...
    }
}

function showPopup(msg) {
    let p = popupComp.createObject(root, { message: msg })
    p.closed.connect(() => p.destroy())
}
```

`createObject(parent, properties)` returns the new instance. Pass `null` as parent for an engine-owned object — but be aware that without a parent, the GC manages lifetime and it may be collected sooner than expected.

`p.destroy()` schedules the object for deletion at the end of the current event loop iteration (not immediately). This is intentional: it prevents destroying an object that still has pending signal emissions.

`Qt.createQmlObject()`

For fully dynamic QML string creation — useful in prototyping but avoid in production:

```
let obj = Qt.createQmlObject(
    'import QtQuick; Rectangle { color: "red"; width: 50; height: 50
}',
    parent,
    "dynamic_rect"
)
```

The third argument is a source URL used in error messages. This bypasses the component cache and should not be used in hot paths.

Deferred Loading of QML Files

Deferred loading is the practice of delaying the parsing, compilation, and instantiation of a QML component until it is actually needed, rather than at application startup. It is one of the most impactful techniques for reducing time-to-first-frame and keeping memory usage proportional to what is visible.

Qt Quick provides three mechanisms at different levels of control:

Mechanism	When to use
`Loader`	Declarative, condition-driven show/hide of a sub-tree
`Component.incubateObject()`	Non-blocking instantiation with progress callbacks

Mechanism	When to use
`QQmlComponent` (C++)	Full control from C++; custom incubation strategies

Loader: The Idiomatic QML Approach

`Loader` is an invisible item that manages a single child component. It handles the full lifecycle: parsing the source file, instantiating the component, sizing itself to the loaded item, and destroying the item when it is no longer needed.

`source` vs `sourceComponent`

`source` takes a URL (string or `Qt.resolvedUrl()`); the engine fetches and parses the file on demand:

```
Loader {
    id: detailLoader
    source: "DetailPane.qml"
}
```

`sourceComponent` takes an already-parsed `Component` object. Use this when you have an inline `Component` and want `Loader` to manage instantiation:

```
Component {
    id: heavyComp
    HeavyWidget { }
}

Loader {
    sourceComponent: showWidget ? heavyComp : null
}
```

Assigning `null` to either property destroys the loaded item and releases the memory. This is the primary memory-management tool for optional UI regions.

Conditional Loading with `active`

`active` is the clean way to gate loading on a condition without constantly reassigning `source`:

```
Loader {
    active: userIsLoggedIn
    source: "UserDashboard.qml"
}
```

When `active` becomes `false`, the loaded item is destroyed. When it becomes `true` again, the item is re-created from scratch. If you want the item to persist in memory while hidden, use `visible: false` on the `Loader` instead of `active: false`.

Asynchronous Loading

By default, `Loader` blocks the main thread while it compiles and instantiates the component. For large or complex components this causes a visible frame drop. Setting `asynchronous: true` moves compilation off the main thread:

```
Loader {
    id: editorLoader
    source: "RichTextEditor.qml"
    asynchronous: true

    // Show a placeholder until the real component is ready
    Rectangle {
        anchors.fill: parent
        visible: editorLoader.status === Loader.Loading
        color: "#1e1e2e"

        BusyIndicator { anchors.centerIn: parent; running: true }
    }

    onLoaded: editorLoader.item.forceActiveFocus()
}
```

Asynchronous loading uses Qt's thread pool for compilation. Instantiation still happens on the main thread (because the item tree must be built there), but it

is interleaved with the event loop rather than blocking it — each instantiation step yields between frames.

> **Component cache interaction**: Once a QML file has been compiled, its bytecode is cached. A second `Loader` pointing at the same URL re-uses the cache and instantiates synchronously even with `asynchronous: true`, because there is no compilation work left to defer.

`status` and `progress`

`Loader.status` tracks the loading lifecycle:

Value	Meaning
`Loader.Null`	No source assigned
`Loader.Loading`	Fetch or compilation in progress
`Loader.Ready`	Item instantiated and available via `item`
`Loader.Error`	Load failed; check `Loader.sourceUrl` for diagnostics

`Loader.progress` is a `real` in `[0.0, 1.0]` — useful for remote URLs where the file must be fetched over the network before compilation:

```
Loader {
    id: remoteLoader
    source: "https://internal.example.com/ui/Dashboard.qml"
    asynchronous: true

    ProgressBar {
        anchors.fill: parent
        visible: remoteLoader.status === Loader.Loading
        value: remoteLoader.progress
    }
}
```

Accessing the Loaded Item

`Loader.item` is the instantiated root object of the loaded component. It is `null` while `status !== Loader.Ready`:

```
Button {
    text: "Focus editor"
    enabled: editorLoader.status === Loader.Ready
    onClicked: editorLoader.item.forceActiveFocus()
}
```

Bind to properties on the loaded item via `item`:

```
Text {
    text: editorLoader.item ? editorLoader.item.wordCount.toString()
: "—"
}
```

Passing Data Into a Loaded Item

With `setSource()`: Pass initial property values atomically before the component is fully instantiated, preventing a frame where properties are at their default values:

```
Loader {
    id: profileLoader
}

function openProfile(userId) {
    profileLoader.setSource("ProfileView.qml", { userId: userId })
}
```

`setSource()` is equivalent to assigning `source` then setting properties, but atomic — the component sees the correct values from its first binding evaluation.

With `required property`: The cleanest contract. Declare properties on the loaded component as `required`; the `Loader` must supply them:

```
// ProfileView.qml
Item {
    required property string userId
    required property string displayName
    // ...
}


// Caller
Loader {
    source: "ProfileView.qml"
    onLoaded: {
        item.userId = currentUser.id
        item.displayName = currentUser.name
    }
}
```

Or using `setSource()` to supply them at load time:

```
profileLoader.setSource("ProfileView.qml", {
    userId: currentUser.id,
    displayName: currentUser.name
})
```

If a `required property` is not supplied, the engine emits a runtime error — making the missing dependency explicit rather than silently producing a default value.

Sending data back out: Connect to signals on `item` after loading:

```
Loader {
    source: "EditDialog.qml"
    onLoaded: {
        item.accepted.connect((result) => applyEdit(result))
        item.rejected.connect(() => loader.active = false)
    }
}
```

Component Status Lifecycle

`Component` itself has a `status` property that mirrors `Loader.status`. This is relevant when you hold a `Component` object and instantiate it manually, or when a `Loader`'s `sourceComponent` is a dynamically created component:

```
Component {
    id: myComp
    // status is Component.Ready immediately for inline components
}
```

For components loaded from a URL via `Qt.createComponent()`:

```
Component.onCompleted: {
    let comp = Qt.createComponent("HeavyWidget.qml")
    if (comp.status === Component.Ready) {
        comp.createObject(root)
    } else if (comp.status === Component.Loading) {
        comp.statusChanged.connect(finishCreation)
    } else {
        console.error("Error:", comp.errorString())
    }
}

function finishCreation() {
    let comp = sender()   // not available directly — use a closure
    if (comp.status === Component.Ready)
        comp.createObject(root)
    else
        console.error(comp.errorString())
}
```

A cleaner pattern avoids the `statusChanged` dance entirely by always using `asynchronous: true` on a `Loader`, which handles the status polling internally. Reserve `Qt.createComponent()` for cases where you need a `Component` reference independently of a `Loader`.

`Component.status` value	Meaning
`Component.Null`	No source
`Component.Loading`	File fetch or compilation in progress

`Component.status` value	Meaning
`Component.Ready`	Safe to call `createObject()` or `incubateObject()`
`Component.Error`	Failed; `errorString()` contains details

Non-Blocking Instantiation with `incubateObject()`

`Component.createObject()` is synchronous — it blocks the main thread until the entire object tree is built. For components with very deep item hierarchies or expensive `Component.onCompleted` handlers, this can still cause a frame drop even when the *compilation* was asynchronous.

`incubateObject()` spreads instantiation across multiple event loop iterations, yielding between frames:

```
Component {
    id: dashboardComp
    Dashboard { }
}

function loadDashboard() {
    let incubator = dashboardComp.incubateObject(root, { userId:
session.userId })

    if (incubator.status === Component.Ready) {
        // Already done (cached, trivial component)
        onDashboardReady(incubator.object)
        return
    }

    incubator.onStatusChanged = function(status) {
        if (status === Component.Ready)
            onDashboardReady(incubator.object)
        else if (status === Component.Error)
            console.error(incubator.errorString())
    }
}

function onDashboardReady(dashboard) {
    // dashboard is fully instantiated
    mainStack.push(dashboard)
}
```

`incubateObject()` accepts the same arguments as `createObject()`. The returned *incubator* object has `status`, `object` (the item, once ready), and `onStatusChanged`. The incubator can also be given a custom completion callback as a third argument:

```
dashboardComp.incubateObject(root, { userId: session.userId },
Component.Asynchronous)
```

The third argument sets the incubation mode:

Mode	Behaviour
`Component.PreferSynchronous`	Synchronous if cached, async otherwise (default)

Mode	Behaviour
`Component.Asynchronous`	Always spread across event loop iterations

Patterns

Progressive Application Startup

Load the application shell immediately; defer feature areas until after the first frame:

```
ApplicationWindow {
    id: root
    visible: true

    AppShell {
        id: shell
        anchors.fill: parent
    }

    // Load the heavy initial screen after the window is shown
    Loader {
        id: initialScreen
        anchors.fill: parent
        asynchronous: true

        // Defer until the window has painted its first frame
        Timer {
            id: startupTimer
            interval: 0   // fires after the current event loop
iteration
            running: true
            onTriggered: initialScreen.source = "HomeScreen.qml"
        }
    }
}
```

An `interval: 0` timer fires as soon as the event loop is idle — after the first frame has been rendered — giving the user an immediate window before any heavy loading begins.

Lazy Tab Content

Pages in a `TabBar` + `StackLayout` arrangement are all instantiated at startup by default. Replace each page with a `Loader`:

```
StackLayout {
    currentIndex: tabBar.currentIndex

    // Tab 0 — loaded immediately (default landing page)
    HomeScreen { }

    // Tab 1 — loaded only when first selected
    Loader {
        active: tabBar.currentIndex >= 1
        source: "AnalyticsScreen.qml"
        asynchronous: true
    }

    // Tab 2 — same pattern
    Loader {
        active: tabBar.currentIndex >= 2
        source: "SettingsScreen.qml"
        asynchronous: true
    }
}
```

Once loaded, pages remain in memory (because `active` stays `true` after first activation). To aggressively reclaim memory, reset `active` to `false` when the tab is no longer selected — at the cost of re-instantiation on re-visit.

Error Handling

Always handle the `Loader.Error` state in production code:

```
Loader {
    id: pageLoader
    source: currentPage.source
    asynchronous: true

    states: [
        State {
            name: "error"
            when: pageLoader.status === Loader.Error
            PropertyChanges { target: errorOverlay; visible: true }
        }
    ]
}

Rectangle {
    id: errorOverlay
    visible: false
    anchors.fill: parent
    color: "#cc000000"

    Column {
        anchors.centerIn: parent
        spacing: 12
        Text { text: "Failed to load page"; color: "white";
font.pixelSize: 18 }
        Button {
            text: "Retry"
            onClicked: {
                pageLoader.source = ""
                pageLoader.source = currentPage.source
            }
        }
    }
}
```

JavaScript Integration: Scope, Closures, and When to Avoid JS

Scope in QML

QML defines several nested scopes that JavaScript expressions can access:

1. **Component scope**: Properties and IDs of items defined in the same `.qml` file
2. **Import scope**: Types and singletons brought in via `import` statements
3. **JS file scope**: Functions and variables defined in imported `.js` files

IDs in QML are effectively global within the component scope. They are not variables — they are compile-time names resolved by the QML engine to object references.

```
Item {
    id: root

    Rectangle {
        id: box
        width: 100
    }

    Text {
        text: box.width.toString()  // 'box' resolved at compile
time
    }
}
```

Closures and Memory Leaks

JavaScript closures in QML can create memory leaks by capturing references to QML objects, preventing garbage collection:

```
// Potential leak: closure captures 'heavyObject'
Component.onCompleted: {
    let heavyObject = someExpensiveModel
    timer.triggered.connect(() => {
        heavyObject.doSomething()  // heavyObject captured
    })
}
```

If `timer` outlives the intended scope of `heavyObject`, the closure keeps it alive. Prefer using `Connections` with a `target` and `enabled` property instead of manually connecting closures.

JavaScript Libraries

QML can import `.js` files as modules:

```
import "utils.js" as Utils

Text {
    text: Utils.formatDate(model.timestamp)
}
```

The `.js` file uses standard ECMAScript module-style exports (in Qt 6, `.mjs` files for ES modules are also supported). This is the right place for pure utility functions: date formatting, string manipulation, data transformation. Keep these functions stateless where possible.

When to Avoid JavaScript

JavaScript in QML is convenient but carries costs:

- **Binding overhead**: Complex binding expressions in JavaScript are slower than equivalent C++ property computations.
- **JIT cold starts**: The V4 JIT warms up over time; code executed rarely stays in interpreter mode.
- **Debugging difficulty**: JS errors in QML report file and line numbers but lack the stack-trace richness of C++ or Python.

- **Type safety gaps**: `var`-typed properties and untyped JS defeat the QML type system and prevent `qmlsc` optimization.

The guideline: use JavaScript in QML for UI logic only — reacting to input, driving animations, coordinating component state. Move computation, data transformation, and I/O to Python or C++ backends.

Summary

QML's language design is intentionally minimal on the declarative side and pragmatic on the JavaScript side. Its power lies in the binding engine, which makes reactive UI natural. The signal system gives clean inter-component communication without tight coupling. Deferred loading — through `Loader`, `incubateObject()`, and the component status lifecycle — is essential for keeping startup fast and memory proportional to what is actually on screen; `required property` declarations make the data contract between a caller and a loaded component explicit and enforced. JavaScript fills in the remaining gaps but should be kept thin — the more logic lives in QML JS, the harder the application is to test, profile, and maintain.

Chapter 3: Qt Quick Core Primitives

The Item Tree: Geometry, Anchoring, and the Coordinate System

`Item`: The Universal Base

Every visual element in Qt Quick is an `Item` or a subclass of it. `Item` itself is invisible — it has no fill, no border, no text. Its role is to define a rectangular region in the scene, provide a coordinate origin for children, and carry common properties shared by all visual types: `x`, `y`, `width`, `height`, `opacity`, `visible`, `enabled`, `clip`, `z`, and `scale`.

The item tree is a parent-child hierarchy. Children are positioned in their parent's coordinate space. A child at `x: 10, y: 10` is 10 logical pixels from the parent's top-left corner, regardless of where the parent sits in the window.

```
Item {
    id: container
    x: 50; y: 50
    width: 200; height: 200

    Rectangle {
        // This is at window coordinates (60, 60), but declared at
(10, 10)
        x: 10; y: 10
        width: 50; height: 50
        color: "steelblue"
    }
}
```

Logical Pixels and DPI

Qt Quick works in *logical pixels*, not physical pixels. On a high-DPI display, one logical pixel corresponds to multiple physical pixels. The mapping is controlled by the device pixel ratio (`Screen.devicePixelRatio`). Qt handles scaling automatically for most cases — images loaded via `Image` are selected at the appropriate resolution if `@2x` variants are available.

For custom rendering code (scene graph nodes, `Canvas`), you must account for the device pixel ratio manually by multiplying by `Screen.devicePixelRatio` when specifying physical pixel dimensions.

The Coordinate Transformation API

Converting coordinates between items is a frequent need:

```
// Map a point from item A's coordinate space to item B's
let pointInB = itemA.mapToItem(itemB, Qt.point(10, 10))

// Map to/from the scene (window) coordinate space
let scenePoint = myItem.mapToScene(Qt.point(0, 0))
let localPoint = myItem.mapFromScene(Qt.point(mouseX, mouseY))
```

These are essential when working with drag-and-drop, tooltips, or any feature where items at different levels of the hierarchy need to share coordinates.

Anchoring

Anchors are a declarative constraint system built into `Item` . They express geometric relationships between items' edges and centers without explicit `x` / `y` values:

```
Rectangle {
    id: toolbar
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    height: 48
}

Rectangle {
    id: content
    anchors.top: toolbar.bottom
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
}
```

Anchor shortcuts reduce verbosity:

```
anchors.fill: parent          // fills parent completely
anchors.centerIn: parent      // centers in parent
anchors.horizontalCenter: parent.horizontalCenter
anchors.verticalCenter: parent.verticalCenter
```

Anchors have margins:

```
anchors.fill: parent
anchors.margins: 16       // uniform margin on all sides
anchors.topMargin: 8      // override individual sides
```

Anchor Constraints and Layout Conflicts

Anchors and explicit `x` / `y` / `width` / `height` values conflict. If an item has `anchors.left` and `anchors.right`, assigning `width` directly is meaningless — the width is determined by the anchors. Qt will emit a warning.

Anchors and `Layout` attached properties also conflict. Items inside a `RowLayout` or `GridLayout` must use `Layout.preferredWidth` etc., not anchors for the axis managed by the layout engine.

Transforms, Opacity, and the Layer System

Transforms

The `transform` property accepts a list of `Transform` subclasses applied in order:

```
Rectangle {
    width: 100; height: 100
    transform: [
        Translate { x: 50 },
        Rotation { angle: 45; origin.x: 50; origin.y: 50 },
        Scale { xScale: 1.5; yScale: 1.5; origin.x: 50; origin.y: 50
}
    ]
}
```

Convenience properties on `Item` cover the common cases:

```
Item {
    rotation: 45        // degrees, around transform origin
    scale: 1.5          // uniform scale around transform origin
    transformOrigin: Item.Center   // or TopLeft, BottomRight, etc.
}
```

These convenience properties are shortcuts to the `transform` list and can be animated directly.

Opacity

`opacity` ranges from `0.0` (fully transparent) to `1.0` (fully opaque) and is inherited by the entire subtree. An item with `opacity: 0.5` makes all its children appear at half opacity, compounded with any opacity set on individual children.

Setting `visible: false` is more efficient than `opacity: 0` when the item does not need to be in the scene graph at all — invisible items are still traversed and

their children are still rendered (into an offscreen buffer), just not composited.

Layers

Applying `layer.enabled: true` to an item causes Qt Quick to render that item's subtree into an offscreen texture, then composite that texture into the scene. This enables several effects:

- **Correct opacity on overlapping children**: without a layer, each child composites independently, so overlapping semi-transparent children show through each other. With a layer, the whole subtree is flattened first.
- **`layer.effect`**: apply a `ShaderEffect` or `MultiEffect` to the layer texture.
- **Caching**: static content rendered once to a layer avoids re-rasterization each frame.

```
Rectangle {
    layer.enabled: true
    layer.effect: MultiEffect {
        blurEnabled: true
        blurMax: 32
        blur: 0.5
    }
}
```

Layers have a cost: they allocate GPU texture memory and require an additional render pass. Enable them only when the visual effect requires it.

Input Handling: MouseArea, TapHandler, PointerHandler, and Multi-touch

The Old Model: `MouseArea`

`MouseArea` is the original Qt Quick input handler. It covers a rectangular region and reports mouse button presses, releases, clicks, double-clicks, hover, and wheel events:

```
Rectangle {
    MouseArea {
        anchors.fill: parent
        onClicked: console.log("clicked at", mouseX, mouseY)
        onPressed: console.log("pressed, button:", pressedButtons)
        hoverEnabled: true
        onEntered: parent.color = "lightblue"
        onExited: parent.color = "white"
    }
}
```

`MouseArea` has a significant limitation: it participates in a press-grab model where the first `MouseArea` to accept a press grabs it exclusively. Composing multiple `MouseArea`s in a hierarchy — for example, a clickable item inside a `Flickable` — requires careful management of `propagateComposedEvents` and `preventStealing`.

The New Model: Pointer Handlers

Qt Quick 2's pointer handler framework (`TapHandler`, `DragHandler`, `PinchHandler`, `WheelHandler`, `HoverHandler`) addresses `MouseArea`'s composition limitations. Multiple handlers on the same item or hierarchy can cooperate without stealing from each other.

`TapHandler` : Replaces `MouseArea` for click detection:

```
Rectangle {
    TapHandler {
        onTapped: console.log("tapped, point:", eventPoint.position)
        onDoubleTapped: console.log("double tap")
        onLongPressed: console.log("long press")
        acceptedButtons: Qt.LeftButton | Qt.RightButton
    }
}
```

`DragHandler` : Replaces manual drag logic:

```
Rectangle {
    id: draggable
    DragHandler {
        onActiveChanged: if (!active) snapToGrid()
    }
}
```

`PinchHandler` : Pinch-to-zoom and rotate:

```
Image {
    PinchHandler {
        minimumScale: 0.5
        maximumScale: 4.0
        minimumRotation: -45
        maximumRotation: 45
    }
}
```

`HoverHandler` : Hover detection without the full MouseArea overhead:

```
HoverHandler {
    id: hover
    cursorShape: Qt.PointingHandCursor
}
Text { color: hover.hovered ? "blue" : "black" }
```

`WheelHandler` : Mouse wheel and trackpad scroll:

```
WheelHandler {
    onWheel: (event) => zoomFactor += event.angleDelta.y / 1200
}
```

Handler Grab and Exclusivity

Handlers use a grab model with three levels:

- **Passive grab**: The handler receives events but does not prevent others from receiving them.
- **Exclusive grab**: The handler takes sole control of the pointer point.
- **Cancel**: The handler relinquishes its grab.

By default, a handler that recognizes its gesture takes an exclusive grab. Use `grabPermissions` to tune this:

```
DragHandler {
    grabPermissions: PointerHandler.CanTakeOverFromItems |

PointerHandler.CanTakeOverFromHandlersOfDifferentType
}
```

Multi-touch

Pointer handlers are inherently multi-touch aware. Each active touch point is an `eventPoint`. `PinchHandler` tracks two points; a custom handler can track more via `minimumPointCount` and `maximumPointCount`. Mouse input is treated as a single-point touch, so the same handlers work for both.

Focus Management and Keyboard Input

The Focus System

Qt Quick has two orthogonal focus systems that interact:

Active focus (`activeFocus`): The item currently receiving keyboard events. Only one item in the entire scene has active focus at a time.

Focus scope (`FocusScope`): An item subtype that creates an isolated focus domain. Within a focus scope, one item holds the scope's *internal* focus. When the scope gains active focus, it delegates to its internally-focused item.

```
FocusScope {
    // This scope manages focus for its children independently
    TextField { id: nameField; focus: true }
    TextField { id: emailField }
}
```

Setting `focus: true` requests focus within the item's containing focus scope. If no focus scope contains the item, it requests focus in the root scope (the window).

`forceActiveFocus()` unconditionally gives active focus to an item, bypassing the scope hierarchy. Use it to implement "click to focus" or "tab to focus" manually.

Key Events

Items with active focus receive `Keys` attached property events:

```
Item {
    focus: true
    Keys.onPressed: (event) => {
        if (event.key === Qt.Key_Escape) {
            event.accepted = true
            closeDialog()
        }
    }
    Keys.onReturnPressed: submitForm()
}
```

Setting `event.accepted = true` prevents the event from propagating to the parent item. The propagation path goes from the focused item up to its ancestors, then to any `Keys.forwardTo` targets.

`Keys.forwardTo` allows routing key events to multiple targets:

```
Keys.forwardTo: [searchField, listView]
```

Shortcut

For application-wide key bindings that work regardless of focus, use `Shortcut`:

```
Shortcut {
    sequence: StandardKey.Save
    onActivated: document.save()
}

Shortcut {
    sequence: "Ctrl+Shift+F"
    onActivated: searchBar.open()
}
```

`Shortcut` items are typically placed at the window root or in the application shell.

`TextInput` and `TextEdit`

`TextInput` is a single-line input; `TextEdit` is multi-line. Both are low-level primitives — they have no visual decoration. Qt Quick Controls' `TextField` and `TextArea` wrap these with styling and accessibility support and should be preferred in production UIs.

Key properties for text input primitives:

```
TextInput {
    text: model.name
    onTextEdited: model.name = text  // fires only on user edits,
not programmatic changes
    onEditingFinished: validate()    // fires on Return/Enter or
focus loss
    validator: RegularExpressionValidator { regularExpression:
/^\d+$/ }
    echoMode: TextInput.Password     // for password fields
}
```

Summary

Qt Quick's item primitives form the foundation everything else is built on. The geometry system — logical pixels, the coordinate transformation API, and anchors — gives precise control over layout at the item level. Transforms and layers enable visual effects without leaving the retained scene graph model. The pointer handler framework supersedes `MouseArea` for composable, multi-touch-aware input. And the dual focus system — active focus plus focus scopes — gives principled keyboard routing across arbitrarily complex UI hierarchies. These are the tools you reach for when Qt Quick Controls' higher-level components are not sufficient, and understanding them deeply is essential for building custom components.

Chapter 4: Qt Quick Controls 2: Internals and Customization

Control Architecture: `background`, `contentItem`, `overlay`, and Delegates

Qt Quick Controls 2 (the `QtQuick.Controls` module) is not a thin wrapper around platform widgets. It is a fully custom, GPU-rendered control library that achieves platform-appropriate appearance through interchangeable *styles*, not through native widget APIs. This architecture gives complete rendering control at the cost of requiring explicit style implementations.

The Control Anatomy

Every control in the library derives from `Control`, which defines a three-layer compositing model:

```
┌──────────────────────────┐
│        overlay           │  z-order above everything
│  ┌──────────────────┐    │
│  │   contentItem    │    │  the control's primary content
│  └──────────────────┘    │
│      background          │  z-order below contentItem
└──────────────────────────┘
```

`background`: An arbitrary `Item` that renders behind the control's content. Typically a `Rectangle` with styling. The control resizes it to match its own dimensions.

`contentItem`: The primary visual content. For `Button`, it is a `Text` (the label). For `Slider`, it is the track and handle assembly. For `TextField`, it is the `TextInput`. The control positions `contentItem` inside its padding area.

`overlay` : Available on `Popup` and its subclasses. A fullscreen `Item` that sits above everything else in the window, used for modal backdrops.

Accessing these in a control subclass:

```
Button {
    id: btn

    background: Rectangle {
        color: btn.pressed ? "#1a6e9a" : btn.hovered ? "#2196a6" :
"#3daee9"
        radius: 4
        border.color: btn.visualFocus ? "#f3b730" : "transparent"
        border.width: 2
    }

    contentItem: Text {
        text: btn.text
        font: btn.font
        color: "white"
        horizontalAlignment: Text.AlignHCenter
        verticalAlignment: Text.AlignVCenter
    }
}
```

Padding and Insets

`Control` has two sizing systems that experienced developers often conflate:

Padding (`topPadding` , `bottomPadding` , `leftPadding` , `rightPadding`): Controls the space between the control's bounding box and the `contentItem` . The `contentItem` is sized and positioned within the padded area.

Insets (`topInset` , `bottomInset` , `leftInset` , `rightInset`): Controls the space between the control's bounding box and the `background` . Positive insets shrink the background relative to the control bounds; negative insets extend it beyond the bounds (useful for drop shadows that extend outside the control's layout area).

```
Button {
    topInset: -4; bottomInset: -4
    background: Rectangle {
        // Extends 4px above and below the button's layout rect
        radius: height / 2
        layer.enabled: true
        layer.effect: DropShadow { ... }
    }
}
```

Delegates

Controls that render lists of items — `ComboBox`, `SpinBox`, menus, etc. — expose a `delegate` property for the item template. The delegate is instantiated for each item in the model, and the control manages positioning. This is the same delegate pattern as `ListView`.

```
ComboBox {
    model: ["Alpha", "Beta", "Gamma"]

    delegate: ItemDelegate {
        width: control.width
        text: modelData
        highlighted: control.highlightedIndex === index
        font.bold: control.currentIndex === index
    }
}
```

The `QStyle`-less Approach and Platform Abstraction

Qt Widgets relies on `QStyle` — a C++ virtual interface with platform-specific implementations that paint widget chrome using QPainter. Qt Quick Controls 2 has no equivalent mechanism. Instead, it uses *QML-based styles*: a set of QML

files that provide the `background`, `contentItem`, and other visual pieces for each control type.

Available Styles

Qt ships several built-in styles:

Style	Character
`Basic`	Minimal, no platform pretense, suitable as a base for custom styles
`Fusion`	Cross-platform desktop look, closer to Qt Widgets' Fusion
`Material`	Google Material Design 2
`Universal`	Microsoft Fluent Design
`iOS`	Platform-appropriate for Apple mobile
`macOS`	Uses native-looking controls via QML
`Windows`	Platform-appropriate for Windows
`Imagine`	Image-asset-driven styling (nine-patch images)

Select the style at startup:

```
// C++
QQuickStyle::setStyle("Material");
```

```
# Python
from PySide6.QtQuickControls2 import QQuickStyle
QQuickStyle.setStyle("Material")
```

Or via environment variable: `QT_QUICK_CONTROLS_STYLE=Material`.

Style Fallback Chain

Qt Quick Controls uses a *fallback style* mechanism. If a control is not defined in the active style, the engine falls back to the `Basic` style implementation. This

means custom styles only need to define the controls they want to customize — undefined controls get `Basic` defaults automatically.

Building and Distributing a Custom Style from Scratch

A custom style is a directory of QML files, one per control type, following a specific naming and structure convention.

Directory Layout

```
MyStyle/
├── MyStyle.conf          (optional: configure fallback style)
├── Button.qml
├── TextField.qml
├── CheckBox.qml
├── ...
└── qmldir
```

The `qmldir` registers the style's QML module:

```
module MyStyle
Button 2.15 Button.qml
TextField 2.15 TextField.qml
```

A Minimal `Button.qml`

```
// MyStyle/Button.qml
import QtQuick
import QtQuick.Controls.impl    // for IconLabel, etc.
import QtQuick.Templates as T   // for T.Button base

T.Button {
    id: control

    implicitWidth: Math.max(
        implicitBackgroundWidth + leftInset + rightInset,
        implicitContentWidth + leftPadding + rightPadding
    )
    implicitHeight: Math.max(
        implicitBackgroundHeight + topInset + bottomInset,
        implicitContentHeight + topPadding + bottomPadding
    )

    padding: 12
    horizontalPadding: 16
    spacing: 8

    icon.width: 20
    icon.height: 20
    icon.color: control.enabled ? "#ffffff" : "#80ffffff"

    background: Rectangle {
        implicitWidth: 80
        implicitHeight: 36
        radius: 4
        color: control.pressed ? Qt.darker("#3daee9", 1.2)
             : control.hovered ? Qt.lighter("#3daee9", 1.1)
             : "#3daee9"
        opacity: control.enabled ? 1 : 0.6

        Behavior on color { ColorAnimation { duration: 80 } }
    }

    contentItem: IconLabel {
        spacing: control.spacing
        mirrored: control.mirrored
        display: control.display

        icon: control.icon
        text: control.text
        font: control.font
```

```
        color: control.enabled ? "#ffffff" : "#80ffffff"
    }
}
```

Note the use of `T.Button` from `QtQuick.Templates` — this is the non-visual base that provides all button logic (press detection, toggle behavior, checked state) without any visual opinion. Your `Button.qml` adds the visuals on top.

Registering the Style

Register your style module in `CMakeLists.txt`:

```
qt_add_qml_module(myapp
    URI MyApp
    QML_FILES Main.qml
)

qt_add_qml_module(mystyle
    URI MyStyle
    QML_FILES
        MyStyle/Button.qml
        MyStyle/TextField.qml
        # ...
)
```

Or in Python using `qmldir` and `QQmlEngine.addImportPath()`.

Style Configuration File

`MyStyle.conf` (an INI file) sets the fallback style and custom palette/font defaults:

```
[Controls]
Style=MyStyle

[MyStyle]
Variant=Normal

[Palette]
Window=#1e1e2e
WindowText=#cdd6f4
Base=#313244
```

Distributing as a Plugin

For sharing a style across applications, compile it as a QML plugin (`qt_add_plugin` with `PLUGIN_TYPE qmltypes` in CMake). The resulting `.so` / `.dll` is placed in the Qt Quick Controls style search path.

Common Customization Patterns

Overriding a Single Control Without a Full Style

For one-off customizations within a specific component, override the `background` or `contentItem` inline:

```
TextField {
    background: Rectangle {
        color: "transparent"
        border.color: parent.activeFocus ? "#3daee9" : "#555"
        border.width: parent.activeFocus ? 2 : 1
        radius: 4

        Behavior on border.color { ColorAnimation { duration: 100 }
}
    }
}
```

Palette Integration

All controls read colors from their `palette` property, which propagates down the item hierarchy. Set a palette at the `ApplicationWindow` level to theme the entire application:

```
ApplicationWindow {
    palette.button: "#3daee9"
    palette.buttonText: "#ffffff"
    palette.highlight: "#27ae60"
    palette.highlightedText: "#ffffff"
    // ...
}
```

Custom styles that use `palette.button` etc. in their `background` color expressions will respond automatically.

Font Propagation

Like `palette`, `font` propagates down the hierarchy. Set `font.family` and `font.pixelSize` at the root and all controls inherit it.

Summary

Qt Quick Controls 2 gives complete rendering flexibility through its background/contentItem/overlay architecture and QML-based styles. The `QStyle`-less approach means there is no platform style engine to fight — what you write in QML is exactly what renders. For serious applications, a custom style provides consistent branding across all controls without per-instance overrides. Understanding the `T.*` template types from `QtQuick.Templates` is essential: they are where all the control logic lives, and your visual implementations build on them.

Chapter 5: Layouts, Positioners, and Responsive Design

`RowLayout`, `ColumnLayout`, `GridLayout`: Sizing Policies and Attached Properties

Qt Quick provides two distinct mechanisms for arranging multiple items: *positioners* and *layouts*. They look similar in QML but operate on fundamentally different principles. Positioners (`Row` , `Column` , `Grid` , `Flow`) arrange items based on their *natural* (implicit) size and do not negotiate size with children. Layouts (`RowLayout` , `ColumnLayout` , `GridLayout`) participate in a constraint-solving pass that distributes available space among children according to sizing policies.

For any UI that needs to fill space, respond to window resizing, or align items across rows, layouts are the correct choice.

The Layout Engine

Qt Quick Layouts uses a two-pass algorithm per axis:

1. **Minimum / preferred / maximum size collection**: Each child's `Layout.minimumWidth` , `Layout.preferredWidth` , and `Layout.maximumWidth` (and height equivalents) are collected.
2. **Space distribution**: Available space is distributed. First, all items get their preferred size. If there is extra space, items with `Layout.fillWidth: true` share it proportionally via `Layout.horizontalStretchFactor` . If there is insufficient space, items are shrunk toward their minimum.

This mirrors the box model in CSS flexbox and Qt Widgets' `QSizePolicy` .

Attached Layout Properties

Children communicate their sizing preferences to the layout through *attached properties* on the `Layout` type:

```
RowLayout {
    spacing: 8

    TextField {
        Layout.fillWidth: true          // take all available
horizontal space
        Layout.minimumWidth: 100
        Layout.preferredWidth: 200
    }

    Button {
        text: "Search"
        Layout.preferredWidth: 80
        Layout.fillWidth: false         // fixed size
    }
}
```

Full set of attached properties:

Property	Effect
`Layout.fillWidth` / `Layout.fillHeight`	Expand to fill available space on that axis
`Layout.minimumWidth` / `Layout.minimumHeight`	Hard lower bound
`Layout.preferredWidth` / `Layout.preferredHeight`	Target size when space allows
`Layout.maximumWidth` / `Layout.maximumHeight`	Hard upper bound
`Layout.horizontalStretchFactor` / `Layout.verticalStretchFactor`	Relative weight when distributing surplus space among filling items
`Layout.alignment`	Alignment within the cell (`Qt.AlignLeft`, `Qt.AlignVCenter`, etc.)

Property	Effect
`Layout.margins`	Per-item margins (additive with `spacing`)
`Layout.columnSpan` / `Layout.rowSpan`	Span multiple cells in `GridLayout`
`Layout.column` / `Layout.row`	Explicit cell placement in `GridLayout`

Implicit Size Propagation

Layouts compute their own `implicitWidth` and `implicitHeight` from the children's preferred sizes plus spacing. This propagates up the item tree, allowing a layout inside a container to size the container correctly without hardcoded dimensions:

```
Dialog {
    // Dialog sizes itself to its contentItem's implicit size
    contentItem: ColumnLayout {
        spacing: 12
        // Each item's implicit size drives the column's implicit
size
        Label { text: "Username" }
        TextField { Layout.preferredWidth: 240 }
        Label { text: "Password" }
        TextField { echoMode: TextInput.Password;
Layout.preferredWidth: 240 }
    }
}
```

`RowLayout`

Items arranged horizontally. `spacing` sets the gap between items (not at the edges). By default, items are sized to their `implicitWidth` and aligned to the top of the layout's height.

```
RowLayout {
    anchors.fill: parent
    spacing: 4

    ToolButton { icon.source: "back.svg" }
    Label {
        text: currentPage.title
        Layout.fillWidth: true
        elide: Text.ElideRight
    }
    ToolButton { icon.source: "menu.svg" }
}
```

ColumnLayout

Items arranged vertically. The same principles apply on the vertical axis.

```
ColumnLayout {
    anchors.fill: parent
    spacing: 0

    HeaderBar { Layout.fillWidth: true }
    ScrollView {
        Layout.fillWidth: true
        Layout.fillHeight: true
        contentItem: DocumentView { }
    }
    StatusBar { Layout.fillWidth: true }
}
```

GridLayout

Items arranged in a two-dimensional grid. Column count is set via `columns` (or equivalently `rows` for row-dominant flow):

```
GridLayout {
    columns: 2
    rowSpacing: 8
    columnSpacing: 16

    Label { text: "First name" }
    TextField { Layout.fillWidth: true }

    Label { text: "Last name" }
    TextField { Layout.fillWidth: true }

    Label { text: "Email" }
    TextField {
        Layout.fillWidth: true
        Layout.columnSpan: 1
    }
}
```

Explicit placement via `Layout.row` and `Layout.column` is useful when items appear in a non-sequential or sparse grid:

```
GridLayout {
    columns: 3

    Label { text: "A"; Layout.row: 0; Layout.column: 0 }
    Label { text: "B"; Layout.row: 0; Layout.column: 2 }  // skip
column 1
    Label { text: "C"; Layout.row: 1; Layout.column: 0;
Layout.columnSpan: 3 }
}
```

Anchors vs. Layouts: Performance Trade-offs and When to Choose Each

When to Use Anchors

Anchors are evaluated by the binding engine — each anchor is a binding that updates when the referenced item's geometry changes. They are optimal for:

- **Two-item relationships**: "this item's right edge aligns with that item's left edge"
- **Filling a parent**: `anchors.fill: parent` is the fastest way to make an item fill its container
- **Centering**: `anchors.centerIn: parent`
- **Items not in a list or grid**: toolbars, overlays, decorations

```
// Correct use of anchors: simple parent-relative positioning
Rectangle {
    anchors.fill: parent
    anchors.margins: 16
}
```

When to Use Layouts

Layouts are evaluated by the Qt Quick Layouts engine, which runs a separate constraint-solving pass. They are optimal for:

- **Variable numbers of items**: adding or removing items from a layout is automatic
- **Space distribution**: items that share available space proportionally
- **Form layouts**: label-field pairs with aligned columns
- **Any sequence of items whose total size must fill the container**

The Mixing Problem

Do not mix anchors and layout attached properties on the same item. An item inside a layout should not have `anchors.left`, `anchors.right` etc. — the layout manages its position. Use `Layout.alignment` for alignment within a layout cell.

The combination that is valid and common: use anchors to fit a layout into its parent, then use layout properties inside:

```
ColumnLayout {
    anchors.fill: parent   // anchor the layout itself into its
parent
    spacing: 8

    TextField { Layout.fillWidth: true }   // layout properties on
children
    Button { text: "Submit" }
}
```

Performance Comparison

For simple cases (2–10 items, static structure), the difference is negligible. At scale:

- **Anchors**: O(n) binding evaluations per layout pass, each potentially triggering further bindings. Complex anchor webs can create cascading binding updates.
- **Layouts**: A single constraint pass runs once per layout invalidation, resolving all items in one sweep. More efficient for large, dynamic item sets.

For items inside `ListView` delegates, prefer neither anchors nor layouts for the few outermost items — instead use explicit `x` / `y` / `width` / `height` where possible, since delegates are instantiated in the thousands.

Adaptive UIs: `Screen`, `Window` Geometry, and Multi-DPI Handling

The `Screen` Attached Type

The `Screen` attached type provides information about the physical display:

```
Item {
    property real dpr: Screen.devicePixelRatio
    property bool isHighDPI: Screen.devicePixelRatio > 1.5
    property bool isLandscape: Screen.width > Screen.height
    property int screenWidth: Screen.width      // logical pixels
    property int screenHeight: Screen.height
    property real screenDPI: Screen.pixelDensity * 25.4  // ppi
}
```

Attach `Screen` to any `Item` to read the screen hosting that item — relevant in multi-monitor setups where different windows may be on screens with different DPRs.

Responsive Layout Strategies

Qt Quick does not have a built-in breakpoint system like CSS media queries, but one is straightforward to build:

```
ApplicationWindow {
    id: root

    // Breakpoints
    readonly property bool isCompact: width < 600
    readonly property bool isMedium: width >= 600 && width < 1200
    readonly property bool isExpanded: width >= 1200

    RowLayout {
        anchors.fill: parent

        // Navigation: sidebar on expanded, drawer on compact
        NavigationDrawer {
            visible: root.isExpanded
            Layout.preferredWidth: 256
            Layout.fillHeight: true
        }

        StackView {
            Layout.fillWidth: true
            Layout.fillHeight: true
        }
    }

    Drawer {
        visible: !root.isExpanded
        // Shown as overlay on compact screens
    }
}
```

Binding visual structure to `width` ranges reacts smoothly as the user resizes the window, which is appropriate for desktop. For mobile, where the screen size is fixed at launch, you can check once in `Component.onCompleted` and set a mode property.

Multi-DPI Image Handling

Qt automatically selects high-resolution image assets when they follow the `@2x` / `@3x` naming convention:

```
images/
├── icon.png          (1x — 32×32)
├── icon@2x.png       (2x — 64×64)
└── icon@3x.png       (3x — 96×96)
```

```
Image {
    source: "images/icon.png"  // Qt selects @2x or @3x
automatically
    width: 32; height: 32      // logical size; physical size varies
with DPR
}
```

For SVG assets, DPI scaling is automatic — SVGs are rasterized at the physical pixel size and always appear sharp.

Window Sizing and Constraints

```
ApplicationWindow {
    minimumWidth: 480
    minimumHeight: 320
    width: 1024
    height: 768

    // Save and restore window geometry
    Component.onCompleted: {
        if (settings.windowWidth > 0) {
            width = settings.windowWidth
            height = settings.windowHeight
            x = settings.windowX
            y = settings.windowY
        }
    }

    onClosing: {
        settings.windowWidth = width
        settings.windowHeight = height
        settings.windowX = x
        settings.windowY = y
    }
}
```

`Settings` from `Qt.labs.settings` persists values across sessions automatically.

`FontMetrics` and Density-Independent Sizing

Avoid hardcoding pixel values where the intent is to track text size. Use `FontMetrics` to size containers relative to font metrics:

```
FontMetrics {
    id: fm
    font: myFont
}

Rectangle {
    height: fm.height * 2    // two line heights, scales with font
size
}
```

This is essential for accessibility: users who increase system font sizes expect UI elements to accommodate the larger text.

Positioners: `Row`, `Column`, `Grid`, `Flow`

For completeness — positioners are appropriate when:

- Item sizes are fixed and known
- No space-filling is needed
- You want to avoid the overhead of the layout engine

```
// Flow wraps items like word-wrap text
Flow {
    width: parent.width
    spacing: 8

    Repeater {
        model: tags
        delegate: Chip { text: modelData }
    }
}
```

`Flow` is the positioner with no layout equivalent — it wraps items to the next line when horizontal space is exhausted, useful for tag clouds and chip groups.

Summary

The layouts system in Qt Quick is a full constraint-solving engine that distributes space, enforces minimums and maximums, and propagates implicit sizes up the hierarchy. Understanding the difference between anchors (binding-based, two-item relationships) and layouts (constraint-based, multi-item space distribution) determines which to reach for in any given situation. Combining them correctly — anchor layouts into parents, use layout attached properties inside — produces clean, maintainable geometry code. Layer responsive breakpoints on top of this to handle the full range of screen sizes and densities.

Chapter 6: Models, Views, and Delegates

`ListView`, `GridView`, `TableView`, and `TreeView` Internals

Qt Quick's model-view framework is built around a separation between data (the model), the scrollable container (the view), and the visual representation of each item (the delegate). This pattern scales from a 10-item dropdown to a table with millions of rows because views only instantiate delegates for *visible* items — this is the critical architectural property that makes Qt Quick views efficient.

`ListView`

`ListView` displays items in a linear sequence, either vertical (default) or horizontal. It virtualizes its content: only the delegates for visible items (plus a configurable buffer) exist in memory at any time.

```
ListView {
    id: view
    anchors.fill: parent
    model: contactModel
    spacing: 1

    delegate: ContactDelegate {
        width: view.width
    }

    ScrollBar.vertical: ScrollBar { }
}
```

Key properties:

Property	Description
`model`	The data source
`delegate`	The template for each item
`spacing`	Gap between delegates
`orientation`	`ListView.Vertical` (default) or `ListView.Horizontal`
`cacheBuffer`	Pixels of off-screen delegates to keep alive
`clip`	Clip content to the view bounds (almost always `true`)
`currentIndex`	The currently highlighted item
`highlight`	An item rendered behind the current delegate
`highlightFollowsCurrentItem`	Animate the highlight to follow `currentIndex`
`snapMode`	`ListView.NoSnap`, `ListView.SnapToItem`, `ListView.SnapOneItem`
`headerItem` / `footerItem`	Static items above/below the list content

Sections: `ListView` supports grouping items into sections with a header per group:

```
ListView {
    model: sortedContactModel
    section.property: "lastName"
    section.criteria: ViewSection.FirstCharacter
    section.delegate: SectionHeader {
        text: section
    }
}
```

GridView

`GridView` is a virtualized grid. All cells have the same size (`cellWidth × cellHeight`), and items flow left-to-right, top-to-bottom:

```
GridView {
    anchors.fill: parent
    model: photoModel
    cellWidth: 120
    cellHeight: 120

    delegate: PhotoThumbnail {
        width: GridView.view.cellWidth - 4
        height: GridView.view.cellHeight - 4
    }
}
```

For variable-size grids (masonry/waterfall layouts), `GridView` is not sufficient — implement a custom positioner or use a `Flow`-based approach with a custom model that pre-computes sizes.

TableView

`TableView` handles two-dimensional tabular data. It virtualizes both rows and columns, making it suitable for large datasets:

```
TableView {
    id: tableView
    anchors.fill: parent
    model: tableModel   // must implement QAbstractTableModel or be
a 2D ListModel

    columnWidthProvider: (column) => column === 0 ? 200 : 100
    rowHeightProvider: () => 40

    delegate: Rectangle {
        color: row % 2 === 0 ? "#f0f0f0" : "white"

        Text {
            anchors.centerIn: parent
            text: display   // standard Qt role name
        }
    }
}
```

`TableView` in Qt 6 supports:

- `selectionModel` with `ItemSelectionModel` for multi-selection
- `syncView` for synchronized scrolling between a header view and the data view
- `HorizontalHeaderView` and `VerticalHeaderView` as companion components

```
HorizontalHeaderView {
    syncView: tableView
    Layout.fillWidth: true
}
```

TreeView

`TreeView` (introduced in Qt 6.3) displays hierarchical data:

```
TreeView {
    anchors.fill: parent
    model: fileSystemModel

    delegate: TreeViewDelegate {
        // Built-in delegate handles expand/collapse indicators
        contentItem: Label {
            text: model.display
            leftPadding: depth * 20   // indent by depth
        }
    }
}
```

`TreeView` exposes `depth`, `isTreeNode`, `expanded`, and `hasChildren` from the `TreeView` attached type in delegates.

Internal Architecture: Virtualization and Reuse

All Qt Quick views use a *delegate recycling pool*. When a delegate scrolls out of the visible area, it is not destroyed — instead it is placed in a reuse pool. When a new item scrolls into view, a delegate is taken from the pool and populated with the new item's data. This dramatically reduces the cost of scrolling in large lists.

The reuse pool behavior is controlled by:

```
ListView {
    reuseItems: true   // default in Qt 6 (opt-in in Qt 5)
}
```

When `reuseItems` is true, delegates must handle being "reset" with new data. Any state in the delegate that is not driven by the model must be cleared in the `TableView.onReused` (or `ListView.onReused`) handler:

```
delegate: ItemDelegate {
    property bool selected: false   // local state, not from model

    TableView.onReused: selected = false   // reset on reuse
}
```

Delegate Lifecycle, Pooling, and Performance Pitfalls

Delegate Lifecycle

A delegate goes through these states:

1. **Created**: Instantiated from the delegate component, possibly from the reuse pool
2. **Active**: Visible on screen, receiving binding updates and events
3. **In pool**: Off-screen, binding updates suspended, `TableView.onPooled` signal emitted
4. **Reused**: Taken from pool for a new item, `TableView.onReused` signal emitted
5. **Destroyed**: Pool evicted due to memory pressure or view teardown

Performance Pitfalls

1. Heavy delegate instantiation

Every item entering the view triggers delegate creation (or reuse). An expensive `Component.onCompleted` — loading images, making network requests, building complex sub-hierarchies — causes jank during fast scrolling.

Mitigation: Keep delegates lightweight. Defer heavy work with `Loader` and `asynchronous: true`:

```
delegate: Item {
    Loader {
        asynchronous: true
        active: visible
        source: "HeavyContent.qml"
    }
}
```

2. Binding storms in delegates

Each delegate's bindings re-evaluate when the model notifies a data change. If a model emits `dataChanged` for the entire range on every update, all visible delegates re-evaluate all their bindings simultaneously.

Mitigation: Emit granular `dataChanged` signals with tight index ranges and specific role lists.

3. Anchors in delegates

Anchors in frequently-instantiated delegates add binding overhead that compounds with the number of visible items. Explicit width/height and `x` / `y` are faster.

4. Deep component hierarchies in delegates

Each level of nesting adds to instantiation cost. Flatten delegate trees where possible.

5. `clip: true` on delegates

Clipping requires a stencil operation per frame. Clip at the view level, not on individual delegates.

6. Image loading in delegates

Images load asynchronously but still block the main thread briefly during texture upload. Use `Image.asynchronous: true` and a placeholder:

```
Image {
    source: model.avatarUrl
    asynchronous: true
    fillMode: Image.PreserveAspectCrop

    Rectangle {
        anchors.fill: parent
        visible: parent.status !== Image.Ready
        color: "#e0e0e0"
    }
}
```

`DelegateChooser`, Section Headers, and Complex Delegate Composition

`DelegateChooser`

`DelegateChooser` (from `Qt.labs.qmlmodels`) selects between multiple delegate types based on a model role:

```
import Qt.labs.qmlmodels

ListView {
    model: messageModel

    delegate: DelegateChooser {
        role: "messageType"

        DelegateChoice {
            roleValue: "text"
            delegate: TextMessageDelegate { }
        }

        DelegateChoice {
            roleValue: "image"
            delegate: ImageMessageDelegate { }
        }

        DelegateChoice {
            roleValue: "system"
            delegate: SystemMessageDelegate { }
        }
    }
}
```

Each `DelegateChoice` provides a separate component. The chooser evaluates the `role` on each model item and instantiates the matching delegate. This is cleaner than a single delegate with `Loader` switching on role values.

Section Headers

Section headers divide a list into labeled groups. The `section.delegate` is instantiated once per group transition and sits between the last item of one group and the first item of the next:

```
ListView {
    model: sortedEmailModel
    section.property: "date"
    section.delegate: Item {
        width: parent.width
        height: 32

        Rectangle {
            anchors.fill: parent
            color: "#e8e8e8"
        }

        Text {
            anchors.centerIn: parent
            text: section
            font.bold: true
        }
    }
}
```

Section headers are part of the list's content flow and scroll with the list. For a *sticky* (pinned) header that stays visible as the corresponding section scrolls, you need to implement it manually:

```
ListView {
    id: listView

    // Sticky section header overlay
    Item {
        anchors.top: parent.top
        width: parent.width
        height: sectionHeaderHeight
        z: 2

        Text {
            anchors.fill: parent
            text: listView.currentSection
        }
    }
}
```

`currentSection` updates as the user scrolls past section boundaries.

Complex Delegate Composition

For chat applications, feed readers, or other complex list UIs, delegates often need:

- Variable height based on content
- Lazy-loaded sub-content
- Multiple interactive regions
- Animated state transitions

A well-structured complex delegate:

```
// MessageDelegate.qml
Item {
    id: root
    required property string authorName
    required property string bodyText
    required property string avatarUrl
    required property date timestamp
    required property bool isOwn

    width: ListView.view.width
    height: layout.implicitHeight + 24

    RowLayout {
        id: layout
        anchors {
            left: parent.left; right: parent.right
            top: parent.top
            margins: 12
        }
        spacing: 8
        layoutDirection: root.isOwn ? Qt.RightToLeft :
Qt.LeftToRight

        // Avatar
        RoundedImage {
            Layout.preferredWidth: 36
            Layout.preferredHeight: 36
            Layout.alignment: Qt.AlignTop
            source: root.avatarUrl
        }

        // Bubble
        MessageBubble {
            Layout.maximumWidth: root.width * 0.7
            Layout.fillWidth: true
            text: root.bodyText
            isOwn: root.isOwn
        }
    }
}
```

Using `required property` declarations (introduced in Qt 5.15) enforces that the view supplies model data, catching misconfigurations at runtime. In Qt 6, `qmlsc` can catch them at compile time.

Summary

Qt Quick's view components achieve scalability through delegate virtualization and recycling. The choice of view type — `ListView`, `GridView`, `TableView`, `TreeView` — depends on the dimensionality and structure of the data. Delegate performance is dominated by instantiation cost and binding depth; keeping delegates lean is the most impactful optimization. `DelegateChooser` cleanly handles heterogeneous lists, and section headers with `currentSection` implement grouped navigation. For complex delegates, the `required property` pattern makes model-delegate contracts explicit and type-safe.

Chapter 7: Navigation Patterns and Application Shell

`StackView`, `SwipeView`, `TabBar`, and `Drawer`

Qt Quick Controls provides several high-level navigation components that implement established UX patterns out of the box. Each is appropriate for different contexts; using the wrong one leads to non-idiomatic UIs that users find confusing.

`StackView`

`StackView` implements a navigation stack — the model behind back/forward navigation in mobile apps and wizard-style desktop dialogs. Pages are pushed onto the stack and popped from it; the view animates transitions between pages.

```
StackView {
    id: stack
    anchors.fill: parent
    initialItem: HomeScreen { }
}

// Elsewhere: push a new page
stack.push(Qt.resolvedUrl("DetailScreen.qml"), { itemId: model.id })

// Go back
stack.pop()

// Replace the current top (no back navigation)
stack.replace(Qt.resolvedUrl("SettingsScreen.qml"))
```

`push()` accepts:

- A URL string or `Qt.resolvedUrl()` result — the QML file is loaded on demand
- A `Component` — instantiated immediately
- An already-created `Item` — placed on the stack directly
- A `{properties}` map as the second argument — merged into the pushed item

The stack maintains page instances by default, allowing them to preserve their state when navigated back to. To release memory, use `StackView.ForceLoad` or manage instances explicitly.

Custom transitions: override `pushEnter`, `pushExit`, `popEnter`, `popExit` to define custom animations:

```
StackView {
    pushEnter: Transition {
        PropertyAnimation { property: "opacity"; from: 0; to: 1;
duration: 200 }
    }
    pushExit: Transition {
        PropertyAnimation { property: "opacity"; from: 1; to: 0;
duration: 200 }
    }
}
```

Depth and back navigation: `stack.depth` gives the number of items. The current item is `stack.currentItem`. Connecting the hardware back button (Android, etc.) to `stack.pop()` when `stack.depth > 1` implements standard back behavior:

```
ApplicationWindow {
    onClosing: (close) => {
        if (stack.depth > 1) {
            close.accepted = false
            stack.pop()
        }
    }
}
```

SwipeView

`SwipeView` presents pages that the user swipes through horizontally (or vertically). All pages are logically accessible; there is no concept of a stack — swiping moves through a flat index sequence.

```
SwipeView {
    id: swipeView
    anchors.fill: parent
    currentIndex: tabBar.currentIndex

    OnboardingPage1 { }
    OnboardingPage2 { }
    OnboardingPage3 { }
}

PageIndicator {
    anchors.bottom: swipeView.bottom
    anchors.horizontalCenter: parent.horizontalCenter
    count: swipeView.count
    currentIndex: swipeView.currentIndex
}
```

Bind `SwipeView.currentIndex` to a `TabBar` or `PageIndicator` for synchronized navigation. The binding is bidirectional via index synchronization:

```
TabBar {
    id: tabBar
    currentIndex: swipeView.currentIndex

    TabButton { text: "Home" }
    TabButton { text: "Feed" }
    TabButton { text: "Profile" }
}
```

`SwipeView` lazily instantiates pages as they are approached — only the current page and its immediate neighbors are active. Pages further away are unloaded if `SwipeView.isCurrentItem` / `SwipeView.isPreviousItem` / `SwipeView.isNextItem` is used as an `active` condition on a `Loader`.

`TabBar` and `TabButton`

`TabBar` is a navigation control that pairs with `SwipeView` or any indexed content container. It does not contain the content — it only manages selection:

```
ColumnLayout {
    TabBar {
        id: bar
        Layout.fillWidth: true

        TabButton { text: "Documents" }
        TabButton { text: "Recents" }
        TabButton { text: "Shared" }
    }

    StackLayout {
        Layout.fillWidth: true
        Layout.fillHeight: true
        currentIndex: bar.currentIndex

        DocumentsPane { }
        RecentsPane { }
        SharedPane { }
    }
}
```

`StackLayout` is a lightweight alternative to `SwipeView` for tab-driven content switching when swipe gestures are not desired. It does not animate transitions and keeps all children instantiated (unlike `SwipeView`'s lazy loading).

`Drawer`

`Drawer` implements a panel that slides in from the edge of the window. It is used for side navigation menus (the “hamburger menu” pattern) on smaller screens.

```
Drawer {
    id: drawer
    width: Math.min(parent.width * 0.8, 300)
    height: parent.height
    edge: Qt.LeftEdge    // or RightEdge, TopEdge, BottomEdge

    ColumnLayout {
        anchors.fill: parent
        spacing: 0

        // App header inside drawer
        DrawerHeader {
            Layout.fillWidth: true
        }

        // Navigation items
        Repeater {
            model: navItems
            delegate: NavigationItem {
                Layout.fillWidth: true
                onClicked: {
                    drawer.close()
                    stack.push(model.page)
                }
            }
        }

        Item { Layout.fillHeight: true }   // spacer

        // Footer items
        NavigationItem {
            Layout.fillWidth: true
            text: "Settings"
        }
    }
}
```

`Drawer` can be *modal* (blocks interaction with the rest of the UI while open) or *non-modal*. A non-modal drawer can be permanently visible on wider screens as a sidebar:

```
Drawer {
    modal: root.isCompact    // modal on small screens, non-modal on
large
    interactive: root.isCompact   // disable swipe gesture on large
screens
    position: root.isCompact ? 0.0 : 1.0   // always open on large
screens
}
```

Application-Wide Navigation State Machines

The Problem with Ad-hoc Navigation

As applications grow, ad-hoc navigation calls (`stack.push(...)` , `drawer.close()`) scattered through event handlers become difficult to reason about. Navigation logic entangles with presentation logic. Deep links, programmatic navigation, and back-stack management become error-prone.

The solution is to centralize navigation state in an explicit state machine or a navigation controller object.

A Navigation Controller Singleton

A QML singleton acting as a navigation controller provides a single API for all navigation actions:

```
// NavigationController.qml (registered as singleton)
pragma Singleton
import QtQuick

QtObject {
    id: controller

    // Signals that the shell responds to
    signal navigateTo(string page, var args)
    signal navigateBack()
    signal drawerOpen()
    signal drawerClose()

    // Public navigation API
    function goHome() { navigateTo("Home", {}) }
    function goToDetail(itemId) { navigateTo("Detail", { id: itemId
}) }
    function goToSettings() { navigateTo("Settings", {}) }
    function back() { navigateBack() }
}
```

The application shell connects to these signals and performs the actual navigation:

```
// ApplicationShell.qml
StackView {
    id: stack
    initialItem: homePage

    Connections {
        target: NavigationController
        function onNavigateTo(page, args) {
            stack.push(Qt.resolvedUrl(page + "Screen.qml"), args)
        }
        function onNavigateBack() {
            if (stack.depth > 1) stack.pop()
        }
    }
}
```

Individual screens call the controller, not the stack directly:

```
// DetailScreen.qml
Button {
    text: "Back"
    onClicked: NavigationController.back()
}
```

This decouples screens from the navigation implementation. The same controller API works whether the shell uses `StackView`, `SwipeView`, or a web-based router when targeting WebAssembly.

State Machine Navigation

For applications with complex conditional navigation rules — authentication flows, onboarding, feature flags — model navigation as a state machine using `StateGroup`:

```
QtObject {
    id: appState

    property string current: "splash"
    readonly property bool isAuthenticated: authService.isLoggedIn
    readonly property bool hasCompletedOnboarding:
settings.onboardingDone

    StateGroup {
        states: [
            State {
                name: "splash"
                when: appState.current === "splash"
            },
            State {
                name: "auth"
                when: !appState.isAuthenticated
            },
            State {
                name: "onboarding"
                when: appState.isAuthenticated &&
!appState.hasCompletedOnboarding
            },
            State {
                name: "main"
                when: appState.isAuthenticated &&
appState.hasCompletedOnboarding
            }
        ]
    }
}
```

The shell's `Loader` or `StackView` binds to `appState.current`:

```
Loader {
    source: {
        switch (appState.current) {
            case "splash": return "SplashScreen.qml"
            case "auth": return "AuthFlow.qml"
            case "onboarding": return "OnboardingFlow.qml"
            case "main": return "MainShell.qml"
        }
    }
}
```

Multi-Window and Multi-Screen Applications

Opening Additional Windows

Additional windows are created by instantiating `Window` or `ApplicationWindow`:

```
Window {
    id: secondWindow
    title: "Inspector"
    width: 400; height: 600
    visible: false
}

Button {
    text: "Open Inspector"
    onClicked: secondWindow.show()
}
```

Or dynamically:

```
Component {
    id: windowComponent
    Window {
        property var data
        title: "Detail: " + data.name
        onClosing: destroy()
    }
}

function openDetailWindow(item) {
    let w = windowComponent.createObject(null, { data: item })
    w.show()
}
```

Multi-Screen Awareness

`Screen.virtualX` and `Screen.virtualY` give a window's position in the virtual desktop coordinate space. To open a window on a specific screen:

```
Window {
    x: targetScreen.virtualX + (targetScreen.width - width) / 2
    y: targetScreen.virtualY + (targetScreen.height - height) / 2
}
```

`Qt.application.screens` lists all available screens as a model:

```
Repeater {
    model: Qt.application.screens
    delegate: Text {
        text: modelData.name + ": " + modelData.width + "×" +
modelData.height
    }
}
```

Sharing Data Between Windows

QML objects in different windows belong to the same QML engine and can reference each other directly if they share a common ancestor or are registered as singletons. The most robust pattern is a shared singleton model:

```
// SharedState singleton
pragma Singleton
QtObject {
    property var selectedItem: null
    signal selectionChanged()
}
```

Both windows bind to `SharedState.selectedItem`, and changes propagate automatically through the binding engine regardless of which window made the change.

Summary

Navigation in Qt Quick has well-established patterns for each use case: `StackView` for hierarchical navigation with back traversal, `SwipeView` +

`TabBar` for flat peer navigation, `Drawer` for hidden navigation panels. As complexity grows, centralizing navigation in a controller object or state machine prevents the spaghetti that emerges from scattered imperative navigation calls. Multi-window applications are straightforward — Qt's single-engine model means all windows share the same type registry, binding engine, and can reference shared singleton state naturally.

Chapter 8: Animation and State Machines

The Animation Framework: `Transition`, `Behavior`, `SequentialAnimation`

Qt Quick's animation framework is declarative: animations are described as properties of items or as standalone objects, not as procedural code that runs step by step. The framework integrates with the binding engine and the scene graph's frame clock so animations run at display refresh rate with smooth interpolation.

Property Animations

`PropertyAnimation` is the fundamental animation type. It interpolates a single property between two numeric values:

```
PropertyAnimation {
    id: fadeOut
    target: myRect
    property: "opacity"
    from: 1.0; to: 0.0
    duration: 300
    easing.type: Easing.InOutQuad
}

Button {
    text: "Fade"
    onClicked: fadeOut.start()
}
```

Specialized subclasses are preferred when the type is known:

Type	Interpolates
`NumberAnimation`	Numeric properties
`ColorAnimation`	`color` properties (interpolates in RGBA space)
`RotationAnimation`	Rotation angle with direction awareness (clockwise, shortest, counterclockwise)
`Vector3dAnimation`	3D vector properties
`AnchorAnimation`	Anchor changes within a state transition
`PathAnimation`	Moves an item along a `Path`
`SmoothedAnimation`	Tracks a value continuously, with velocity limits
`SpringAnimation`	Physics-based spring tracking

Behavior

A `Behavior` automatically animates every change to a property, regardless of what caused the change:

```
Rectangle {
    color: highlighted ? "#3daee9" : "#2d2d2d"

    Behavior on color {
        ColorAnimation { duration: 150; easing.type: Easing.OutCubic
}
    }
}
```

Now every assignment to `color` — from bindings, JavaScript, state changes — is intercepted and animated. This is the most declarative form of animation: you specify what to animate, not when.

Behaviors can be disabled temporarily:

```
Behavior on x {
    enabled: !instantMove
    NumberAnimation { duration: 300; easing.type: Easing.OutBack }
}
```

Composite Animations

`SequentialAnimation` : runs animations one after another:

```
SequentialAnimation {
    id: bounceIn

    NumberAnimation {
        target: item; property: "scale"
        from: 0; to: 1.1; duration: 200
        easing.type: Easing.OutCubic
    }
    NumberAnimation {
        target: item; property: "scale"
        from: 1.1; to: 1.0; duration: 100
        easing.type: Easing.InCubic
    }
}
```

`ParallelAnimation` : runs animations simultaneously:

```
ParallelAnimation {
    NumberAnimation { target: item; property: "opacity"; to: 1;
duration: 300 }
    NumberAnimation { target: item; property: "scale"; from: 0.8;
to: 1; duration: 300 }
}
```

`PauseAnimation` : introduces a delay within a sequence:

```
SequentialAnimation {
    NumberAnimation { target: item; property: "opacity"; to: 0;
duration: 200 }
    PauseAnimation { duration: 100 }
    NumberAnimation { target: item; property: "opacity"; to: 1;
duration: 200 }
}
```

`ScriptAction` : runs JavaScript at a specific point in a sequence:

```
SequentialAnimation {
    NumberAnimation { target: flyout; property: "y"; to: -
flyout.height; duration: 250 }
    ScriptAction { script: flyout.visible = false }
}
```

Easing Functions

The easing curve shapes the interpolation between start and end values. Qt provides the full set of Robert Penner's easing functions plus bezier curves:

```
easing.type: Easing.OutElastic
easing.amplitude: 1.0     // for elastic easings
easing.period: 0.3        // for elastic easings
easing.overshoot: 1.70158 // for back easings

// Custom bezier
easing.type: Easing.BezierSpline
easing.bezierCurve: [0.25, 0.1, 0.25, 1.0]  // control points
```

`Transition`

A `Transition` defines animations that run when an item changes state (see the `State` section below). It is the bridge between the state machine and the animation framework:

```
transitions: [
    Transition {
        from: "normal"; to: "expanded"
        NumberAnimation {
            properties: "width,height"
            duration: 300
            easing.type: Easing.OutCubic
        }
        ColorAnimation { duration: 300 }
    },
    Transition {
        from: "expanded"; to: "normal"
        NumberAnimation {
            properties: "width,height"
            duration: 200
        }
    }
]
```

Omitting `from` and `to` makes the transition run on any state change.

`SmoothedAnimation` and `SpringAnimation`

These are special animations that track a *changing target value* rather than animating to a fixed endpoint:

`SmoothedAnimation` : provides velocity-limited smooth tracking:

```
Rectangle {
    x: targetX   // targetX changes externally

    Behavior on x {
        SmoothedAnimation {
            velocity: 200   // max pixels per second
            easing.type: Easing.InOutQuad
        }
    }
}
```

`SpringAnimation` : simulates a spring-mass system:

```
Behavior on x {
    SpringAnimation {
        spring: 3
        damping: 0.2
        epsilon: 0.25
    }
}
```

These are ideal for following cursor position, tracking list scroll positions, or animating data-driven visualizations where the endpoint is continuously updated.

`State` and `StateGroup`: Modeling UI State Declaratively

`State`

A `State` represents a named configuration of property values that an item can be in. The `state` property of any `Item` holds the name of the current state:

```
Rectangle {
    id: panel
    width: 200; height: 48
    color: "#2d2d2d"

    property bool expanded: false
    state: expanded ? "expanded" : "normal"

    states: [
        State {
            name: "normal"
            PropertyChanges { target: panel; height: 48; color:
"#2d2d2d" }
            PropertyChanges { target: content; opacity: 0; visible:
false }
        },
        State {
            name: "expanded"
            PropertyChanges { target: panel; height: 240; color:
"#1e1e1e" }
            PropertyChanges { target: content; opacity: 1; visible:
true }
        }
    ]

    transitions: [
        Transition {
            NumberAnimation { properties: "height"; duration: 250;
easing.type: Easing.OutCubic }
            ColorAnimation { duration: 250 }
            NumberAnimation { target: content; property: "opacity";
duration: 200 }
        }
    ]
}
```

When `expanded` changes, the `state` binding re-evaluates, the new state's `PropertyChanges` are applied, and the matching `Transition` animates the change.

PropertyChanges

`PropertyChanges` records which properties to set when entering a state, and restores them when leaving (unless `restoreEntryValues: false` is set):

```
State {
    name: "selected"
    PropertyChanges { target: item; scale: 1.05; z: 10 }
    PropertyChanges { target: item.background; color: "#3daee9" }
    PropertyChanges { target: shadowLayer; visible: true }
}
```

`PropertyChanges` can also change signal handler implementations, effectively switching behavior by state:

```
State {
    name: "editMode"
    PropertyChanges {
        target: tapHandler
        onTapped: openEditor()
    }
}
```

StateGroup

`StateGroup` allows multiple independent state machines on one item:

```
Item {
    StateGroup {
        id: visibilityStates
        states: [
            State { name: "hidden"; PropertyChanges { target: root;
opacity: 0 } },
            State { name: "visible"; PropertyChanges { target: root;
opacity: 1 } }
        ]
    }

    StateGroup {
        id: selectionStates
        states: [
            State { name: "unselected" },
            State { name: "selected"; PropertyChanges { target:
highlight; visible: true } }
        ]
    }
}
```

Each `StateGroup` has its own `state` property and `transitions`. This avoids combinatorial explosion: two independent axes of state with 2 values each would require 4 combined states in a single state machine; two `StateGroup`s require only 2 + 2.

`AnimatedSprite`, `PathAnimation`, and Canvas-Based Effects

`AnimatedSprite`

`AnimatedSprite` plays a spritesheet animation — a sequence of frames laid out on a single image:

```
AnimatedSprite {
    id: character
    width: 64; height: 64
    source: "character.png"

    frameCount: 8
    frameWidth: 64
    frameHeight: 64
    frameRate: 12          // frames per second

    loops: Animation.Infinite
    running: true
}
```

For complex sprite state machines (idle → walk → run → jump), use `SpriteSequence` with named `Sprite` states and transitions between them:

```
SpriteSequence {
    sprites: [
        Sprite {
            name: "idle"
            source: "character.png"; frameCount: 4; frameX: 0
            frameDuration: 200
            to: { "walk": 1 }   // can transition to "walk"
        },
        Sprite {
            name: "walk"
            source: "character.png"; frameCount: 8; frameX: 256
            frameDuration: 80
            to: { "idle": 0.3, "run": 0.7 }
        }
    ]
}
```

Trigger transitions with `SpriteSequence.jumpTo("walk")`.

PathAnimation

`PathAnimation` moves an item along a `Path`:

```
Path {
    id: trackPath
    startX: 0; startY: 100
    PathCubic {
        x: 400; y: 100
        control1X: 100; control1Y: 0
        control2X: 300; control2Y: 200
    }
}

PathAnimation {
    id: moveAlong
    target: carIcon
    path: trackPath
    duration: 2000
    orientToPath: true      // rotate item to follow path tangent
    loops: Animation.Infinite
}
```

`orientToPath: true` automatically rotates the target item to face the direction of travel — essential for vehicles, arrows, and any directional element.

Canvas-Based Effects

`Canvas` provides a 2D drawing API equivalent to the HTML5 Canvas. It is the escape hatch for custom rendering that the scene graph primitives cannot express:

```
Canvas {
    width: 300; height: 300
    onPaint: {
        let ctx = getContext("2d")
        ctx.clearRect(0, 0, width, height)

        // Draw a filled arc (pie chart segment)
        ctx.beginPath()
        ctx.moveTo(width / 2, height / 2)
        ctx.arc(width / 2, height / 2, 120, 0, Math.PI * 2 *
progress)
        ctx.closePath()
        ctx.fillStyle = "#3daee9"
        ctx.fill()
    }

    property real progress: 0.65
    onProgressChanged: requestPaint()
}
```

`requestPaint()` schedules a repaint on the next frame. `Canvas` renders on the main thread and uploads its result to a GPU texture, making it less efficient than scene graph nodes for high-frequency updates. For frequently-updated custom rendering, prefer a C++ `QQuickItem` with a custom scene graph node (Chapter 17).

For static or infrequently-updated effects, `Canvas` is entirely appropriate and avoids the complexity of C++.

Animation Performance Considerations

Animating on the Render Thread

The Qt Quick render thread can run animations independently of the main thread for a subset of property animations. `NumberAnimation` on `x`, `y`, `width`, `height`, `opacity`, `rotation`, and `scale` can be promoted to render-thread animation when:

1. The property is on a scene graph node directly
2. No JavaScript runs in the animation
3. The item's `layer.enabled` is false

In practice, most simple `Behavior`-driven animations run smoothly even on the main thread because they are driven by the scene graph's frame clock and Qt Quick processes them before scene sync.

The sign of main-thread animation bottleneck is jank during concurrent JavaScript execution. The mitigation is to keep JS handlers brief and move computation to C++.

Transform vs. Position Animation

Animating `rotation` and `scale` (transforms) is cheaper than animating `width` and `height` (geometry), because transforms only update a matrix node in the scene graph. Geometry changes may cause layout recalculation and child repositioning. When implementing a "zoom in" effect, prefer `scale` animation over `width` / `height` animation.

Summary

Qt Quick's animation framework covers the full spectrum from simple property interpolation to physics-based tracking to complex coordinated sequences. The `Behavior` type provides the most declarative form — automatic animation of all property changes — while `Transition` bridges state machines and animations. States and `StateGroup` give a principled way to model multi-axis UI state without combinatorial explosion. `AnimatedSprite` and `PathAnimation` address game-adjacent use cases. `Canvas` serves as a flexible 2D drawing escape hatch. Together, these tools make it possible to implement sophisticated, fluid UIs entirely in QML.

Chapter 9: Qt for Python (PySide6): Architecture and Tooling

Shiboken Bindings: How C++ Types Are Exposed to Python

PySide6 is the official Qt for Python binding library, maintained by The Qt Company. It exposes the Qt C++ API to Python through *Shiboken6*, a binding generator that inspects Qt's C++ headers and generates Python extension modules (`.so` / `.pyd` files) containing the glue code between Python objects and Qt C++ objects.

The Binding Architecture

For each Qt C++ class, Shiboken generates:

1. **A Python type object**: The `PySide6.QtCore.QObject` type, for example, wraps `QObject*` .
2. **Method wrappers**: Each `Q_INVOKABLE` , public slot, and public member function becomes a callable Python function.
3. **Property descriptors**: `Q_PROPERTY` declarations become Python descriptors with `__get__` and `__set__` methods.
4. **Signal objects**: Each signal becomes a `PySide6.QtCore.Signal` descriptor that supports `connect()` , `disconnect()` , and `emit()` .

When you access a property in Python:

```
label = QLabel("Hello")
text = label.text()     # calls generated wrapper -> QLabel::text()
-> Python str
label.setText("World")  # Python str -> generated wrapper ->
QLabel::setText()
```

The generated code handles:

- Reference counting synchronization between Python's GC and Qt's parent-child ownership
- Type conversion between Python types and Qt types (`str ↔ QString` , `list ↔ QList` , etc.)
- Signal-slot connections across the Python/C++ boundary

Memory Model

PySide6 objects follow a dual ownership model:

Python-owned: The Python GC manages lifetime. When the Python reference count drops to zero, the object is deleted. This applies to Qt objects with no C++ parent.

C++-owned: Qt's parent-child mechanism owns the object. Even if Python holds no reference, the object lives until its C++ parent is deleted. PySide6 tracks these objects internally to prevent double deletion.

```
parent = QWidget()
child = QLabel(parent)  # C++ owned — parent=parent
del child               # Python reference gone, but QLabel lives
(owned by parent)
parent.show()           # QLabel still shows correctly
```

The dangerous case: a C++-owned object outliving the Python wrapper. If `parent` is deleted from C++ (e.g., by a C++ function), accessing the Python `parent` object raises `RuntimeError: Internal C++ object already deleted` .

Type System and Conversion

Shiboken's type conversion table for common types:

Python type	Qt type
`str`	`QString`
`bytes`	`QByteArray`
`int`	`int` , `qint64` , `uint` , enum values
`float`	`double` , `float`
`bool`	`bool`
`list[T]`	`QList<T>`
`dict`	`QVariantMap` / `QHash`
`tuple`	`QSize` , `QPoint` , `QRect` (for 2/4-tuples in some contexts)
`None`	`nullptr`
`object`	`QVariant`

For Qt value types without a natural Python equivalent (`QColor` , `QFont` , `QMatrix4x4`), PySide6 wraps them as Python objects. They behave like Python objects but are backed by the C++ value type.

The `QmlElement`, `QmlSingleton`, and `QmlAnonymous` Decorators

Qt 6.3+ introduced a class-decorator approach for registering Python classes with the QML type system, mirroring the C++ `QML_ELEMENT` macro family.

`QmlElement`

`@QmlElement` registers a Python `QObject` subclass as a QML type. The class name becomes the QML type name; the URI and version are taken from the enclosing `QmlModule` or specified in the decorator:

```python
from PySide6.QtCore import QObject, Property, Signal, Slot
from PySide6.QtQml import QmlElement

QML_IMPORT_NAME = "com.example.backend"
QML_IMPORT_MAJOR_VERSION = 1

@QmlElement
class Counter(QObject):
    valueChanged = Signal(int)

    def __init__(self, parent=None):
        super().__init__(parent)
        self._value = 0

    @Property(int, notify=valueChanged)
    def value(self):
        return self._value

    @value.setter
    def value(self, v):
        if self._value != v:
            self._value = v
            self.valueChanged.emit(v)

    @Slot()
    def increment(self):
        self.value += 1
```

In QML:

```
import com.example.backend

Counter {
    id: counter
    onValueChanged: console.log("count:", value)
}
Button {
    text: "Increment"
    onClicked: counter.increment()
}
```

QmlSingleton

`@QmlSingleton` combined with `@QmlElement` registers the type as a QML singleton — one shared instance for the entire engine:

```
@QmlElement
@QmlSingleton
class AppSettings(QObject):
    themeChanged = Signal(str)

    def __init__(self, parent=None):
        super().__init__(parent)
        self._theme = "dark"

    @Property(str, notify=themeChanged)
    def theme(self):
        return self._theme
```

QML accesses it without instantiating:

```
import com.example.backend

Rectangle {
    color: AppSettings.theme === "dark" ? "#1e1e2e" : "#eff1f5"
}
```

The singleton instance is created on first access by the engine.

QmlAnonymous

`@QmlAnonymous` registers a type that can be used as a property value type in QML but cannot be instantiated directly by QML code. Use this for types that only appear as the return value of a C++/Python function:

```python
@QmlAnonymous
class SearchResult(QObject):
    def __init__(self, title, url, parent=None):
        super().__init__(parent)
        self._title = title
        self._url = url

    @Property(str, constant=True)
    def title(self): return self._title

    @Property(str, constant=True)
    def url(self): return self._url
```

QML can hold a reference and read properties, but cannot write `SearchResult { }` in a QML file.

`QmlNamedElement`

When the QML name should differ from the Python class name:

```python
@QmlElement
@QmlNamedElement("FileIO")
class PythonFileHelper(QObject):
    ...
```

Project Structure, `pyproject.toml`, and the PySide6 Build System

Project Layout

A well-organized PySide6 project separates Python backend code from QML frontend code:

```
my_app/
├── pyproject.toml
├── main.py
├── backend/
│   ├── __init__.py
│   ├── models.py           # QAbstractItemModel subclasses
│   ├── settings.py         # QmlSingleton classes
│   └── services.py         # business logic, no Qt dependency
├── qml/
│   ├── qmldir
│   ├── Main.qml
│   ├── components/
│   │   ├── SearchBar.qml
│   │   └── ResultList.qml
│   └── screens/
│       ├── HomeScreen.qml
│       └── DetailScreen.qml
└── resources/
    ├── images/
    └── fonts/
```

pyproject.toml

PySide6 uses `pyproject.toml` both as a standard Python packaging manifest and as the configuration file for `pyside6-project` tools:

```toml
[build-system]
requires = ["setuptools", "wheel"]
build-backend = "setuptools.backends.legacy:build"

[project]
name = "my-app"
version = "1.0.0"
requires-python = ">=3.10"
dependencies = ["PySide6>=6.6"]

[tool.pyside6-project]
# Files to include in pyside6-deploy
input_file = "main.py"

[tool.pyside6-project.nuitka]
# Nuitka compilation flags
extra-nuitka-args = ["--follow-import-to=backend"]
```

Application Entry Point

```python
# main.py
import sys
from pathlib import Path

from PySide6.QtGui import QGuiApplication
from PySide6.QtQml import QQmlApplicationEngine
from PySide6.QtQuickControls2 import QQuickStyle

# Import all QmlElement-decorated modules to ensure registration
import backend.models      # noqa: F401
import backend.settings    # noqa: F401

def main():
    app = QGuiApplication(sys.argv)
    QQuickStyle.setStyle("Material")

    engine = QQmlApplicationEngine()
    engine.addImportPath(str(Path(__file__).parent / "qml"))
    engine.loadFromModule("MyApp", "Main")

    if not engine.rootObjects():
        sys.exit(-1)

    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

The `import backend.models` lines are necessary because Python only executes module-level code — including `@QmlElement` decorator registration — when the module is imported. Failing to import a module means its types are unknown to the QML engine.

QML Module Registration

In Qt 6, QML files are organized into *modules* with a URI. For PySide6, declare the module in `qmldir`:

```
# qml/qmldir
module MyApp
Main 1.0 Main.qml
```

Or use `QQmlEngine.addImportPath()` and follow the directory naming convention: a directory named after the module URI with dots replaced by slashes.

For typed QML modules (compatible with `qmlsc` ahead-of-time compilation), generate a `.pyi` stub and `qmltypes` file using `pyside6-genpyi` and register them.

PySide6 Developer Tooling

`pyside6-uic`

Converts Qt Designer `.ui` files to Python code. Less relevant for Qt Quick projects (which use `.qml`), but useful for hybrid applications using Qt Widgets for dialogs.

`pyside6-rcc`

Compiles Qt resource files (`.qrc`) into Python modules that embed images, fonts, and other assets into the application binary:

```
pyside6-rcc resources.qrc -o resources_rc.py
```

```
import resources_rc  # registers resources; must be imported before
use
```

`pyside6-designer`

Launches Qt Designer with PySide6 plugin support for visually editing `.ui` files.

`pyside6-genpyi`

Generates Python stub files (`.pyi`) for PySide6 modules. Enables IDE type-checking and autocompletion for Qt types in Python.

`pyside6-deploy`

Packages the application into a standalone executable using Nuitka (covered in depth in Chapter 13).

`pyside6-project`

Orchestrates builds, resource compilation, and type stub generation from `pyproject.toml` .

Summary

PySide6's Shiboken-generated bindings give Python code natural access to the full Qt API with automatic type conversion and memory management. The `@QmlElement` , `@QmlSingleton` , and `@QmlAnonymous` decorators provide a clean, Pythonic way to expose backend types to QML without manual registration boilerplate. A disciplined project structure — separating Python backend from QML frontend — keeps the codebase maintainable as it grows. The PySide6 toolchain covers the full development cycle from UI design through resource compilation to deployment.

Chapter 10: Exposing Python Objects to QML

Registering Types: `qmlRegisterType` vs. the Decorator Approach

There are two API generations for registering Python types with the QML engine: the classic `qmlRegisterType` family of functions (ported from C++) and the modern decorator-based approach introduced in PySide6 6.3. Both are supported, but new code should use decorators.

Classic Registration with `qmlRegisterType`

```
from PySide6.QtQml import qmlRegisterType, qmlRegisterSingletonType

# Register a creatable type
qmlRegisterType(Counter, "com.example.backend", 1, 0, "Counter")

# Register a singleton
def create_settings(engine):
    return AppSettings(engine)

qmlRegisterSingletonType(AppSettings, "com.example.backend", 1, 0,
                         "AppSettings", create_settings)
```

The `qmlRegisterType` family requires explicit URI, major version, minor version, and QML type name. This separates registration from the class definition and can lead to registration calls scattered across the codebase.

Decorator-Based Registration

```
QML_IMPORT_NAME = "com.example.backend"
QML_IMPORT_MAJOR_VERSION = 1

@QmlElement
class Counter(QObject):
    ...
```

The module-level `QML_IMPORT_NAME` and `QML_IMPORT_MAJOR_VERSION` variables define the namespace. All `@QmlElement`-decorated classes in the module are registered under this URI. This is DRY, co-located with the class definition, and plays well with tooling like `qmlsc`.

Choosing an Approach

Use decorators for all new code. Use `qmlRegisterType` only when:

- Integrating with older codebases that already use it
- The URI or version needs to differ per registration (unusual)
- The type is defined in a third-party library you cannot modify

---

`Property`, `Signal`, and `Slot` with Proper Type Annotations

`Signal`

PySide6 signals are class-level descriptors. They must be declared at the class level, not in `__init__`. Each signal instance carries its parameter type signature:

```python
from PySide6.QtCore import Signal

class DocumentModel(QObject):
    # Signals with typed parameters
    titleChanged = Signal(str)
    pageCountChanged = Signal(int)
    contentChanged = Signal()          # no parameters
    errorOccurred = Signal(str, int)   # message, error code

    # Overloaded signal (multiple signatures)
    dataReady = Signal([str], [bytes])
```

Emit a signal: call it like a function:

```python
self.titleChanged.emit("New Title")
self.errorOccurred.emit("File not found", 404)
```

Signal with `name` override (for signals whose Python-reserved name would conflict):

```python
destroyed_ = Signal(name="destroyed")  # avoid shadowing
QObject.destroyed
```

Property

`Property` creates a Qt property — a named value with a getter, optional setter, and optional notifier signal. QML binding expressions that read a property automatically subscribe to its `notify` signal:

```python
from PySide6.QtCore import Property, Signal

class UserProfile(QObject):
    nameChanged = Signal()
    ageChanged = Signal(int)

    def __init__(self, parent=None):
        super().__init__(parent)
        self._name = ""
        self._age = 0

    # Read-only property (no setter)
    @Property(str, notify=nameChanged)
    def name(self):
        return self._name

    @name.setter
    def name(self, value: str):
        if self._name != value:
            self._name = value
            self.nameChanged.emit()

    # Read-write property with typed notify signal
    @Property(int, notify=ageChanged)
    def age(self):
        return self._age

    @age.setter
    def age(self, value: int):
        if self._age != value:
            self._age = value
            self.ageChanged.emit(value)

    # Constant property (never changes — no notify needed)
    @Property(str, constant=True)
    def userId(self):
        return self._user_id
```

Property types map to QML types:

Python type annotation	QML type
`str`	`string`
`int`	`int`
`float`	`real`

Python type annotation	QML type
`bool`	`bool`
`list`	`var`
`QObject` subclass	object type
`QUrl`	`url`
`QColor`	`color`

The equality check before emitting is important. Without it, a setter that assigns the same value still emits the notify signal, causing unnecessary binding re-evaluation throughout QML.

`Slot`

`@Slot` marks a method as a Qt slot — callable from QML, connectable to signals, and visible to the meta-object system:

```
from PySide6.QtCore import Slot

class FileManager(QObject):

    @Slot()
    def openFile(self):
        ...

    @Slot(str)
    def openFilePath(self, path: str):
        ...

    @Slot(str, result=bool)
    def fileExists(self, path: str) -> bool:
        return Path(path).exists()

    @Slot(int, int, result=float)
    def divide(self, a: int, b: int) -> float:
        return a / b if b != 0 else 0.0
```

The `result` keyword specifies the return type. Without `result`, the method returns nothing to QML (even if it returns a Python value). With `result`, QML

can use the return value in expressions:

```
Button {
    onClicked: {
        if (fileManager.fileExists(pathField.text)) {
            fileManager.openFilePath(pathField.text)
        }
    }
}
```

Slots without `@Slot`: Python methods on `QObject` subclasses are still callable from QML through the meta-object dynamic invocation path, but they lack type information and may behave differently with overloads. Always use `@Slot` for methods intended for QML consumption.

Overloaded Slots

Use multiple `@Slot` decorators (stacked) for overloaded methods:

```
@Slot(str)
@Slot(str, int)
def search(self, query: str, maxResults: int = 20):
    ...
```

Context Properties vs. Registered Singletons: Trade-offs and Best Practices

There are three primary ways to make a Python object available in QML:

1. **Context properties** (`QQmlContext.setContextProperty`)
2. **Registered singletons** (`@QmlElement` + `@QmlSingleton`)
3. **Constructor injection** (passing objects as initial property values)

Context Properties

```
engine = QQmlApplicationEngine()
settings = AppSettings()
engine.rootContext().setContextProperty("appSettings", settings)
engine.load("Main.qml")
```

In QML, `appSettings` is available as a global name (no import needed):

```
Text { text: appSettings.theme }
```

Pros: Simple to set up; works with any `QObject` instance; no registration required.

Cons:

- Global namespace pollution — every context property is a global name in every QML file
- No IDE autocompletion or type checking (the name is a string, the type unknown to tooling)
- Breaks `qmlsc` ahead-of-time compilation (context properties are dynamic)
- `setContextProperty` must be called before `load()` — ordering dependency

Context properties are appropriate for rapid prototyping. Avoid them in production code.

Registered Singletons

```
@QmlElement
@QmlSingleton
class AppSettings(QObject):
    ...
```

In QML (with an explicit import):

```
import com.example.backend

Text { text: AppSettings.theme }
```

Pros:

- Explicit import makes dependencies visible
- Full IDE support: type-aware autocompletion
- Compatible with `qmlsc`
- Singleton instance is lazily created by the engine on first use
- Accessible from any QML file that imports the module

Cons:

- Requires the module to be imported in every file that uses it
- The singleton instance must not require constructor arguments (the engine creates it)

Singletons are the recommended pattern for application-wide services: settings, authentication state, theme, feature flags.

Constructor Injection

Pass objects as initial property values when loading the root component:

```
engine.setInitialProperties({
    "documentModel": document_model,
    "userSession": session
})
engine.loadFromModule("MyApp", "Main")
```

In QML:

```
// Main.qml
ApplicationWindow {
    required property DocumentModel documentModel
    required property UserSession userSession
}
```

Pros:

- Explicit dependencies — the root component declares what it needs via `required property`
- Testable: different instances can be injected for testing
- No global state

Cons:

- Only works for the root component (children must receive values via property propagation)
- Slightly more boilerplate

Constructor injection is the cleanest pattern for the root object. Combine with singletons for truly global services.

Propagating Objects Down the Tree

Once an object is available in the root component, pass it down via explicit property bindings:

```
// Main.qml
ApplicationWindow {
    required property DocumentModel documentModel

    StackView {
        initialItem: EditorScreen {
            model: documentModel   // passed down explicitly
        }
    }
}
```

Avoid reaching up the hierarchy with `parent.parent.model` — this creates invisible coupling. Explicit property passing makes the data flow visible and refactorable.

Practical Patterns

Two-Way Property Binding

When QML and Python both modify a property, implement change guards in both directions:

```
class SyncedValue(QObject):
    valueChanged = Signal(float)

    def __init__(self):
        super().__init__()
        self._value = 0.0
        self._updating = False

    @Property(float, notify=valueChanged)
    def value(self):
        return self._value

    @value.setter
    def value(self, v):
        if abs(self._value - v) > 1e-9 and not self._updating:
            self._updating = True
            self._value = v
            self.valueChanged.emit(v)
            self._updating = False
```

Returning Complex Objects from Slots

When a slot needs to return a complex object:

```
@Slot(str, result=QObject)
def findUser(self, userId: str) -> Optional[UserProfile]:
    user = self._db.get_user(userId)
    if user is None:
        return None
    profile = UserProfile(user, parent=self)  # parent self to avoid
GC
    return profile
```

The `parent=self` ensures the returned object lives at least as long as the backend object. QML's GC will not collect it prematurely.

Invoking QML from Python

Occasionally Python needs to call back into QML logic. Use `QMetaObject.invokeMethod` for cross-thread safety:

```
from PySide6.QtCore import QMetaObject, Qt

# Call a slot on the root QML object
QMetaObject.invokeMethod(
    engine.rootObjects()[0],
    "showNotification",
    Qt.QueuedConnection,
    Q_ARG(str, "Upload complete")
)
```

For simpler cases where you are already on the main thread, call methods directly on the root object:

```
root = engine.rootObjects()[0]
root.showNotification("Upload complete")
```

Summary

Exposing Python objects to QML requires understanding the interplay between Qt's meta-object system and PySide6's binding layer. `Property`, `Signal`, and `Slot` provide the vocabulary; equality guards in setters prevent notification storms; and `result` types in slots enable QML to use return values. For making objects available in QML, prefer registered singletons for global services and constructor injection for root component dependencies — avoiding context properties in production code. Explicit property propagation down the component tree keeps data flow visible and testable.

Chapter 11: Data Models in Python

Subclassing `QAbstractListModel`, `QAbstractTableModel`, and `QSortFilterProxyModel`

The Qt model-view framework defines a protocol — the `QAbstractItemModel` interface — that views use to query data, receive change notifications, and (optionally) submit edits. Implementing this protocol in Python gives you a model that any Qt Quick view can consume without the view knowing or caring whether the data comes from a Python list, a database, a network API, or a sensor stream.

The Model Protocol

A model must implement a small core interface:

Method	Purpose
`rowCount(parent)`	Returns number of rows
`data(index, role)`	Returns data for a cell at role
`roleNames()`	Returns a dict mapping role integers to QML role name bytes

For tables, also:

Method	Purpose
`columnCount(parent)`	Returns number of columns

For mutable models, additionally:

Method	Purpose
`setData(index, value, role)`	Applies an edit
`flags(index)`	Returns item flags (e.g., `Qt.ItemIsEditable`)

Change notification is communicated through signals defined on `QAbstractItemModel`:

Signal	When to emit
`dataChanged(topLeft, bottomRight, roles)`	Data changed in a range
`beginInsertRows(parent, first, last)` + `endInsertRows()`	Rows are being inserted
`beginRemoveRows(parent, first, last)` + `endRemoveRows()`	Rows are being removed
`beginResetModel()` + `endResetModel()`	Complete model restructuring

`QAbstractListModel`

A flat list model. `parent` in `rowCount()` and `data()` is always `QModelIndex()` (the root) for list models — pass it through but do not use it:

```python
from PySide6.QtCore import (
    QAbstractListModel, QModelIndex, Qt, Signal
)

class ContactModel(QAbstractListModel):
    # Custom roles start above Qt.UserRole (256)
    NameRole = Qt.UserRole + 1
    EmailRole = Qt.UserRole + 2
    AvatarRole = Qt.UserRole + 3

    def __init__(self, parent=None):
        super().__init__(parent)
        self._contacts: list[dict] = []

    # --- Required interface ---

    def rowCount(self, parent=QModelIndex()) -> int:
        if parent.isValid():
            return 0  # flat list: no children
        return len(self._contacts)

    def data(self, index: QModelIndex, role: int = Qt.DisplayRole):
        if not index.isValid() or not (0 <= index.row() <
len(self._contacts)):
            return None
        contact = self._contacts[index.row()]
        match role:
            case self.NameRole:  return contact["name"]
            case self.EmailRole: return contact["email"]
            case self.AvatarRole: return contact["avatar_url"]
            case Qt.DisplayRole: return contact["name"]
            case _: return None

    def roleNames(self) -> dict[int, bytes]:
        return {
            self.NameRole:   b"name",
            self.EmailRole:  b"email",
            self.AvatarRole: b"avatar",
        }

    # --- Mutation API ---

    def appendContact(self, name: str, email: str, avatar: str =
""):
        self.beginInsertRows(QModelIndex(), len(self._contacts),
len(self._contacts))
        self._contacts.append({"name": name, "email": email,
```

```
"avatar_url": avatar})
        self.endInsertRows()

    def removeContact(self, row: int):
        if not (0 <= row < len(self._contacts)):
            return
        self.beginRemoveRows(QModelIndex(), row, row)
        del self._contacts[row]
        self.endRemoveRows()

    def updateEmail(self, row: int, email: str):
        if not (0 <= row < len(self._contacts)):
            return
        self._contacts[row]["email"] = email
        index = self.index(row, 0)
        self.dataChanged.emit(index, index, [self.EmailRole])
```

In QML:

```
ListView {
    model: contactModel   // Python ContactModel instance
    delegate: ContactDelegate {
        required property string name
        required property string email
        required property string avatar
    }
}
```

Role names defined in `roleNames()` become accessible as named properties in delegates. The `b"name"` bytes are the QML property names.

QAbstractTableModel

A two-dimensional model. Additionally implement `columnCount()`:

```
class DataFrameModel(QAbstractTableModel):
    def __init__(self, df, parent=None):
        super().__init__(parent)
        self._df = df

    def rowCount(self, parent=QModelIndex()) -> int:
        return 0 if parent.isValid() else len(self._df)

    def columnCount(self, parent=QModelIndex()) -> int:
        return 0 if parent.isValid() else len(self._df.columns)

    def data(self, index: QModelIndex, role=Qt.DisplayRole):
        if not index.isValid():
            return None
        if role == Qt.DisplayRole:
            value = self._df.iat[index.row(), index.column()]
            return str(value)
        return None

    def headerData(self, section: int, orientation: Qt.Orientation,
role=Qt.DisplayRole):
        if role != Qt.DisplayRole:
            return None
        if orientation == Qt.Horizontal:
            return str(self._df.columns[section])
        return str(section + 1)
```

This wraps a pandas `DataFrame` as a Qt table model. The `TableView` component in QML will query `rowCount`, `columnCount`, `data`, and `headerData` automatically.

Batch Updates with `beginResetModel`

When the underlying data is completely replaced (e.g., a new query result), use `beginResetModel` / `endResetModel`:

```
def loadData(self, new_data: list[dict]):
    self.beginResetModel()
    self._contacts = new_data
    self.endResetModel()
```

The view discards all cached delegates and rebuilds from scratch. This is appropriate for complete replacements but heavy for incremental changes — prefer `beginInsertRows` / `beginRemoveRows` / `dataChanged` for incremental mutations.

Thread-Safe Model Mutation and `QMetaObject.invokeMethod`

The Threading Constraint

`QAbstractItemModel` and all Qt Quick views run on the main thread. You must never call `beginInsertRows()`, `endInsertRows()`, `dataChanged.emit()`, or any other model method from a background thread. Doing so corrupts the view's internal state and causes crashes.

Background threads can safely:

- Read data (if the read is protected by a lock)
- Compute results
- Store results in a thread-safe queue

They must not:

- Call any model mutation method directly
- Access any `QObject` on the main thread

Pattern 1: `QMetaObject.invokeMethod` with `QueuedConnection`

Post a method call to the main thread's event loop from a background thread:

```
import threading
from PySide6.QtCore import QMetaObject, Qt, Q_ARG

class LiveDataModel(QAbstractListModel):
    ...

    def startStreaming(self):
        thread = threading.Thread(target=self._stream_worker,
daemon=True)
        thread.start()

    def _stream_worker(self):
        for packet in data_source.stream():
            QMetaObject.invokeMethod(
                self,
                "_appendPacket",
                Qt.QueuedConnection,
                Q_ARG("QVariant", packet)
            )

    @Slot("QVariant")
    def _appendPacket(self, packet):
        # Now on the main thread
        self.beginInsertRows(QModelIndex(), len(self._data),
len(self._data))
        self._data.append(packet)
        self.endInsertRows()
```

Pattern 2: Signal from Worker Thread

Emit a signal from the worker thread connected with a `QueuedConnection`. Qt automatically marshals the signal emission to the receiver's thread:

```
class DataWorker(QObject):
    newDataAvailable = Signal(list)

    def run(self):
        # In a QThread
        while True:
            batch = self._fetch_batch()
            self.newDataAvailable.emit(batch)

class LiveModel(QAbstractListModel):
    def __init__(self):
        super().__init__()
        self._worker = DataWorker()
        self._thread = QThread(self)
        self._worker.moveToThread(self._thread)
        self._worker.newDataAvailable.connect(
            self._onNewData, Qt.QueuedConnection
        )
        self._thread.start()
        QMetaObject.invokeMethod(self._worker, "run",
Qt.QueuedConnection)

    @Slot(list)
    def _onNewData(self, batch: list):
        # On the main thread due to QueuedConnection
        self.beginInsertRows(QModelIndex(), len(self._data),
len(self._data) + len(batch) - 1)
        self._data.extend(batch)
        self.endInsertRows()
```

`moveToThread()` changes the *thread affinity* of a `QObject`. Its slots are subsequently invoked on the target thread. This is the canonical Qt threading pattern.

Async Data Loading: Integrating `asyncio` with the Qt Event Loop

Python's `asyncio` event loop and Qt's event loop are both single-threaded event loops, but they are different loops. Running them together requires bridging.

qasync

The `qasync` library (installable via pip) integrates `asyncio` with Qt's event loop by running the asyncio loop as a Qt event source:

```
import asyncio
import qasync
from PySide6.QtWidgets import QApplication

async def main():
    app = QApplication.instance()
    # ... setup engine, backend ...
    await asyncio.sleep(0)  # yield to Qt event loop
    # Application runs here via qasync

if __name__ == "__main__":
    app = QApplication(sys.argv)
    loop = qasync.QEventLoop(app)
    asyncio.set_event_loop(loop)

    with loop:
        loop.run_until_complete(main())
```

With `qasync`, Python async functions can await I/O without blocking the Qt event loop:

```
class ApiClient(QObject):
    dataReady = Signal(list)

    async def fetch_items(self):
        async with aiohttp.ClientSession() as session:
            async with session.get("https://api.example.com/items")
as resp:
                data = await resp.json()
        self.dataReady.emit(data)

    @Slot()
    def refreshData(self):
        asyncio.ensure_future(self.fetch_items())
```

Using `QThreadPool` and `QRunnable`

For CPU-bound work, Python's asyncio gives no parallelism benefit (due to the GIL). Use Qt's thread pool instead:

```
from PySide6.QtCore import QRunnable, QThreadPool, Slot, Signal,
QObject

class WorkerSignals(QObject):
    finished = Signal(object)
    error = Signal(str)

class ComputeWorker(QRunnable):
    def __init__(self, fn, *args, **kwargs):
        super().__init__()
        self.fn = fn
        self.args = args
        self.kwargs = kwargs
        self.signals = WorkerSignals()

    @Slot()
    def run(self):
        try:
            result = self.fn(*self.args, **self.kwargs)
            self.signals.finished.emit(result)
        except Exception as e:
            self.signals.error.emit(str(e))

# Usage
def heavy_computation(data):
    # CPU-intensive work
    return process(data)

worker = ComputeWorker(heavy_computation, large_dataset)
worker.signals.finished.connect(model.loadData)
worker.signals.error.connect(errorHandler.showError)
QThreadPool.globalInstance().start(worker)
```

`QRunnable` is not a `QObject` (no signals), which is why `WorkerSignals` is a separate `QObject`. The `finished` signal fires on the main thread (via `QueuedConnection` default for cross-thread signals), making it safe to call model methods in the slot.

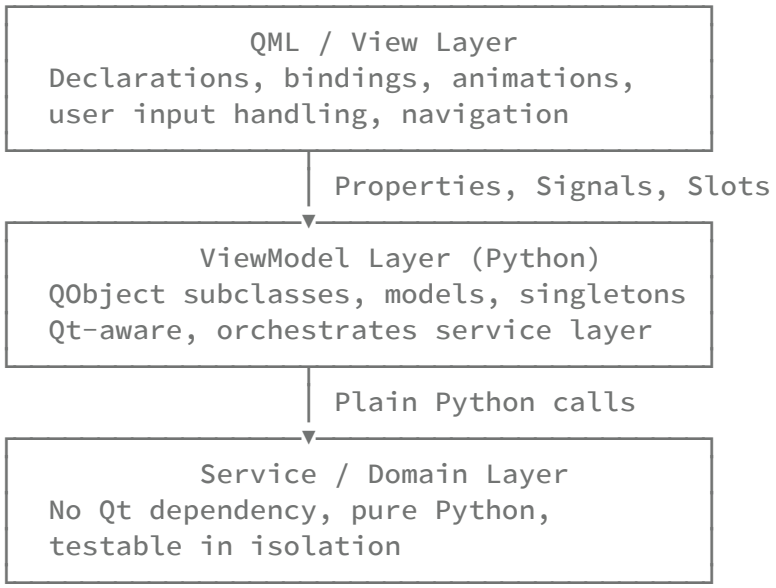
Summary

`QAbstractListModel` and `QAbstractTableModel` are the contracts that Qt Quick views consume. Implementing them correctly — with `roleNames()` for QML property names, proper begin/end bracketing for mutations, and granular `dataChanged` emissions — gives views the information they need to update efficiently. Threading is the critical constraint: all model mutations must happen on the main thread. The `QMetaObject.invokeMethod` / `QueuedConnection` pattern and the `moveToThread` + signal pattern are the two standard ways to safely deliver background thread results to the main thread model. For async I/O, `qasync` bridges asyncio and Qt naturally; for CPU-bound work, `QThreadPool + QRunnable` bypasses the GIL limitation.

Chapter 12: Python Backend Patterns

Business Logic Separation: Keeping QML as Pure View Layer

The most sustainable architecture for a PySide6/QML application separates concerns into three layers:



The Service Layer

The service layer contains domain logic with no Qt dependency. It can be unit-tested with pytest without a `QApplication`, without an event loop, and without a display:

```python
# services/search_service.py
from dataclasses import dataclass
from typing import Protocol

@dataclass
class SearchResult:
    title: str
    url: str
    snippet: str
    score: float

class SearchBackend(Protocol):
    def search(self, query: str, max_results: int) ->
list[SearchResult]: ...

class SearchService:
    def __init__(self, backend: SearchBackend):
        self._backend = backend

    def search(self, query: str, max_results: int = 20) ->
list[SearchResult]:
        if not query.strip():
            return []
        results = self._backend.search(query.strip(), max_results)
        return sorted(results, key=lambda r: r.score, reverse=True)
```

No `QObject`, no `Signal`, no `Property`. This code runs anywhere Python runs.

The ViewModel Layer

The viewmodel wraps service calls in Qt-compatible wrappers:

```python
# viewmodels/search_viewmodel.py
from PySide6.QtCore import QObject, Property, Signal, Slot
from PySide6.QtQml import QmlElement
from services.search_service import SearchService
from models.search_result_model import SearchResultModel

QML_IMPORT_NAME = "com.example.app"
QML_IMPORT_MAJOR_VERSION = 1

@QmlElement
class SearchViewModel(QObject):
    queryChanged = Signal()
    searchingChanged = Signal()
    resultsChanged = Signal()

    def __init__(self, service: SearchService, parent=None):
        super().__init__(parent)
        self._service = service
        self._query = ""
        self._searching = False
        self._results = SearchResultModel(parent=self)

    @Property(str, notify=queryChanged)
    def query(self):
        return self._query

    @query.setter
    def query(self, value: str):
        if self._query != value:
            self._query = value
            self.queryChanged.emit()

    @Property(bool, notify=searchingChanged)
    def searching(self):
        return self._searching

    @Property(QObject, notify=resultsChanged)
    def results(self):
        return self._results

    @Slot()
    def performSearch(self):
        if not self._query:
            return
        self._searching = True
        self.searchingChanged.emit()
```

```
        # Run in a thread (Chapter 11 pattern)
        worker = ComputeWorker(self._service.search, self._query)
        worker.signals.finished.connect(self._onSearchComplete)
        QThreadPool.globalInstance().start(worker)

    @Slot(object)
    def _onSearchComplete(self, results):
        self._results.setResults(results)
        self._searching = False
        self.searchingChanged.emit()
        self.resultsChanged.emit()
```

QML interacts only with the viewmodel's public interface. The service layer is invisible to QML.

What Belongs in QML

QML is responsible for:

- Declaring item structure and visual properties
- Binding properties to viewmodel state
- Calling viewmodel slots in response to user interaction
- Animating transitions between states
- Navigation decisions (which screen to show, when)

QML should **not** contain:

- Business rules ("if user has premium plan, enable feature X")
- Data transformation ("convert bytes to human-readable size")
- Validation logic ("is this email address valid")
- Network or file access
- Significant JavaScript computation

The test: could you swap QML for a terminal UI or a REST API without changing the Python layer? If yes, the separation is clean.

Integrating Python Libraries (NumPy, Pandas, SQLAlchemy) as Qt Models

NumPy Integration

NumPy arrays as model data require conversion at the boundary. For numeric displays (charts, tables, heatmaps), wrap a numpy array in a `QAbstractTableModel`:

```
import numpy as np
from PySide6.QtCore import QAbstractTableModel, QModelIndex, Qt

class NumpyTableModel(QAbstractTableModel):
    def __init__(self, array: np.ndarray, parent=None):
        super().__init__(parent)
        self._data = array

    def rowCount(self, parent=QModelIndex()) -> int:
        return 0 if parent.isValid() else self._data.shape[0]

    def columnCount(self, parent=QModelIndex()) -> int:
        return 0 if parent.isValid() else self._data.shape[1] if
self._data.ndim > 1 else 1

    def data(self, index: QModelIndex, role=Qt.DisplayRole):
        if not index.isValid():
            return None
        value = self._data[index.row(), index.column()]
        if role == Qt.DisplayRole:
            return f"{value:.4f}" if isinstance(value, float) else
str(value)
        if role == Qt.UserRole:  # raw value for QML computations
            return float(value)
        return None

    def updateData(self, new_array: np.ndarray):
        self.beginResetModel()
        self._data = new_array
        self.endResetModel()
```

For chart data (series of x/y points), pass numpy arrays directly to a `Slot` and unpack them in QML using JavaScript or a specialized series model:

```
@Slot(result="QVariantList")
def getChartData(self) -> list:
    # Convert numpy to plain Python list for QML consumption
    return [{"x": float(x), "y": float(y)}
            for x, y in zip(self._xdata, self._ydata)]
```

Pandas Integration

Pandas `DataFrame`s map naturally to `QAbstractTableModel` (each row is a data row, each column is a field). The earlier `DataFrameModel` example applies directly. Additional considerations:

Filtering and sorting without copying:

```
def applyFilter(self, column: str, value: str):
    self.beginResetModel()
    mask = self._source_df[column].astype(str).str.contains(value,
case=False)
    self._df = self._source_df[mask]
    self.endResetModel()
```

Using `QSortFilterProxyModel` with a DataFrame model: The built-in proxy model works but re-sorts Python-side when the filtered model is a custom `QAbstractItemModel`. For large DataFrames, implement filtering directly in the model using pandas operations (vectorized, fast) rather than relying on the proxy's row-by-row `filterAcceptsRow`.

SQLAlchemy Integration

SQLAlchemy provides an ORM for relational databases. Integrating it as a Qt model requires careful session management:

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import Session
from PySide6.QtCore import QAbstractListModel, QModelIndex, Qt, Slot

class ContactRepository:
    """Pure Python, no Qt. Manages SQLAlchemy sessions."""

    def __init__(self, db_url: str):
        self._engine = create_engine(db_url)

    def get_all(self) -> list[Contact]:
        with Session(self._engine) as session:
            return
session.query(Contact).order_by(Contact.name).all()

    def add(self, name: str, email: str) -> Contact:
        with Session(self._engine) as session:
            contact = Contact(name=name, email=email)
            session.add(contact)
            session.commit()
            session.refresh(contact)
            return contact

class ContactListModel(QAbstractListModel):
    NameRole = Qt.UserRole + 1
    EmailRole = Qt.UserRole + 2

    def __init__(self, repository: ContactRepository, parent=None):
        super().__init__(parent)
        self._repo = repository
        self._contacts: list[Contact] = []
        self.refresh()

    @Slot()
    def refresh(self):
        self.beginResetModel()
        self._contacts = self._repo.get_all()
        self.endResetModel()

    @Slot(str, str)
    def addContact(self, name: str, email: str):
        contact = self._repo.add(name, email)
        self.beginInsertRows(QModelIndex(), len(self._contacts),
len(self._contacts))
        self._contacts.append(contact)
        self.endInsertRows()
```

Keep SQLAlchemy sessions scoped to the repository methods — never hold a long-lived session open. This prevents lock contention on SQLite and avoids stale-data issues on other databases.

Background Threads, `QThreadPool`, and `QRunnable` from Python

Choosing a Threading Strategy

Scenario	Recommended approach
I/O-bound (network, file)	`asyncio` + `qasync`, or `QThread` with blocking I/O
CPU-bound, short tasks	`QThreadPool` + `QRunnable`
Long-running background service	Dedicated `QThread` with `moveToThread` worker
Reactive streams / event-driven	`QThread` worker emitting signals

`QThread` with Worker Object Pattern

The `moveToThread` pattern is the canonical Qt approach for long-running workers:

```python
from PySide6.QtCore import QObject, QThread, Signal, Slot

class DataSyncWorker(QObject):
    syncComplete = Signal(int)   # number of records synced
    errorOccurred = Signal(str)
    progressUpdated = Signal(int, int)  # current, total

    @Slot()
    def startSync(self):
        try:
            records = fetch_remote_records()
            total = len(records)
            for i, record in enumerate(records):
                store_record(record)
                self.progressUpdated.emit(i + 1, total)
            self.syncComplete.emit(total)
        except Exception as e:
            self.errorOccurred.emit(str(e))

class SyncManager(QObject):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._worker = DataSyncWorker()
        self._thread = QThread(parent=self)

        self._worker.moveToThread(self._thread)
        self._thread.started.connect(self._worker.startSync)
        self._worker.syncComplete.connect(self._thread.quit)
        self._worker.syncComplete.connect(self._onSyncComplete)
        self._worker.errorOccurred.connect(self._onError)

    @Slot()
    def sync(self):
        if not self._thread.isRunning():
            self._thread.start()

    @Slot(int)
    def _onSyncComplete(self, count: int):
        print(f"Synced {count} records")

    @Slot(str)
    def _onError(self, message: str):
        print(f"Sync error: {message}")
```

Key points:

- The worker is created on the main thread but moved before starting the thread
- Never call worker methods directly from the main thread after `moveToThread` — use `invokeMethod` or connect-and-emit
- The thread and worker lifetimes must be managed carefully — use parent relationships

Cancellation

Qt does not have a built-in cancellation mechanism. The standard approach is a cancellation flag:

```
class CancellableWorker(QObject):
    def __init__(self):
        super().__init__()
        self._cancelled = False

    @Slot()
    def cancel(self):
        self._cancelled = True

    @Slot()
    def run(self):
        for item in large_dataset:
            if self._cancelled:
                break
            process(item)
```

Connect `cancelButton.clicked` to `worker.cancel`. The flag is set on the main thread and read on the worker thread; for a simple boolean, this is typically safe in practice, though a `threading.Event` is more formally correct.

Summary

Clean Python backends for QML applications separate domain logic (no Qt, fully testable) from viewmodel logic (Qt-aware, thin wrappers). Popular Python data

libraries — NumPy, Pandas, SQLAlchemy — integrate through `QAbstractItemModel` subclasses that handle the impedance mismatch between Python data structures and Qt's row/column/role model. Threading is the most error-prone aspect: all model mutations must happen on the main thread, and Qt's signal/slot system with `QueuedConnection` is the safest bridge between worker threads and the main thread. The `moveToThread` + worker object pattern handles long-running background operations cleanly and idiomatically.

Chapter 13: Packaging and Deployment

`pyside6-deploy`: Nuitka-Based Packaging and Platform Targets

Distributing a PySide6 application to end users who do not have Python or PySide6 installed requires packaging the interpreter, the PySide6 libraries, Qt libraries, and all application code into a self-contained bundle. `pyside6-deploy` automates this using Nuitka — a Python-to-C++ transpiler that compiles Python code and bundles all dependencies.

How `pyside6-deploy` Works

`pyside6-deploy` orchestrates the following pipeline:

1. **Dependency analysis**: Scans the application entry point for imports and builds a dependency graph.
2. **Nuitka compilation**: Transpiles Python source to C++, compiles it, and links it with the Python runtime.
3. **Qt dependency collection**: Identifies which Qt modules and plugins are used and copies only those (not the full Qt installation).
4. **QML cache compilation**: Pre-compiles QML files to bytecode (`.qmlc`) for faster startup.
5. **Resource embedding**: Bundles images, fonts, and QML files into the executable or alongside it.
6. **Platform bundle creation**: Produces a platform-appropriate output (see below).

Basic Usage

```
# Install pyside6-deploy (included with PySide6)
pyside6-deploy main.py

# With explicit configuration
pyside6-deploy --config pysidedeploy.spec main.py
```

The output is a `deployment/` directory containing the standalone executable and any required shared libraries.

Configuration File

`pyside6-deploy` generates a `pysidedeploy.spec` (INI format) on first run:

```
[app]
title = MyApplication
project_dir = .
input_file = main.py
project_file =
exec_directory = deployment

[python]
python_path = /usr/bin/python3
packages = PySide6,numpy,sqlalchemy

[qt]
qml_files = qml/Main.qml,qml/components
qt_plugins = imageformats,iconengines,platforms

[nuitka]
extra_args = --follow-import-to=backend --include-package=backend
```

Key options:

Option	Purpose
`packages`	Python packages to include (analyzed recursively)
`qml_files`	QML files to embed as resources

Option	Purpose
`qt_plugins`	Qt plugins to include (image formats, platforms, SQL drivers)
`extra_args`	Additional Nuitka arguments

Platform Targets

Linux: Produces a directory with an ELF executable and `.so` libraries. For distribution, wrap in an AppImage (see below).

Windows: Produces a directory with a `.exe` and DLLs. For distribution, wrap with NSIS or WiX installer, or use Windows Package Manager (winget).

macOS: Produces a `.app` bundle. For distribution, sign and notarize, then wrap in a `.dmg`.

Android (experimental): Cross-compilation via Qt for Android. `pyside6-deploy` generates a Gradle project.

WebAssembly (experimental): Targets the browser via Qt for WebAssembly. Produces `.wasm` + `.html` + `.js`.

Resource Bundling with `pyside6-rcc`

Qt's resource system compiles arbitrary files (QML, images, fonts, config files) into a binary format that can be embedded directly into the application executable or loaded from a compiled `.rcc` file.

The Resource Collection File

Define resources in a `.qrc` XML file:

```
<!-- resources.qrc -->
<RCC>
    <qresource prefix="/qml">
        <file>qml/Main.qml</file>
        <file>qml/components/SearchBar.qml</file>
        <file>qml/screens/HomeScreen.qml</file>
    </qresource>
    <qresource prefix="/images">
        <file alias="logo.png">resources/images/logo.png</file>
        <file
alias="logo@2x.png">resources/images/logo@2x.png</file>
    </qresource>
    <qresource prefix="/fonts">
        <file>resources/fonts/Inter-Regular.ttf</file>
        <file>resources/fonts/Inter-Bold.ttf</file>
    </qresource>
</RCC>
```

Compiling Resources

```
pyside6-rcc resources.qrc -o resources_rc.py
```

The output is a Python module containing a function that registers the resources with Qt's virtual filesystem.

Using Resources

```
# main.py — import the compiled resources module before loading QML
import resources_rc  # noqa: F401

engine = QQmlApplicationEngine()
# Now load QML from the virtual filesystem
engine.load("qrc:/qml/Main.qml")
```

In QML, resource paths use the `qrc:` scheme:

```
Image { source: "qrc:/images/logo.png" }
Text { font.family: "Inter" }  // After registering the font
```

Font Registration

Fonts embedded in resources need explicit registration with Qt's font database:

```
from PySide6.QtGui import QFontDatabase

QFontDatabase.addApplicationFont(":/fonts/Inter-Regular.ttf")
QFontDatabase.addApplicationFont(":/fonts/Inter-Bold.ttf")
```

After registration, use the font by family name in QML as normal.

Automatic Resource Embedding in `pyside6-deploy`

When using `pyside6-deploy`, specify QML files in the `qml_files` configuration option. The tool automatically compiles them into resources and removes the separate file dependencies from the output package.

Cross-Platform Distribution

Linux: AppImage

AppImage packages the application into a single self-contained file that runs on any Linux distribution without installation:

```
# Install appimagetool
wget
https://github.com/AppImage/AppImageKit/releases/latest/download/app
imagetool-x86_64.AppImage
chmod +x appimagetool-x86_64.AppImage

# Create AppDir structure
mkdir -p MyApp.AppDir/usr/bin
mkdir -p MyApp.AppDir/usr/lib

# Copy pyside6-deploy output
cp -r deployment/* MyApp.AppDir/usr/bin/

# Create AppRun script
cat > MyApp.AppDir/AppRun << 'EOF'
#!/bin/bash
HERE=$(dirname $(readlink -f "$0"))
exec "$HERE/usr/bin/MyApp" "$@"
EOF
chmod +x MyApp.AppDir/AppRun

# Create .desktop file
cat > MyApp.AppDir/MyApp.desktop << 'EOF'
[Desktop Entry]
Name=MyApp
Exec=MyApp
Icon=myapp
Type=Application
Categories=Utility;
EOF

# Copy icon
cp resources/images/logo.png MyApp.AppDir/myapp.png

# Build AppImage
./appimagetool-x86_64.AppImage MyApp.AppDir MyApp-1.0-
x86_64.AppImage
```

macOS: `.app` Bundle and Notarization

`pyside6-deploy` produces a `.app` bundle on macOS. For distribution outside the Mac App Store:

1. **Code signing** (required for notarization):

```
codesign --deep --force --verify --verbose \
    --sign "Developer ID Application: Your Name (TEAMID)" \
    --options runtime \
    MyApp.app
```

2. **Notarization**:

```
# Create a ZIP of the .app for upload
ditto -c -k --keepParent MyApp.app MyApp.zip

# Submit for notarization
xcrun notarytool submit MyApp.zip \
    --apple-id "your@email.com" \
    --password "@keychain:APP_PASSWORD" \
    --team-id "TEAMID" \
    --wait

# Staple the notarization ticket
xcrun stapler staple MyApp.app
```

3. **DMG creation**:

```
# create-dmg is a popular tool for styled DMG creation
create-dmg \
    --volname "MyApp Installer" \
    --background "resources/dmg_background.png" \
    --window-size 600 400 \
    --app-drop-link 450 185 \
    "MyApp-1.0.dmg" \
    "MyApp.app"
```

Windows: Installer with NSIS

NSIS (Nullsoft Scriptable Install System) creates traditional Windows installers:

```
; MyApp.nsi
!include "MUI2.nsh"

Name "MyApp"
OutFile "MyApp-1.0-Setup.exe"
InstallDir "$PROGRAMFILES64\MyApp"
RequestExecutionLevel admin

!insertmacro MUI_PAGE_WELCOME
!insertmacro MUI_PAGE_DIRECTORY
!insertmacro MUI_PAGE_INSTFILES
!insertmacro MUI_PAGE_FINISH

Section "Main"
    SetOutPath "$INSTDIR"
    File /r "deployment\*.*"
    CreateShortCut "$DESKTOP\MyApp.lnk" "$INSTDIR\MyApp.exe"
    WriteUninstaller "$INSTDIR\Uninstall.exe"
SectionEnd

Section "Uninstall"
    RMDir /r "$INSTDIR"
    Delete "$DESKTOP\MyApp.lnk"
SectionEnd


makensis MyApp.nsi
```

Alternatively, use WiX Toolset for MSI packages (better for enterprise deployment with Group Policy support).

CI/CD Integration

A GitHub Actions workflow that builds for all platforms:

```yaml
# .github/workflows/release.yml
name: Build and Release

on:
  push:
    tags: ['v*']

jobs:
  build:
    strategy:
      matrix:
        include:
          - os: ubuntu-22.04
            target: linux
          - os: macos-14
            target: macos
          - os: windows-2022
            target: windows

    runs-on: ${{ matrix.os }}

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install PySide6 nuitka

      - name: Build
        run: pyside6-deploy --config pysidedeploy.spec main.py

      - name: Package (Linux)
        if: matrix.target == 'linux'
        run: |
          # Build AppImage
          ./scripts/build_appimage.sh

      - name: Package (macOS)
        if: matrix.target == 'macos'
        env:
          APPLE_ID: ${{ secrets.APPLE_ID }}
          APPLE_PASSWORD: ${{ secrets.APPLE_PASSWORD }}
          TEAM_ID: ${{ secrets.TEAM_ID }}
```

```yaml
      run: |
        ./scripts/sign_and_notarize.sh

    - name: Package (Windows)
      if: matrix.target == 'windows'
      run: |
        makensis scripts/installer.nsi

    - name: Upload artifact
      uses: actions/upload-artifact@v4
      with:
        name: myapp-${{ matrix.target }}
        path: dist/
```

Startup Performance

QML Ahead-of-Time Compilation

`qmlsc` (the QML static compiler) compiles QML to C++ at build time, significantly reducing startup time and improving runtime performance. For PySide6 projects:

```
pyside6-deploy --qml-compile-mode=all main.py
```

This requires:

- All QML types are registered (not just used as `var`)
- `QML_IMPORT_NAME` and version are set correctly
- No `Qt.createQmlObject()` with dynamic strings

Lazy Module Import

Python imports are synchronous and can be slow for large packages. Defer heavy imports:

```python
# main.py — fast startup: only import essentials
import sys
from PySide6.QtGui import QGuiApplication
from PySide6.QtQml import QQmlApplicationEngine

def main():
    app = QGuiApplication(sys.argv)
    engine = QQmlApplicationEngine()
    engine.load("qrc:/qml/Main.qml")
    # Heavy imports happen in response to user actions:
    # from backend.heavy_module import HeavyThing
    sys.exit(app.exec())
```

Summary

`pyside6-deploy` with Nuitka provides a one-command packaging pipeline that handles Python compilation, Qt dependency collection, and QML pre-compilation. Resource bundling with `pyside6-rcc` embeds all assets into the executable, eliminating loose file dependencies. Platform-specific distribution — AppImage on Linux, signed `.app` bundles on macOS, NSIS installers on Windows — follows standard conventions for each platform. Automating this in CI/CD with GitHub Actions produces reproducible release artifacts for all platforms from a single codebase.

Chapter 14: Qt Quick from C++: Project Structure and Build Systems

CMake with Qt6: `qt_add_qml_module`, Backing Types, and URI Namespaces

Qt 6 standardized on CMake as the primary build system. The Qt CMake integration provides high-level commands that handle QML module registration, resource compilation, and type stub generation automatically — replacing the manual `RESOURCES` lists and `qmlRegisterType` calls of the Qt 5 era.

Minimal Project Structure

```
my_app/
├── CMakeLists.txt
├── main.cpp
├── backend/
│   ├── CMakeLists.txt
│   ├── counter.h
│   ├── counter.cpp
│   └── contactmodel.h
│   └── contactmodel.cpp
└── qml/
    ├── CMakeLists.txt
    ├── Main.qml
    ├── components/
    │   └── ContactDelegate.qml
    └── screens/
        └── ContactListScreen.qml
```

Root `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.27)
project(MyApp VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Qt6 6.6 REQUIRED COMPONENTS
    Core Gui Qml Quick QuickControls2
)

qt_standard_project_setup(REQUIRES 6.6)

add_subdirectory(backend)
add_subdirectory(qml)

qt_add_executable(myapp main.cpp)

target_link_libraries(myapp PRIVATE
    Qt6::Core
    Qt6::Gui
    Qt6::Qml
    Qt6::Quick
    myapp_backend
    myapp_qml
)
```

`qt_add_qml_module`

`qt_add_qml_module` is the central command for registering a QML module. It:

- Compiles QML files as resources
- Generates type registration code from `QML_ELEMENT` macros
- Produces `qmltypes` files for tooling
- Optionally invokes `qmlsc` for ahead-of-time compilation

```
# backend/CMakeLists.txt
qt_add_library(myapp_backend STATIC)

qt_add_qml_module(myapp_backend
    URI "com.example.backend"
    VERSION 1.0
    SOURCES
        counter.h counter.cpp
        contactmodel.h contactmodel.cpp
    # No QML_FILES here — backend has no QML
)

target_link_libraries(myapp_backend PUBLIC
    Qt6::Core Qt6::Qml Qt6::Quick
)


# qml/CMakeLists.txt
qt_add_library(myapp_qml STATIC)

qt_add_qml_module(myapp_qml
    URI "com.example.app"
    VERSION 1.0
    QML_FILES
        Main.qml
        components/ContactDelegate.qml
        screens/ContactListScreen.qml
    RESOURCE_PREFIX "/qt/qml"
)

target_link_libraries(myapp_qml PUBLIC
    Qt6::Quick
    myapp_backend
)
```

The `RESOURCE_PREFIX` Convention

Qt 6.5+ standardizes on `/qt/qml` as the resource prefix for QML modules. QML files under this prefix are accessible as `qrc:/qt/qml/<URI path>/FileName.qml`. The engine automatically searches this prefix when resolving module URIs — no `addImportPath` needed.

main.cpp

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQuickStyle>

// Include the generated module registration header
#include "myapp_backend/myapp_backendplugin.h"

int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);
    QQuickStyle::setStyle("Material");

    QQmlApplicationEngine engine;

    // Qt 6.5+: modules are auto-registered via linker
initialization
    // For older versions or static builds, explicit registration
may be needed:
    // MyApp_backendPlugin::registerTypes("com.example.backend");

    engine.loadFromModule("com.example.app", "Main");

    if (engine.rootObjects().isEmpty())
        return -1;

    return app.exec();
}
```

For static builds (`STATIC` libraries), the linker may strip unreferenced symbols including the auto-registration functions. Add explicit linker hints:

```
target_link_libraries(myapp PRIVATE
    Qt6::QmlImportScanner
)
qt_import_qml_plugins(myapp)   # ensures static QML plugins are
linked
```

The QML Type Registration System: `QML_ELEMENT`, `QML_SINGLETON`, `QML_FOREIGN`

These macros are placed in the class definition header and processed by `qt_add_qml_module`'s code generator (`qmlcachegen`) to produce registration boilerplate.

`QML_ELEMENT`

Registers the C++ class as a creatable QML type. The QML type name equals the C++ class name by default:

```
// counter.h
#pragma once
#include <QObject>
#include <QtQml/qqml.h>

class Counter : public QObject
{
    Q_OBJECT
    QML_ELEMENT

    Q_PROPERTY(int value READ value WRITE setValue NOTIFY
valueChanged)

public:
    explicit Counter(QObject *parent = nullptr);
    int value() const;
    void setValue(int v);

public slots:
    void increment();
    void reset();

signals:
    void valueChanged(int value);

private:
    int m_value = 0;
};
```

In QML (after `import com.example.backend`):

```
Counter {
    id: counter
    onValueChanged: console.log("Count:", value)
}
```

QML_SINGLETON

Registers as a singleton. The engine calls the static `create()` factory or the constructor if no factory is provided:

```
class AppSettings : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    QML_SINGLETON

    Q_PROPERTY(QString theme READ theme WRITE setTheme NOTIFY
themeChanged)

public:
    // Static factory — called by the engine
    static AppSettings *create(QQmlEngine *engine, QJSEngine
*scriptEngine)
    {
        Q_UNUSED(engine)
        Q_UNUSED(scriptEngine)
        static AppSettings instance;
        return &instance;
    }

    // ... rest of implementation
};
```

QML_ANONYMOUS

Registers a type that can be used as a property type (returned from C++ functions, held in `Q_PROPERTY`) but cannot be instantiated in QML:

```
class SearchResult : public QObject
{
    Q_OBJECT
    QML_ANONYMOUS

    Q_PROPERTY(QString title READ title CONSTANT)
    Q_PROPERTY(QUrl url READ url CONSTANT)

public:
    SearchResult(QString title, QUrl url, QObject *parent =
nullptr);
    QString title() const { return m_title; }
    QUrl url() const { return m_url; }

private:
    QString m_title;
    QUrl m_url;
};
```

QML_NAMED_ELEMENT

Override the QML name when the C++ name is unsuitable:

```
class InternalHttpClient : public QObject
{
    Q_OBJECT
    QML_NAMED_ELEMENT(HttpClient)   // exposed as "HttpClient" in
QML
    // ...
};
```

QML_FOREIGN

Registers a type defined in external code (a third-party library or a Qt class) that you cannot modify to add macros. Create a wrapper struct:

```
// Expose QProcess to QML without modifying Qt's source
struct QProcessForeign
{
    Q_GADGET
    QML_FOREIGN(QProcess)
    QML_NAMED_ELEMENT(Process)
    QML_UNCREATABLE("Use processManager.create()")
};
```

QML_UNCREATABLE

Marks a type as non-instantiable (error message shown if QML tries to create it directly), while still allowing it as a property type:

```
class AbstractDevice : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    QML_UNCREATABLE("AbstractDevice cannot be instantiated
directly")
    // ...
};
```

QML_INTERFACE

Declares a C++ interface (pure virtual class) as a QML interface type. QML does not use this for instantiation but tooling and `qmlsc` use it for type checking.

`qmllint` and `qmlsc`: Static Analysis and Ahead-of-Time Compilation

`qmllint`

`qmllint` performs static analysis on QML files, catching:

- Undefined properties and signals
- Type mismatches
- Deprecated API usage
- Import issues
- Unused imports

```
qmllint --qmltypes path/to/module.qmltypes qml/Main.qml
```

With CMake, enable linting as a build step:

```
qt_add_qml_module(myapp_qml
    URI "com.example.app"
    VERSION 1.0
    QML_FILES Main.qml
    ENABLE_TYPE_COMPILER   # enables qmlsc
)

# Add linting target
set_target_properties(myapp_qml PROPERTIES
    QT_QML_MODULE_LINTING ON
)
```

Run `cmake --build . --target myapp_qml_qmllint` to lint all QML files in the module.

Common `qmllint` warning categories and how to address them:

Warning	Fix
`Unqualified access`	Use `id.property` instead of bare `property`
`Deprecated`	Update to the current API

Warning	Fix
`Missing type`	Ensure all imports are present
`Type mismatch`	Correct the type annotation or cast

`qmlsc` (QML Static Compiler)

`qmlsc` compiles QML to C++ during the build, replacing runtime JavaScript interpretation with compiled C++ for binding expressions and simple functions. Benefits:

- Faster startup (no JIT warm-up for bindings)
- Better type checking at compile time
- Potentially faster runtime for property-access-heavy bindings

Enable in `qt_add_qml_module`:

```
qt_add_qml_module(myapp_qml
    URI "com.example.app"
    VERSION 1.0
    QML_FILES Main.qml
    ENABLE_TYPE_COMPILER    # CMake ≥ 3.25, Qt ≥ 6.4
)
```

Requirements for `qmlsc` compatibility:

- All properties must have explicit type annotations
- No `Qt.createQmlObject()` with dynamic strings
- No `eval()` or other reflection
- All imported types must have `qmltypes` metadata

Verify compatibility: Set `QT_QMLCOMPILER_PASS_WARNINGS=1` in the environment during the build to see which parts of your QML `qmlsc` could not compile and why.

Generating `qmltypes` for Your Modules

`qmltypes` files are metadata that `qmllint`, `qmlsc`, and IDEs use to understand your C++ types. `qt_add_qml_module` generates them automatically:

```
build/
└── com/example/backend/
    ├── backend.qmltypes     # generated from QML_ELEMENT macros
    └── qmldir               # generated module declaration
```

For standalone use (shipping a library to other developers), install the `qmltypes` alongside the header files.

Summary

Qt 6's CMake integration with `qt_add_qml_module` unifies C++ type registration, QML resource compilation, and tooling metadata generation into a single declarative command. The `QML_ELEMENT` macro family provides a clean, in-header API for exposing C++ types to QML without separate registration calls scattered across `main.cpp`. `qmllint` catches QML errors at build time; `qmlsc` compiles QML bindings to C++ for faster startup and tighter type safety. Together, these tools make large-scale C++/QML projects tractable and auditable.

Chapter 15: Exposing C++ Types to QML

Q_PROPERTY, Notify Signals, and Binding-Compatible Design

The Q_PROPERTY Macro

`Q_PROPERTY` declares a named property on a `QObject` subclass, making it accessible from QML as a binding expression target, from JavaScript, and from Qt's reflection APIs:

```
Q_PROPERTY(type name
           READ getter
           [WRITE setter]
           [RESET resetFn]
           [NOTIFY notifySignal]
           [REVISION(majorVersion, minorVersion)]
           [DESIGNABLE bool]
           [STORED bool]
           [CONSTANT]
           [FINAL]
           [REQUIRED])
```

Key clauses:

`NOTIFY` : The signal emitted when the property value changes. QML binding expressions subscribe to this signal — without it, QML reads the property once and never updates when it changes. This is the most common oversight when writing C++ types for QML consumption.

`CONSTANT` : Omits the notifier; declares the value will never change. The QML engine reads it once. Do not use `CONSTANT` on properties that actually change

— the binding engine will not re-evaluate.

`FINAL` : Declares the property will not be overridden in subclasses. The QML compiler can optimize access to `FINAL` properties.

`REQUIRED` : (Qt 6.1+) Marks a property that must be set when the type is instantiated. Useful for mandatory configuration:

```
Q_PROPERTY(QString userId READ userId WRITE setUserId NOTIFY
userIdChanged REQUIRED)
```

Writing Binding-Compatible Properties

A property is “binding-compatible” when:

1. It has a `NOTIFY` signal
2. The signal is emitted *only* when the value actually changes (not on every set)
3. The emit happens *after* the internal state is updated

```qml
// UserProfile.h
class UserProfile : public QObject
{
    Q_OBJECT
    QML_ELEMENT

    Q_PROPERTY(QString displayName READ displayName
               WRITE setDisplayName NOTIFY displayNameChanged FINAL)
    Q_PROPERTY(QUrl avatarUrl READ avatarUrl
               WRITE setAvatarUrl NOTIFY avatarUrlChanged FINAL)
    Q_PROPERTY(bool premium READ isPremium
               NOTIFY premiumChanged FINAL)

public:
    explicit UserProfile(QObject *parent = nullptr);

    QString displayName() const { return m_displayName; }
    void setDisplayName(const QString &name);

    QUrl avatarUrl() const { return m_avatarUrl; }
    void setAvatarUrl(const QUrl &url);

    bool isPremium() const { return m_premium; }

signals:
    void displayNameChanged();
    void avatarUrlChanged();
    void premiumChanged();

private:
    QString m_displayName;
    QUrl m_avatarUrl;
    bool m_premium = false;
};


// UserProfile.cpp
void UserProfile::setDisplayName(const QString &name)
{
    if (m_displayName == name)  // guard: emit only on actual change
        return;
    m_displayName = name;       // update state first
    emit displayNameChanged();  // then notify
}
```

The equality guard is critical. Without it, setting the same value triggers binding re-evaluation, which may cascade through the entire QML binding graph. For expensive-to-compare types, profile before adding the guard, but for strings and numerics, always guard.

Object Properties

Properties that hold `QObject*` pointers work in QML, but ownership must be explicit:

```
Q_PROPERTY(UserProfile *profile READ profile
           WRITE setProfile NOTIFY profileChanged)
```

When QML reads an object property, it gets a reference to the C++ object. The C++ object must outlive the QML reference. Common patterns:

- **Parent the sub-object** to the containing object: lifetime tied to container
- **Use `QQmlEngine::setObjectOwnership`** : explicitly set C++ ownership to prevent QML GC

```
void setProfile(UserProfile *profile) {
    if (m_profile == profile)
        return;
    if (m_profile)
        m_profile->setParent(nullptr);  // release old profile
    m_profile = profile;
    if (m_profile)
        m_profile->setParent(this);     // take ownership
    emit profileChanged();
}
```

Gadgets vs. `QObject`: Value Types in QML with `Q_GADGET`

`Q_GADGET` enables Qt's meta-object features on a class that does *not* inherit `QObject`. Gadgets are value types — they are copied, not referenced, and have no identity, signals, or parent-child relationships.

When to Use Gadgets

Use `Q_GADGET` for:

- Small data transfer objects (DTOs) returned from C++ to QML
- Configuration structures
- Any type that is logically a value (two instances with the same data are equivalent)

Use `Q_OBJECT` for:

- Objects with identity (two instances are distinct even with the same data)
- Objects that emit signals
- Objects with parent-child ownership

Declaring a Gadget

```
// color_info.h
#include <QColor>
#include <QtQml/qqml.h>

class ColorInfo
{
    Q_GADGET
    QML_VALUE_TYPE(colorInfo)   // registers as a QML value type

    Q_PROPERTY(int red READ red CONSTANT)
    Q_PROPERTY(int green READ green CONSTANT)
    Q_PROPERTY(int blue READ blue CONSTANT)
    Q_PROPERTY(float luminance READ luminance CONSTANT)

public:
    ColorInfo() = default;
    explicit ColorInfo(const QColor &color);

    int red() const { return m_color.red(); }
    int green() const { return m_color.green(); }
    int blue() const { return m_color.blue(); }
    float luminance() const;

    Q_INVOKABLE QString toHexString() const;

private:
    QColor m_color;
};

Q_DECLARE_METATYPE(ColorInfo)
```

`QML_VALUE_TYPE(colorInfo)` registers the gadget as a QML type with the given lowercase name. QML can read gadget properties and call `Q_INVOKABLE` methods, but cannot create gadget instances directly in QML (use a factory function).

Returning Gadgets from C++

```
class ColorAnalyzer : public QObject
{
    Q_OBJECT
    QML_ELEMENT

public:
    Q_INVOKABLE ColorInfo analyze(const QColor &color) const;
};
```

In QML:

```
ColorAnalyzer {
    id: analyzer
}

Text {
    property colorInfo info: analyzer.analyze(selectedColor)
    text: "R: " + info.red + " G: " + info.green + " Luminance: " +
info.luminance.toFixed(2)
}
```

Lists of Gadgets

Return a `QList<MyGadget>` from a C++ function to provide a list of value objects:

```
Q_INVOKABLE QList<ColorInfo> analyzeImage(const QUrl &imageUrl)
const;
```

In QML, this becomes a JavaScript array of `colorInfo` objects, iterable with `for...of` and usable in model roles.

Invokable Methods, Enums, and Flags

Q_INVOKABLE

`Q_INVOKABLE` marks a method as callable from QML (and from `QMetaObject::invokeMethod`):

```
class FileManager : public QObject
{
    Q_OBJECT
    QML_ELEMENT

public:
    Q_INVOKABLE bool fileExists(const QString &path) const;
    Q_INVOKABLE QString readTextFile(const QString &path) const;
    Q_INVOKABLE bool writeTextFile(const QString &path, const
QString &content);

    // Async variant: takes a callback
    Q_INVOKABLE void readFileAsync(const QString &path, const
QJSValue &callback);
};
```

`Q_INVOKABLE` methods can take and return any type registered with the meta-object system. For complex return types, use `Q_DECLARE_METATYPE` and `qRegisterMetaType`.

Returning objects vs. values: Returning a raw `QObject*` from an invokable transfers a reference to QML. By default, QML takes ownership (QML engine may GC it). To retain C++ ownership:

```
Q_INVOKABLE QObject *createItem() {
    auto *item = new MyItem(this);  // parent = this, C++ owns it
    QQmlEngine::setObjectOwnership(item, QQmlEngine::CppOwnership);
    return item;
}
```

`Q_INVOKABLE` with `QJSValue` Callbacks

For asynchronous operations, accept a JavaScript function as a callback:

```
void FileManager::readFileAsync(const QString &path, const QJSValue
&callback)
{
    auto *watcher = new QFutureWatcher<QString>(this);
    connect(watcher, &QFutureWatcher<QString>::finished, this,
            [this, watcher, callback]() mutable {
                QString result = watcher->result();
                watcher->deleteLater();
                // Call the JS callback on the main thread
                QJSValue cb = callback;
                cb.call({result});
            });

    watcher->setFuture(QtConcurrent::run([path]() {
        QFile f(path);
        f.open(QIODevice::ReadOnly);
        return QString::fromUtf8(f.readAll());
    }));
}
```

In QML:

```
fileManager.readFileAsync(path, (content) => {
    textEdit.text = content
})
```

`Q_ENUM` and `Q_ENUM_NS`

Enums declared with `Q_ENUM` are accessible from QML with full name qualification:

```
class NetworkStatus : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    QML_SINGLETON

public:
    enum Status {
        Offline,
        Connecting,
        Online,
        Error
    };
    Q_ENUM(Status)

    Q_PROPERTY(Status current READ current NOTIFY currentChanged)
    // ...
};
```

In QML:

```
import com.example.backend

Rectangle {
    color: NetworkStatus.current === NetworkStatus.Online
           ? "green" : "red"
}
```

For namespace-scoped enums (not inside a `QObject`), use `Q_ENUM_NS` within a `Q_NAMESPACE` struct:

```
namespace AppConstants {
    Q_NAMESPACE
    QML_ELEMENT

    enum class Theme { Light, Dark, System };
    Q_ENUM_NS(Theme)
}
```

Q_FLAG and Q_FLAG_NS

Bitfield flag enums:

```
class Permission : public QObject
{
    Q_OBJECT
    QML_ELEMENT

public:
    enum Access {
        None    = 0x0,
        Read    = 0x1,
        Write   = 0x2,
        Execute = 0x4,
        Admin   = 0xFF
    };
    Q_DECLARE_FLAGS(AccessFlags, Access)
    Q_FLAG(Access)

    Q_INVOKABLE bool hasAccess(AccessFlags flags, Access required)
const {
        return flags & required;
    }
};
```

In QML:

```
permission.hasAccess(userFlags, Permission.Write | Permission.Read)
```

Property Bindings and Change Guards: Practical Patterns

Batch Updates with `blockSignals`

When updating multiple related properties at once, use `blockSignals` to suppress intermediate notifications and emit a single comprehensive signal

afterward:

```
void DocumentModel::loadDocument(const Document &doc)
{
    blockSignals(true);
    setTitle(doc.title);
    setAuthor(doc.author);
    setWordCount(doc.wordCount);
    blockSignals(false);

    emit documentLoaded();  // single signal covering all changes
}
```

QML bindings on `title`, `author`, and `wordCount` will re-evaluate once, after all values are set, rather than three times during the update.

`QBindable` and C++20 Bindable Properties

Qt 6.0 introduced `QBindable<T>` — a C++ property binding system that mirrors QML bindings but operates within C++:

```
class TemperatureConverter : public QObject
{
    Q_OBJECT
    QML_ELEMENT

    Q_PROPERTY(double celsius READ celsius WRITE setCelsius NOTIFY
celsiusChanged BINDABLE bindableCelsius)
    Q_PROPERTY(double fahrenheit READ fahrenheit WRITE setFahrenheit
NOTIFY fahrenheitChanged BINDABLE bindableFahrenheit)

public:
    TemperatureConverter() {
        // C++ binding: fahrenheit is always (celsius * 9/5) + 32
        m_fahrenheit.setBinding([this]() {
            return m_celsius.value() * 9.0 / 5.0 + 32.0;
        });
    }

    double celsius() const { return m_celsius; }
    void setCelsius(double v) { m_celsius = v; }
    QBindable<double> bindableCelsius() { return &m_celsius; }

    double fahrenheit() const { return m_fahrenheit; }
    void setFahrenheit(double v) {
        // Setting fahrenheit breaks the binding and recomputes
celsius
        m_fahrenheit = v;
        m_celsius = (v - 32.0) * 5.0 / 9.0;
    }
    QBindable<double> bindableFahrenheit() { return &m_fahrenheit; }

signals:
    void celsiusChanged();
    void fahrenheitChanged();

private:
    Q_OBJECT_BINDABLE_PROPERTY(TemperatureConverter, double,
m_celsius,

&TemperatureConverter::celsiusChanged)
    Q_OBJECT_BINDABLE_PROPERTY(TemperatureConverter, double,
m_fahrenheit,

&TemperatureConverter::fahrenheitChanged)
};
```

`Q_OBJECT_BINDABLE_PROPERTY` is a slot-in replacement for a plain member that participates in Qt's C++ binding graph. Changes propagate automatically in C++ without manual `emit` calls.

Summary

Effective C++-to-QML type exposure requires disciplined use of `Q_PROPERTY` with proper `NOTIFY` signals and change guards. Choosing between `QObject` (reference type, signals, ownership) and `Q_GADGET` (value type, no signals, copied) determines the API contract. `Q_INVOKABLE` methods, `Q_ENUM` / `Q_FLAG`, and `Q_OBJECT_BINDABLE_PROPERTY` complete the toolkit for rich, type-safe C++/QML interfaces. With `QML_ELEMENT` macros and `qt_add_qml_module` in CMake, the boilerplate of type registration is eliminated, leaving clean C++ header declarations that directly map to QML types.

Chapter 16: C++ Models and Data Pipelines

High-Performance `QAbstractItemModel` Implementations

The C++ implementation of `QAbstractItemModel` gives full control over data layout, change notification granularity, and memory management — enabling models that outperform Python equivalents by an order of magnitude for large datasets.

Template Pattern for List Models

Most application list models share the same structural boilerplate. A base template reduces repetition:

```
// listmodelbase.h
#pragma once
#include <QAbstractListModel>
#include <vector>

template<typename T>
class ListModelBase : public QAbstractListModel
{
public:
    explicit ListModelBase(QObject *parent = nullptr)
        : QAbstractListModel(parent) {}

    int rowCount(const QModelIndex &parent = QModelIndex()) const
override {
        return parent.isValid() ? 0 : static_cast<int>
(m_items.size());
    }

    void setItems(std::vector<T> items) {
        beginResetModel();
        m_items = std::move(items);
        endResetModel();
    }

    void appendItem(T item) {
        const int row = static_cast<int>(m_items.size());
        beginInsertRows(QModelIndex(), row, row);
        m_items.push_back(std::move(item));
        endInsertRows();
    }

    void removeItem(int row) {
        if (row < 0 || row >= static_cast<int>(m_items.size()))
            return;
        beginRemoveRows(QModelIndex(), row, row);
        m_items.erase(m_items.begin() + row);
        endRemoveRows();
    }

    void updateItem(int row, T item) {
        if (row < 0 || row >= static_cast<int>(m_items.size()))
            return;
        m_items[row] = std::move(item);
        const QModelIndex idx = index(row);
        emit dataChanged(idx, idx);
    }
```

```
protected:
    std::vector<T> m_items;
};
```

Concrete model inherits and adds roles:

```
// contactmodel.h
struct Contact {
    QString name;
    QString email;
    QUrl avatarUrl;
};

class ContactModel : public ListModelBase<Contact>
{
    Q_OBJECT
    QML_ELEMENT

public:
    enum Roles {
        NameRole = Qt.UserRole + 1,
        EmailRole,
        AvatarUrlRole
    };
    Q_ENUM(Roles)

    explicit ContactModel(QObject *parent = nullptr);

    QVariant data(const QModelIndex &index, int role) const
override;
    QHash<int, QByteArray> roleNames() const override;
};
```

```
// contactmodel.cpp
QHash<int, QByteArray> ContactModel::roleNames() const {
    return {
        {NameRole,     "name"},
        {EmailRole,    "email"},
        {AvatarUrlRole, "avatar"},
    };
}

QVariant ContactModel::data(const QModelIndex &index, int role)
const {
    if (!index.isValid() || index.row() >= static_cast<int>
(m_items.size()))
        return {};

    const Contact &c = m_items[index.row()];
    switch (role) {
    case NameRole:     return c.name;
    case EmailRole:    return c.email;
    case AvatarUrlRole: return c.avatarUrl;
    case Qt::DisplayRole: return c.name;
    default: return {};
    }
}
```

Minimizing dataChanged Overhead

`dataChanged(topLeft, bottomRight, roles)` takes a roles list as the third argument. When roles is empty, the view assumes all roles changed and re-queries all of them. Specify roles precisely:

```
void ContactModel::updateEmail(int row, const QString &email) {
    if (row < 0 || row >= static_cast<int>(m_items.size()))
        return;
    if (m_items[row].email == email)
        return;
    m_items[row].email = email;
    const QModelIndex idx = index(row);
    emit dataChanged(idx, idx, {EmailRole});  // only EmailRole
changed
}
```

For bulk updates (e.g., updating a status field on all items), batch the change:

```
void ContactModel::markAllAsRead() {
    if (m_items.empty()) return;
    for (auto &c : m_items)
        c.unread = false;
    emit dataChanged(index(0), index(rowCount() - 1), {UnreadRole});
}
```

Role-Based Data, `QSortFilterProxyModel`, and Custom Proxy Chains

`QSortFilterProxyModel`

`QSortFilterProxyModel` wraps any `QAbstractItemModel` and provides sorting and filtering without copying data. The view uses the proxy; the proxy delegates to the source model.

```
// Setup in C++
auto *source = new ContactModel(this);
auto *proxy = new QSortFilterProxyModel(this);
proxy->setSourceModel(source);
proxy->setSortRole(ContactModel::NameRole);
proxy->setFilterRole(ContactModel::NameRole);
proxy->setFilterCaseSensitivity(Qt::CaseInsensitive);
proxy->sort(0, Qt::AscendingOrder);


// In QML
TextField {
    onTextChanged: contactProxy.setFilterFixedString(text)
}

ListView {
    model: contactProxy   // the proxy, not the source
}
```

Custom `QSortFilterProxyModel`

For multi-field filtering, subclass and override `filterAcceptsRow`:

```
class ContactFilterProxy : public QSortFilterProxyModel
{
    Q_OBJECT
    QML_ELEMENT

    Q_PROPERTY(QString nameFilter READ nameFilter
               WRITE setNameFilter NOTIFY nameFilterChanged)
    Q_PROPERTY(bool showFavoritesOnly READ showFavoritesOnly
               WRITE setShowFavoritesOnly NOTIFY
showFavoritesOnlyChanged)

public:
    bool filterAcceptsRow(int row, const QModelIndex &parent) const
override {
        const QModelIndex idx = sourceModel()->index(row, 0,
parent);
        const QString name = sourceModel()->data(idx,
ContactModel::NameRole).toString();
        const bool fav = sourceModel()->data(idx,
ContactModel::FavoriteRole).toBool();

        if (m_showFavoritesOnly && !fav)
            return false;
        if (!m_nameFilter.isEmpty() &&
            !name.contains(m_nameFilter, Qt::CaseInsensitive))
            return false;

        return true;
    }

    bool lessThan(const QModelIndex &left, const QModelIndex &right)
const override {
        const QString lName = sourceModel()->data(left,
ContactModel::NameRole).toString();
        const QString rName = sourceModel()->data(right,
ContactModel::NameRole).toString();
        return lName.localeAwareCompare(rName) < 0;
    }

    // ... property accessors and signals
};
```

Proxy Chains

Proxies can be chained — the output of one proxy feeds into another:

```
Source Model
    │
    ▼
FilterProxy (apply search filter)
    │
    ▼
SortProxy (sort by name)
    │
    ▼
GroupingProxy (add section headers)
    │
    ▼
ListView
```

Each proxy is a `QAbstractItemModel` from the next proxy's perspective. This is a powerful composition model for building complex data pipelines without modifying the source model.

Feeding Live Data: Worker Threads, `QFuture`, and `QPromise`

`QFuture` and `QtConcurrent`

`QtConcurrent` provides high-level APIs for running work on a thread pool and monitoring results via `QFuture`:

```
#include <QtConcurrent/QtConcurrent>

class SearchModel : public QAbstractListModel
{
    Q_OBJECT
    QML_ELEMENT

public:
    Q_INVOKABLE void search(const QString &query) {
        // Cancel any running search
        if (m_watcher.isRunning()) {
            m_future.cancel();
            m_watcher.waitForFinished();
        }

        m_future = QtConcurrent::run([query]() ->
QList<SearchResult> {
            return performExpensiveSearch(query);  // runs on thread
pool
        });

        m_watcher.setFuture(m_future);
    }

private slots:
    void onSearchFinished() {
        if (m_future.isCanceled())
            return;
        setResults(m_future.result());
    }

private:
    void setResults(const QList<SearchResult> &results) {
        beginResetModel();
        m_results = results;
        endResetModel();
    }

    QFuture<QList<SearchResult>> m_future;
    QFutureWatcher<QList<SearchResult>> m_watcher{this};
    QList<SearchResult> m_results;

    // Constructor connects watcher:
    // connect(&m_watcher, &QFutureWatcher<...>::finished, this,
    //         &SearchModel::onSearchFinished);
};
```

`QFutureWatcher` emits signals on the main thread even though the work runs on a pool thread — the connection is automatically a `QueuedConnection`.

`QPromise` for Progressive Results

`QPromise` (Qt 6.0+) allows a worker to push results incrementally as they become available:

```
Q_INVOKABLE void loadProgressively(const QStringList &urls) {
    QPromise<ImageData> promise;
    m_future = promise.future();

    // Watch for progressive results
    connect(&m_watcher, &QFutureWatcher<ImageData>::resultReadyAt,
            this, [this](int index) {
                // Called on main thread each time a result is
pushed
                appendItem(m_watcher.resultAt(index));
            });
    m_watcher.setFuture(m_future);

    // Run the work
    QtConcurrent::run([urls, p = std::move(promise)]() mutable {
        p.start();
        for (const QString &url : urls) {
            if (p.isCanceled()) break;
            ImageData data = downloadAndDecode(url);
            p.addResult(std::move(data));
        }
        p.finish();
    });
}
```

As each image loads, `resultReadyAt` fires on the main thread and `appendItem` inserts it into the model. The view updates incrementally — users see content appear as it arrives.

Timer-Driven Live Data

For sensor data, log tailing, or real-time dashboards, a `QTimer` drives periodic model updates:

```
class TelemetryModel : public QAbstractListModel
{
    Q_OBJECT
    QML_ELEMENT

public:
    explicit TelemetryModel(QObject *parent = nullptr) :
QAbstractListModel(parent) {
        m_timer.setInterval(100);   // 10 Hz
        connect(&m_timer, &QTimer::timeout, this,
&TelemetryModel::onTick);
    }

    Q_INVOKABLE void start() { m_timer.start(); }
    Q_INVOKABLE void stop()  { m_timer.stop(); }

private slots:
    void onTick() {
        // Read new data from a lock-free queue filled by a hardware
thread
        QList<TelemetryPoint> batch;
        m_queue.drainTo(batch);

        if (batch.isEmpty())
            return;

        // Trim old data
        const int maxRows = 1000;
        if (m_data.size() + batch.size() > maxRows) {
            const int toRemove = m_data.size() + batch.size() -
maxRows;
            beginRemoveRows(QModelIndex(), 0, toRemove - 1);
            m_data.erase(m_data.begin(), m_data.begin() + toRemove);
            endRemoveRows();
        }

        // Append new batch
        const int first = m_data.size();
        beginInsertRows(QModelIndex(), first, first + batch.size() -
1);
        m_data.append(batch);
        endInsertRows();
    }

    QTimer m_timer;
    LockFreeQueue<TelemetryPoint> m_queue;   // filled by hardware
thread
```

```
    QList<TelemetryPoint> m_data;
};
```

Thread Safety for Model Data

The golden rule: the model's `data()`, `rowCount()`, `beginInsertRows()`, and all other interface methods must be called from the main thread only. Worker threads write to a staging buffer (a queue, a local `std::vector`) and post to the main thread.

For the lock-free queue pattern above:

- Hardware/network thread: pushes to `m_queue` (lock-free)
- Main thread timer: drains `m_queue` and mutates the model

This pattern gives sub-millisecond latency from data arrival to model update, with no main-thread blocking.

Summary

High-performance C++ models use `std::vector` for data storage (cache-friendly, O(1) random access), precise role-based `dataChanged` notifications, and the begin/end mutation API for incremental updates. `QSortFilterProxyModel` and custom proxy chains compose filtering, sorting, and grouping without touching the source model. `QtConcurrent` and `QPromise` provide ergonomic patterns for off-thread data loading with automatic main-thread delivery via `QFutureWatcher`. Timer-driven models with lock-free staging queues serve real-time data scenarios. Together, these patterns support models that remain smooth and responsive at any data scale.

Chapter 17: Extending the Scene Graph

Custom `QQuickItem` Subclasses and the Render Thread

When Qt Quick's built-in visual types are insufficient — custom chart rendering, waveform displays, game sprites, data visualizations — the solution is a custom `QQuickItem` subclass that participates directly in the scene graph.

The `QQuickItem` / Scene Graph Contract

The critical constraint: the UI thread and the render thread run concurrently after the *sync phase*. During sync, the main thread is blocked and scene graph data can be safely copied from `QQuickItem` properties into scene graph nodes. After sync, the render thread runs while the main thread processes events — at this point, touching scene graph nodes from the main thread is a data race.

Qt enforces this contract through `updatePaintNode()`:

```qml
class WaveformItem : public QQuickItem
{
    Q_OBJECT
    QML_ELEMENT

    Q_PROPERTY(QList<float> samples READ samples
               WRITE setSamples NOTIFY samplesChanged)

public:
    explicit WaveformItem(QQuickItem *parent = nullptr);

    QList<float> samples() const { return m_samples; }
    void setSamples(const QList<float> &s);

signals:
    void samplesChanged();

protected:
    // Called on the RENDER THREAD during the sync phase
    QSGNode *updatePaintNode(QSGNode *oldNode,
                             UpdatePaintNodeData *data) override;

    // Called on the MAIN THREAD — geometric changes
    void geometryChange(const QRectF &newGeometry,
                        const QRectF &oldGeometry) override;

private:
    QList<float> m_samples;   // main thread state
    bool m_dirty = false;
};
```

```
void WaveformItem::setSamples(const QList<float> &s)
{
    if (m_samples == s)
        return;
    m_samples = s;
    m_dirty = true;
    emit samplesChanged();
    update();   // Schedule a redraw — triggers updatePaintNode call
}

void WaveformItem::geometryChange(const QRectF &newGeometry,
                                  const QRectF &oldGeometry)
{
    QQuickItem::geometryChange(newGeometry, oldGeometry);
    m_dirty = true;
    update();
}
```

Enabling the Item

A `QQuickItem` that draws must have its `ItemHasContents` flag set:

```
WaveformItem::WaveformItem(QQuickItem *parent)
    : QQuickItem(parent)
{
    setFlag(QQuickItem::ItemHasContents);
}
```

Without this flag, `updatePaintNode` is never called.

QSGNode Trees: Geometry Nodes, Transform Nodes, and Clip Nodes

`QSGGeometryNode`

The leaf node of the scene graph — it holds geometry (vertices) and a material (shader + uniforms).

```qt
QSGNode *WaveformItem::updatePaintNode(QSGNode *oldNode,
                                       UpdatePaintNodeData *)
{
    auto *node = static_cast<QSGGeometryNode *>(oldNode);

    if (!node) {
        node = new QSGGeometryNode;

        // Geometry: line strip with (x, y) per vertex
        auto *geometry = new QSGGeometry(
            QSGGeometry::defaultAttributes_Point2D(),
            0   // vertex count, will resize below
        );
        geometry->setDrawingMode(QSGGeometry::DrawLineStrip);
        geometry->setLineWidth(2.0f);
        node->setGeometry(geometry);
        node->setFlag(QSGNode::OwnsGeometry);

        // Flat color material
        auto *material = new QSGFlatColorMaterial;
        material->setColor(QColor("#3daee9"));
        node->setMaterial(material);
        node->setFlag(QSGNode::OwnsMaterial);
    }

    if (!m_dirty)
        return node;

    m_dirty = false;

    // Rebuild geometry from samples
    auto *geometry = node->geometry();
    geometry->allocate(m_samples.size());

    auto *vertices = geometry->vertexDataAsPoint2D();
    const float w = width();
    const float h = height();
    const float midY = h / 2.0f;

    for (int i = 0; i < m_samples.size(); ++i) {
        float x = (w * i) / (m_samples.size() - 1);
        float y = midY - m_samples[i] * midY;
        vertices[i].set(x, y);
    }

    node->markDirty(QSGNode::DirtyGeometry);
```

```
    return node;
}
```

Key points:

- `oldNode` is the node from the previous frame. Reuse it to avoid allocation.
- `OwnsGeometry` / `OwnsMaterial` flags: the node deletes them when it is deleted.
- `markDirty(DirtyGeometry)` tells the renderer to re-upload the geometry to the GPU.
- All of this runs on the render thread — no QObject access, no signal emission.

QSGTransformNode

Applies a matrix transform to its subtree. Useful for building hierarchical scene graphs:

```
auto *root = new QSGTransformNode;

QMatrix4x4 m;
m.translate(offsetX, offsetY);
m.scale(scale);
root->setMatrix(m);

// Children positioned relative to the transform
root->appendChildNode(geometryNode1);
root->appendChildNode(geometryNode2);

return root;
```

QSGClipNode

Clips its subtree to a rectangular or geometric region:

```
auto *clipNode = new QSGClipNode;
clipNode->setIsRectangular(true);
clipNode->setClipRect(QRectF(0, 0, width(), height()));
clipNode->appendChildNode(contentNode);
```

Clipping is implemented with stencil buffer operations, so it has a GPU cost. Prefer Qt Quick's `clip: true` on items for rectangular clipping — the scene graph handles it the same way.

`QSGOpacityNode`

Sets opacity for its subtree independently of individual item opacity:

```
auto *opacityNode = new QSGOpacityNode;
opacityNode->setOpacity(0.5f);
opacityNode->appendChildNode(contentNode);
```

Node Ownership and the Node Tree

The scene graph owns the root node returned by `updatePaintNode`. When an item's `updatePaintNode` is called again, `oldNode` is the previously returned node. You may modify and return it, or delete it and return a new one. You must never delete the old node yourself outside `updatePaintNode` — the scene graph manages its lifetime.

Custom `QQuickPaintedItem` and When to Use It vs. Raw Scene Graph Nodes

`QQuickPaintedItem` is a simpler alternative to custom scene graph nodes. It provides a QPainter-based API, rendering the item to an offscreen surface that is then uploaded to a GPU texture:

```
class PieChartItem : public QQuickPaintedItem
{
    Q_OBJECT
    QML_ELEMENT

    Q_PROPERTY(QVariantList slices READ slices WRITE setSlices
NOTIFY slicesChanged)

public:
    void paint(QPainter *painter) override {
        painter->setRenderHint(QPainter::Antialiasing);
        const QRectF rect = boundingRect().adjusted(4, 4, -4, -4);
        double startAngle = 0;

        for (const QVariant &v : m_slices) {
            const QVariantMap slice = v.toMap();
            const double sweep = slice["value"].toDouble() * 360.0;
            painter->setBrush(QColor(slice["color"].toString()));
            painter->setPen(Qt::NoPen);
            painter->drawPie(rect,
                             static_cast<int>(startAngle * 16),
                             static_cast<int>(sweep * 16));
            startAngle += sweep;
        }
    }

    // ... properties
};
```

When to Use `QQuickPaintedItem`

Use `QQuickPaintedItem` when:

- Rendering logic is complex and already uses QPainter (porting from QWidget)
- The item updates infrequently (the texture-upload cost amortizes)
- Vector drawing primitives (arcs, beziers, text) are needed
- Development speed matters more than peak throughput

Use raw scene graph nodes when:

- The item updates every frame (charts, waveforms, games)

- Maximum throughput is required (geometry is built directly in GPU-ready format)
- Custom shaders are needed
- You need multi-pass rendering

The texture-upload path of `QQuickPaintedItem` involves CPU rendering + GPU upload each time `update()` is called. For a waveform that redraws at 60 Hz with 10,000 points, this is prohibitively expensive. Raw geometry nodes skip the CPU rasterization entirely.

Integrating External Renderers via `QQuickRenderTarget`

Qt 6.0 introduced `QQuickRenderTarget` — a mechanism for directing Qt Quick's rendering to an externally created framebuffer object, texture, or render buffer. This enables:

- Rendering Qt Quick UI into a Vulkan/Metal/D3D texture for compositing with a 3D engine
- Displaying Qt Quick UI inside a game engine scene
- Multi-stage rendering pipelines

Rendering Qt Quick into an External Texture

```
// Create a texture on the external renderer's device
VkImage externalImage = /* ... created by your Vulkan code ... */;

// Direct Qt Quick to render into it
QQuickRenderTarget target = QQuickRenderTarget::fromVulkanImage(
    externalImage,
    VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
    QSize(1920, 1080)
);
quickWindow->setRenderTarget(target);
```

Embedding External Rendering in a Qt Quick Scene

The `QQuickGraphicsDevice` and `QRhi` (Qt Rendering Hardware Interface) APIs allow Qt Quick and external renderers to share a GPU device:

```
// Tell Qt Quick to use the same Vulkan device as your engine
QQuickGraphicsDevice device =
QQuickGraphicsDevice::fromDeviceObjects(
    physicalDevice, logicalDevice, graphicsQueueFamilyIndex,
graphicsQueueIndex
);
quickWindow->setGraphicsDevice(device);
```

With a shared device, resources (textures, buffers) can be passed between Qt Quick and the external renderer without cross-device copies.

`QRhiWidget` (Qt 6.7+)

`QRhiWidget` is a newer, higher-level API that simplifies embedding custom RHI rendering into a Qt Quick scene. It replaces the older `QQuickFramebufferObject` pattern:

```
class CustomRhiWidget : public QRhiWidget
{
    Q_OBJECT
    QML_ELEMENT

public:
    void initialize(QRhiCommandBuffer *cb) override {
        // Create pipelines, buffers, etc.
        m_pipeline = m_rhi->newGraphicsPipeline();
        // ... configure pipeline ...
        m_pipeline->create();
    }

    void render(QRhiCommandBuffer *cb) override {
        // Issue draw calls — runs on the render thread
        QRhiCommandBuffer::BeginRenderPassFlags flags;
        cb->beginRenderPass(m_renderTarget, flags);
        cb->setGraphicsPipeline(m_pipeline);
        cb->draw(3);
        cb->endRenderPass();
    }

private:
    QRhiGraphicsPipeline *m_pipeline = nullptr;
};
```

`QRhiWidget` appears as a regular item in QML; Qt Quick composites its output into the scene.

Summary

The scene graph extension API gives C++ code direct GPU rendering access, integrated seamlessly into Qt Quick's retained scene graph. `QSGGeometryNode` with a `QSGMaterial` is the building block for custom rendered content; `QSGTransformNode`, `QSGClipNode`, and `QSGOpacityNode` compose them into hierarchies. The threading contract — UI properties on the main thread, scene graph manipulation in `updatePaintNode` on the render thread — is non-negotiable and must be respected. `QQuickPaintedItem` offers an easier QPainter-based path for infrequently-updated items. `QQuickRenderTarget` and

`QRhiWidget` address integration with external GPU rendering pipelines, making Qt Quick a viable overlay or component in 3D-engine-driven applications.

Chapter 18: C++/QML Interop Patterns and Performance

Minimizing the C++/JS Boundary: Batch Updates and Coarse-Grained Signals

The C++/JavaScript boundary in Qt Quick is a significant performance boundary. Every property access from QML that crosses into C++ involves:

1. A `QMetaObject` lookup (name → property index)
2. A virtual call to the property's `READ` function
3. Type conversion from C++ type to `QVariant` to JS value
4. The reverse on property writes

For a single access this is negligible. In a tight loop, or for properties accessed in a delegate with thousands of items, it compounds.

Measure Before Optimizing

Use Qt Creator's QML Profiler before making any change. The profiler shows:

- Time spent in QML binding evaluation
- Time spent in C++ property accessors
- Number of signal emissions and binding re-evaluations
- Per-frame breakdown of time in JavaScript vs. scene graph vs. rendering

Without profiler data, optimization is guesswork.

Coarse-Grained Signal Strategy

Each signal emission triggers re-evaluation of all bindings that depend on the associated property. A model that emits `dataChanged` for individual cell changes on a 1,000-row table may trigger thousands of binding updates per second.

Anti-pattern:

```
// Emitted for every incoming sensor reading — 1000/sec
void SensorModel::onNewReading(int sensorId, double value) {
    m_data[sensorId] = value;
    const QModelIndex idx = index(sensorId);
    emit dataChanged(idx, idx, {ValueRole});   // fires 1000
times/sec
}
```

Better: buffer updates and apply them on a timer:

```
void SensorModel::onNewReading(int sensorId, double value) {
    m_pendingUpdates[sensorId] = value;   // just write to the
buffer
}

void SensorModel::flushUpdates() {  // called by a 60Hz QTimer
    if (m_pendingUpdates.empty())
        return;

    int minRow = INT_MAX, maxRow = 0;
    QList<int> changedRoles = {ValueRole};

    for (auto [id, value] : m_pendingUpdates) {
        m_data[id] = value;
        minRow = std::min(minRow, id);
        maxRow = std::max(maxRow, id);
    }
    m_pendingUpdates.clear();

    // One dataChanged for the entire dirty range
    emit dataChanged(index(minRow), index(maxRow), changedRoles);
}
```

One `dataChanged` per frame instead of 1,000 — a dramatic reduction in binding evaluations.

Batch Property Updates

When multiple properties change together conceptually, avoid emitting their notify signals individually if QML bindings read all of them together:

```
// Instead of separate setters that each emit:
void DocumentViewModel::loadDocument(const Document &doc) {
    // Suppress individual notifications
    const QSignalBlocker blocker(this);
    setTitle(doc.title);
    setAuthor(doc.author);
    setPageCount(doc.pageCount);
    setWordCount(doc.wordCount);
    // Re-enable and emit one comprehensive signal
    emit documentLoaded();
}
```

In QML, connect to `documentLoaded` to refresh the entire view rather than four separate property bindings.

Reduce Cross-Boundary Property Accesses

In JavaScript that runs frequently (e.g., inside a delegate's `Component.onCompleted` or a frequently-triggered signal handler), cache property reads in local variables:

```
// Slow: C++ property read on each comparison
function isValidRange(start, end) {
    return end - start >= model.minimumRange &&
           end - start <= model.maximumRange
}

// Fast: read once, use multiple times
function isValidRange(start, end) {
    const min = model.minimumRange   // one C++ read
    const max = model.maximumRange   // one C++ read
    const range = end - start
    return range >= min && range <= max
}
```

Large Lists: `roleNames` and Role Access Cost

In `ListView` delegates, each property access by role name involves a hash lookup in `roleNames()`. For large lists with many roles, use role integers directly via `model.display` (Qt::DisplayRole) or consolidate multiple fields into a single structured role:

```
// Instead of 10 separate roles, return a structured object
case DataPacketRole:
    return QVariant::fromValue(m_items[index.row()]);  // entire
struct as QVariant
```

In the delegate, access fields via the struct's properties rather than 10 separate model roles.

Profiling with Qt Creator's QML Profiler and Scene Graph Debugger

QML Profiler

The QML Profiler (accessible via **Analyze → QML Profiler** in Qt Creator) instruments the application at runtime and records:

- **Binding evaluations**: which bindings ran, how long they took, how often they ran
- **Signal emissions**: which signals fired and from where
- **JavaScript execution**: time spent in JS functions, handlers, and imported `.js` files
- **Component creation**: time to instantiate each component type
- **Rendering frames**: scene graph preparation and GPU submission times

Typical profiling workflow:

1. Start the application from Qt Creator's Analyze menu
2. Exercise the feature or interaction you want to profile
3. Click "Stop" in the profiler UI
4. Examine the timeline view for long bindings, excessive signal emission, or slow JS

Key findings to look for:

- Binding that re-evaluates more than once per frame → signal emitting too frequently
- `QObject::~QObject` calls during scrolling → delegates being destroyed instead of pooled
- Long JavaScript calls during animation → blocking the animation frame

Scene Graph Debugger

Enable scene graph visualization with environment variables:

```
# Show overdraw (areas drawn multiple times per frame)
QSG_VISUALIZE=overdraw ./myapp

# Show batching (items grouped into single draw calls)
QSG_VISUALIZE=batches ./myapp

# Show clipping regions
QSG_VISUALIZE=clip ./myapp

# Show change tracking (which nodes updated each frame)
QSG_VISUALIZE=changes ./myapp
```

Overdraw visualization: Bright red areas are drawn many times per frame. Stack opacity at 100 layers all painting the same region. Common culprits: deeply nested transparent items, incorrectly placed opaque backgrounds, z-ordered items that are all visible simultaneously.

Batch visualization: Items in the same batch are rendered in a single GPU draw call. Green items are batched; red items are not. Unbatched items that could be batched indicate missed optimization opportunities — typically caused by:

- Non-uniform z-values among sibling items
- `clip: true` on intermediate items (creates new batch boundaries)
- Interleaved `layer.enabled` items
- Non-trivial custom materials in the same subtree

Logging Scene Graph Activity

Enable scene graph logging for detailed diagnostics:

```
QT_LOGGING_RULES="qt.scenegraph.time.renderloop=true" ./myapp
```

Available categories:

Category	Shows
`qt.scenegraph.time.renderloop`	Frame timing breakdown
`qt.scenegraph.time.texture`	Texture upload timing

Category	Shows
`qt.scenegraph.time.compilation`	Shader compilation
`qt.qml.binding.removal`	Binding-breaking assignments

`QML_IMPORT_TRACE=1`

Trace QML import resolution to diagnose slow startup:

```
QML_IMPORT_TRACE=1 ./myapp 2>&1 | head -50
```

Shows each import statement and the paths searched, identifying modules that take long to find.

Memory Ownership Contracts: `QQmlEngine::ObjectOwnership` and Avoiding Leaks

The Two Ownership Modes

`QQmlEngine::ObjectOwnership` has two values:

`CppOwnership` : C++ code owns the object. The QML garbage collector will never delete it. The developer is responsible for ensuring the object outlives any QML references to it.

`JavaScriptOwnership` : The QML engine's garbage collector owns the object. It will be deleted when no more QML/JS references to it exist. This is the default for objects created by `QQmlComponent::create()` and `Qt.createQmlObject()`.

Common Ownership Mistakes

Mistake 1: C++ factory function returns an unparented object

```
// Bug: engine may GC this
Q_INVOKABLE QObject *createItem() {
    return new MyItem();   // no parent, no explicit ownership
}
```

QML receives the pointer, the engine assumes `JavaScriptOwnership`, and may GC the object if no QML variable holds a strong reference.

Fix:

```
Q_INVOKABLE QObject *createItem() {
    auto *item = new MyItem(this);   // parent = this (C++ owns it)
    QQmlEngine::setObjectOwnership(item, QQmlEngine::CppOwnership);
    return item;
}
```

Mistake 2: Storing a QML-owned object in a C++ data structure

```
void MyModel::registerItem(QObject *item) {
    m_items.push_back(item);  // C++ holds a raw pointer
    // If QML drops all references, item is GC'd
    // m_items now holds a dangling pointer
}
```

Fix: either parent the item to `this` (takes C++ ownership), or use `QPointer<QObject>` for nullable weak references, or explicitly set `CppOwnership`.

Mistake 3: `QPointer` as a substitute for ownership

```
QPointer<QObject> m_weakRef = receivedItem;
// m_weakRef becoming null means the object was deleted — safe
if (m_weakRef)
    m_weakRef->doSomething();
```

`QPointer` is a weak reference — it becomes null when the object is deleted. Use it to *observe* an object's lifetime without owning it.

Ownership in Practice: The Patterns

Pattern A: Factory with C++ ownership

```
// Object lives as long as backend lives
Q_INVOKABLE SearchController *createSearch(const QString &query) {
    auto *ctrl = new SearchController(query, this);   // parent =
this
    QQmlEngine::setObjectOwnership(ctrl, QQmlEngine::CppOwnership);
    return ctrl;
}
```

Pattern B: QML-managed temporary objects

For objects created in QML that are short-lived (dialogs, notifications):

```
Button {
    onClicked: {
        let dialog = dialogComponent.createObject(root, { message:
"Confirm?" })
        dialog.accepted.connect(() => {
            doAction()
            dialog.destroy()   // explicit destruction
        })
        dialog.rejected.connect(() => dialog.destroy())
    }
}
```

`dialog.destroy()` schedules deletion at end of the current event loop iteration. This is correct — do not `delete` QML objects from C++.

Pattern C: Model-managed child objects

A list model that creates sub-objects parents them all:

```
void ContactModel::loadContacts(const QList<ContactData> &data) {
    // Delete old children
    for (auto *child : findChildren<Contact*>())
        child->deleteLater();

    beginResetModel();
    m_contacts.clear();
    for (const auto &d : data) {
        auto *c = new Contact(d, this);   // parented to model
        QQmlEngine::setObjectOwnership(c, QQmlEngine::CppOwnership);
        m_contacts.push_back(c);
    }
    endResetModel();
}
```

The model owns all `Contact` objects. When the model is deleted, all contacts are deleted via Qt's parent-child mechanism.

Avoiding Binding Loop Warnings

Binding loops occur when property A's binding reads property B, and property B's binding reads property A, creating a cycle:

```
// Binding loop!
Rectangle {
    width: height * 2
    height: width / 2   // depends on width, which depends on
height...
}
```

Qt detects these and emits a runtime warning, then breaks the loop by returning the property's last stable value.

To avoid loops:

- Use `id`-qualified access: `root.width` instead of `width` in a child that also affects `root.width`
- Use intermediate properties to break cycles

- For two-way synchronization between C++ and QML (e.g., a slider and a text input both setting the same value), guard against re-entrant updates with a flag

```
// Safe two-way sync
Slider {
    value: model.value   // binding from model to slider
    onMoved: model.value = value   // only fires on user
interaction, not binding updates
}
```

`onMoved` (vs. `onValueChanged`) fires only when the user moves the slider, not when `value` is set programmatically — preventing the loop.

Summary

Performance at the C++/QML boundary is dominated by signal emission frequency, cross-boundary property access patterns, and scene graph batching. Batch updates with timers or `QSignalBlocker` to reduce signal chatter. Cache C++ property reads in local JS variables in hot paths. The QML Profiler is indispensable for identifying the actual bottlenecks before optimizing. Memory ownership is the most error-prone interop aspect: always set `QQmlEngine::CppOwnership` on objects returned from C++ factories, and use `QPointer` for weak C++ references to QML-owned objects. Binding loops are a correctness concern, not a performance concern — but the fixes (explicit `id` qualification, `onMoved` vs. `onValueChanged`) are worth internalizing.

Chapter 19: Internationalization, Accessibility, and Theming

`qsTr`, `QTranslator`, and `lupdate/lrelease` Workflows

Marking Strings for Translation

The `qsTr()` function marks a string as translatable. It takes an optional disambiguation comment and plural-form count:

```
// Basic translation
Text { text: qsTr("File") }

// With disambiguation (same source string, different context)
Text { text: qsTr("Edit", "menu item") }
Text { text: qsTr("Edit", "button label") }

// Plural forms
Text { text: qsTr("%n item(s) selected", "", selectionCount) }
```

The string argument to `qsTr()` is the *source string* — the English (or base language) text that `lupdate` extracts. At runtime, Qt looks up the translation for this string in the loaded `.qm` file.

`qsTrId()` : For applications that separate translation IDs from source strings (common in enterprise workflows):

```
Text { text: qsTrId("file-menu-open-action") }
```

`qsTranslate()` : Translates a string from a specific context, rather than the current QML file's context:

```
Text { text: qsTranslate("MainMenu", "File") }
```

The `lupdate` / `lrelease` Workflow

`lupdate` scans source files and extracts translatable strings into `.ts` (Translation Source) XML files. `lrelease` compiles `.ts` files into binary `.qm` (Qt Message) files that Qt loads at runtime.

Step 1: Create or update `.ts` files

```
# lupdate all QML files in the project
lupdate qml/ -ts translations/myapp_de.ts translations/myapp_ja.ts

# Or with a .pro file
lupdate myapp.pro
```

Step 2: Translate

Send `.ts` files to translators. They use Qt Linguist (`linguist`) to enter translations:

```
<!-- translations/myapp_de.ts (excerpt) -->
<TS version="2.1" language="de_DE">
    <context>
        <name>Main</name>
        <message>
            <source>File</source>
            <translation>Datei</translation>
        </message>
        <message>
            <source>%n item(s) selected</source>
            <numerusform>%n Element ausgewählt</numerusform>
            <numerusform>%n Elemente ausgewählt</numerusform>
        </message>
    </context>
</TS>
```

Step 3: Compile

```
lrelease translations/myapp_de.ts -qm translations/myapp_de.qm
```

Step 4: Load at runtime

```
# Python
from PySide6.QtCore import QTranslator, QLocale, QLibraryInfo

translator = QTranslator(app)
locale = QLocale.system()
if translator.load(locale, "myapp", "_", ":/translations"):
    app.installTranslator(translator)
```

```
// C++
QTranslator translator;
const QLocale locale = QLocale::system();
if (translator.load(locale, "myapp", "_", ":/translations"))
    app.installTranslator(&translator);
```

CMake Integration

```
qt_add_translations(myapp
    TS_FILES
        translations/myapp_de.ts
        translations/myapp_fr.ts
        translations/myapp_ja.ts
    QM_FILES_OUTPUT_VARIABLE qm_files
)

qt_add_resources(myapp "translations"
    PREFIX "/translations"
    BASE "${CMAKE_CURRENT_BINARY_DIR}"
    FILES ${qm_files}
)
```

`qt_add_translations` runs `lupdate` during the build and ensures `.qm` files are produced and embedded as resources.

Runtime Language Switching

Switching language at runtime requires reinstalling the translator and signaling QML to retranslate:

```
void App::setLanguage(const QString &languageCode) {
    app->removeTranslator(&m_translator);
    if (m_translator.load(":/translations/myapp_" + languageCode +
".qm")) {
        app->installTranslator(&m_translator);
    }
    // Signal QML to re-evaluate all qsTr() calls
    engine->retranslate();
}
```

`QQmlEngine::retranslate()` (Qt 5.10+) triggers re-evaluation of all `qsTr()` binding expressions in all loaded QML components. UI updates immediately without a restart.

Accessibility Tree and `Accessible` Attached Properties

Qt Quick's accessibility support exposes UI elements to assistive technologies (screen readers, switch access, magnifiers) through the platform's accessibility API (AT-SPI on Linux, UIAutomation on Windows, NSAccessibility on macOS).

The `Accessible` Attached Type

Every `Item` can declare its accessibility role and properties via the `Accessible` attached type:

```
Rectangle {
    id: customButton

    Accessible.role: Accessible.Button
    Accessible.name: buttonLabel.text
    Accessible.description: "Saves the current document"
    Accessible.onPressAction: clicked()

    // Enable focus for keyboard navigation
    focusPolicy: Qt.TabFocus
    Keys.onReturnPressed: clicked()
    Keys.onSpacePressed: clicked()

    signal clicked()

    Text { id: buttonLabel; text: "Save" }
    TapHandler { onTapped: customButton.clicked() }
}
```

Accessibility Roles

`Accessible.Role`	Description
`Accessible.Button`	Pushbutton
`Accessible.CheckBox`	Checkbox with checked state
`Accessible.RadioButton`	Radio button in a group
`Accessible.ComboBox`	Drop-down selection
`Accessible.EditableText`	Text input field
`Accessible.List`	List container
`Accessible.ListItem`	Item in a list
`Accessible.MenuItem`	Menu item
`Accessible.MenuBar`	Menu bar
`Accessible.ScrollBar`	Scroll bar
`Accessible.Slider`	Value slider
`Accessible.StaticText`	Non-interactive text label
`Accessible.Pane`	Container pane (groups related items)

Accessible.Role	Description
`Accessible.Window`	Top-level window

Accessible Properties

```
Accessible.role: Accessible.Slider
Accessible.name: "Volume"               // label read by screen
reader
Accessible.description: "Controls the master volume level"
Accessible.valueText: slider.value + "%"  // current value for
sliders
Accessible.minimumValue: 0
Accessible.maximumValue: 100
Accessible.stepSize: 5
Accessible.focusable: true
Accessible.focused: activeFocus
```

Qt Quick Controls Accessibility

Qt Quick Controls 2 controls include built-in `Accessible` declarations. `Button`, `TextField`, `CheckBox`, `Slider` etc. are all accessible by default — their `Accessible.role`, `Accessible.name`, and state properties are set automatically. When building custom controls from scratch, you must add `Accessible` declarations explicitly.

Testing Accessibility

On Linux, use `accerciser` (AT-SPI inspector) to inspect the accessibility tree. On macOS, use the Accessibility Inspector (in Xcode's Additional Tools). On Windows, use Inspect.exe (from the Windows SDK).

Design Token Systems and Runtime Theme Switching

Design Tokens

Design tokens are named values — colors, spacing, typography — that separate visual style decisions from component implementation. In a QML application, tokens are typically exposed as a singleton:

```
// Theme.qml (singleton)
pragma Singleton
import QtQuick

QtObject {
    readonly property color primary: "#3daee9"
    readonly property color primaryVariant: "#1a6e9a"
    readonly property color secondary: "#27ae60"
    readonly property color background: "#1e1e2e"
    readonly property color surface: "#313244"
    readonly property color onBackground: "#cdd6f4"
    readonly property color onSurface: "#cdd6f4"
    readonly property color error: "#f38ba8"

    readonly property int radiusSmall: 4
    readonly property int radiusMedium: 8
    readonly property int radiusLarge: 16

    readonly property int spacingXS: 4
    readonly property int spacingS: 8
    readonly property int spacingM: 16
    readonly property int spacingL: 24
    readonly property int spacingXL: 40

    readonly property font bodyFont: Qt.font({ family: "Inter",
pixelSize: 14 })
    readonly property font headingFont: Qt.font({ family: "Inter",
pixelSize: 20, weight: Font.DemiBold })
}
```

Components use token names rather than hardcoded values:

```
import com.example.app

Rectangle {
    color: Theme.surface
    radius: Theme.radiusMedium
    border.color: Theme.primary
    border.width: 2

    Text {
        font: Theme.bodyFont
        color: Theme.onSurface
    }
}
```

Runtime Theme Switching

For switchable light/dark or branded themes, make the `Theme` singleton mutable:

```
// Theme.qml
pragma Singleton
import QtQuick

QtObject {
    id: theme

    property string current: "dark"

    readonly property color primary: current === "dark" ? "#3daee9"
: "#1565c0"
    readonly property color background: current === "dark" ?
"#1e1e2e" : "#eff1f5"
    readonly property color onBackground: current === "dark" ?
"#cdd6f4" : "#4c4f69"
    // ...
}
```

When `Theme.current` changes, all bindings that reference `Theme.primary`, `Theme.background`, etc. re-evaluate automatically via the binding engine. The entire UI repaints with the new theme values in one frame.

Integrating with Qt's Palette System

Qt Quick Controls 2's palette system (Chapter 4) and custom design tokens can coexist. Use palette properties for controls that derive from `Control`, and token properties for custom items:

```
ApplicationWindow {
    // Set palette for all Controls
    palette.button: Theme.primary
    palette.buttonText: Theme.onPrimary
    palette.highlight: Theme.secondary

    // Custom items use Theme directly
    MyCustomHeader {
        background: Theme.surface
    }
}
```

This hybrid approach ensures built-in controls adopt theme colors without overriding them component-by-component, while custom items have full token flexibility.

Animated Theme Transitions

Smooth theme switching requires animating the color change across the entire UI. With bindings to `Theme.background` etc., a `Behavior on color` at the root level animates all changes:

```
// Apply to all Rectangles via a base component
Rectangle {
    Behavior on color {
        ColorAnimation { duration: 200; easing.type: Easing.OutCubic
}
    }
}
```

For a more targeted approach, animate the `Theme.current` switch with a opacity-based crossfade:

```
Item {
    Loader { id: currentThemeView; source: "MainContent.qml";
opacity: 1 }
    Loader { id: nextThemeView; opacity: 0 }

    function switchTheme(newTheme) {
        nextThemeView.source = "MainContent.qml"
        // ... crossfade animation ...
        Theme.current = newTheme
    }
}
```

Summary

Internationalization in Qt Quick is built around `qsTr()` for string marking, `lupdate` / `lrelease` for the translation compilation pipeline, and `QQmlEngine::retranslate()` for live language switching. Accessibility requires explicit `Accessible` attached property declarations on custom items; Qt Quick Controls handle this automatically for built-in controls. Theming through design tokens creates a single source of truth for visual values, enabling runtime theme switching through QML's binding engine with zero imperative update code. Combining these three systems produces applications that are globally deployable, universally accessible, and visually adaptable without architectural changes.

Chapter 20: Testing Qt Quick Applications

`qmltest` and `TestCase`: Unit Testing QML Components

`QtTest`'s QML API provides a declarative testing framework where tests are written in QML and run by the `qmltestrunner` tool. Tests live alongside the components they test and can directly manipulate QML items, simulate user input, and assert on property values.

Basic Test Structure

```
// tst_counter.qml
import QtTest
import com.example.backend

TestCase {
    name: "CounterTests"

    Counter {
        id: counter
    }

    function test_initialValue() {
        compare(counter.value, 0)
    }

    function test_increment() {
        counter.increment()
        compare(counter.value, 1)
        counter.increment()
        compare(counter.value, 2)
    }

    function test_reset() {
        counter.value = 10
        counter.reset()
        compare(counter.value, 0)
    }

    function test_valueChanged_signal() {
        let spy = Qt.createQmlObject(
            "import QtTest; SignalSpy { target: counter; signalName:
'valueChanged' }",
            testCase
        )
        counter.increment()
        compare(spy.count, 1)
    }
}
```

All functions whose names start with `test_` are test functions. `TestCase` runs them alphabetically. The framework provides:

Function	Assertion
`compare(actual, expected)`	Deep equality
`fuzzyCompare(a, b, delta)`	Floating point equality within delta
`verify(expression)`	Truthy value
`fail(message)`	Unconditional failure
`skip(message)`	Skip the test
`expectFail(tag, message)`	Expect this test to fail
`tryCompare(object, property, value, timeout)`	Wait for async property change
`tryVerify(expression, timeout)`	Wait for condition to become true

Testing Visual Components

To test a QML component's visual structure and geometry, instantiate it using the `TestCase` item itself as a parent or in a dedicated window:

```
TestCase {
    name: "ButtonTests"
    width: 300; height: 200
    visible: true     // Required for geometry to be computed

    function createButton(props) {
        return createTemporaryQmlObject(
            `import QtQuick.Controls
             Button { ${Object.entries(props).map(([k,v]) => `${k}:
"${v}"`).join(';')} }`,
            parent
        )
    }

    function test_buttonText() {
        let btn = createButton({ text: "Save" })
        compare(btn.text, "Save")
        verify(btn.implicitWidth > 0)
    }

    function test_buttonClick() {
        let clickCount = 0
        let btn = createButton({ text: "OK" })
        btn.clicked.connect(() => clickCount++)

        mouseClick(btn)
        compare(clickCount, 1)
    }
}
```

`createTemporaryQmlObject()` creates an object and automatically destroys it after the test function returns — preventing test state from leaking between tests.

SignalSpy

`SignalSpy` monitors signal emissions and records how many times a signal fired and what arguments it received:

```
SignalSpy {
    id: spy
    target: myObject
    signalName: "dataReady"
}

function test_signalEmitted() {
    spy.clear()
    myObject.fetchData()
    tryCompare(spy, "count", 1, 5000)  // wait up to 5 seconds
    compare(spy.signalArguments[0][0], expectedData)
}
```

`tryCompare` is essential for testing asynchronous operations: it polls the property until it matches or the timeout expires.

Simulating User Input

```
function test_keyboardInput() {
    let field = createTemporaryQmlObject(
        "import QtQuick.Controls; TextField { }", parent)
    field.forceActiveFocus()

    keyClick(Qt.Key_H)
    keyClick(Qt.Key_I)
    compare(field.text, "hi")

    keySequence(StandardKey.SelectAll)
    keyClick(Qt.Key_Delete)
    compare(field.text, "")
}

function test_mouseInteraction() {
    // mousePress, mouseRelease, mouseClick, mouseDoubleClick
    mouseClick(item, 10, 10, Qt.LeftButton)
    mouseDrag(item, 0, 0, 100, 0)
    mouseWheel(item, 0, 0, 0, -120)  // scroll down
}
```

Running Tests

```
# Run with qmltestrunner
qmltestrunner -import path/to/qml/modules -input tests/

# Run specific test file
qmltestrunner -input tests/tst_counter.qml

# With CMake
add_test(NAME QmlTests
    COMMAND qmltestrunner -import ${CMAKE_SOURCE_DIR}/qml -input
${CMAKE_CURRENT_SOURCE_DIR}
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
)
```

`QTest` for C++ Backend Logic

Qt's `QTest` framework is the standard for testing C++ code in Qt projects. It integrates with `ctest` and generates output compatible with CI systems.

Test Class Structure

```
// tst_contactmodel.cpp
#include <QTest>
#include "contactmodel.h"

class TestContactModel : public QObject
{
    Q_OBJECT

private slots:
    // Test lifecycle
    void initTestCase();     // Called once before all tests
    void cleanupTestCase();  // Called once after all tests
    void init();             // Called before each test
    void cleanup();          // Called after each test

    // Test functions
    void test_initiallyEmpty();
    void test_append();
    void test_remove();
    void test_dataChanged_signal();
    void test_filterProxy();
};

void TestContactModel::init() {
    // Called before each test — set up fresh state
}

void TestContactModel::test_initiallyEmpty() {
    ContactModel model;
    QCOMPARE(model.rowCount(), 0);
}

void TestContactModel::test_append() {
    ContactModel model;
    model.appendContact("Alice", "alice@example.com");
    QCOMPARE(model.rowCount(), 1);

    const QModelIndex idx = model.index(0);
    QCOMPARE(model.data(idx, ContactModel::NameRole).toString(),
"Alice");
    QCOMPARE(model.data(idx, ContactModel::EmailRole).toString(),
"alice@example.com");
}

void TestContactModel::test_remove() {
```

```
    ContactModel model;
    model.appendContact("Alice", "alice@example.com");
    model.appendContact("Bob", "bob@example.com");

    model.removeContact(0);

    QCOMPARE(model.rowCount(), 1);
    QCOMPARE(model.data(model.index(0),
ContactModel::NameRole).toString(), "Bob");
}

void TestContactModel::test_dataChanged_signal() {
    ContactModel model;
    model.appendContact("Alice", "alice@example.com");

    QSignalSpy spy(&model, &ContactModel::dataChanged);
    model.updateEmail(0, "new@example.com");

    QCOMPARE(spy.count(), 1);
    // Verify correct roles in the signal
    const QList<int> roles = spy[0][2].value<QList<int>>();
    QVERIFY(roles.contains(ContactModel::EmailRole));
    QVERIFY(!roles.contains(ContactModel::NameRole));
}

QTEST_MAIN(TestContactModel)
#include "tst_contactmodel.moc"
```

QSignalSpy

`QSignalSpy` records signal emissions. It is the QTest equivalent of QML's `SignalSpy`:

```
QSignalSpy spy(&model, &QAbstractItemModel::rowsInserted);
model.appendContact("Test", "test@example.com");

QCOMPARE(spy.count(), 1);
// Arguments: parent, first, last
QCOMPARE(spy[0][1].toInt(), 0);   // first row = 0
QCOMPARE(spy[0][2].toInt(), 0);   // last row = 0
```

Data-Driven Tests with `_data()` Functions

Test functions paired with a `_data()` function run once per row of the data table:

```
void TestContactModel::test_filterProxy_data() {
    QTest::addColumn<QString>("filterString");
    QTest::addColumn<int>("expectedCount");

    QTest::newRow("empty filter") << "" << 3;
    QTest::newRow("partial match") << "ali" << 1;
    QTest::newRow("no match") << "xyz" << 0;
    QTest::newRow("case insensitive") << "BOB" << 1;
}

void TestContactModel::test_filterProxy() {
    QFETCH(QString, filterString);
    QFETCH(int, expectedCount);

    ContactModel source;
    source.appendContact("Alice", "alice@example.com");
    source.appendContact("Bob", "bob@example.com");
    source.appendContact("Charlie", "charlie@example.com");

    QSortFilterProxyModel proxy;
    proxy.setSourceModel(&source);
    proxy.setFilterRole(ContactModel::NameRole);
    proxy.setFilterCaseSensitivity(Qt::CaseInsensitive);
    proxy.setFilterFixedString(filterString);

    QCOMPARE(proxy.rowCount(), expectedCount);
}
```

Testing Asynchronous C++ Code

For code that uses `QFuture`, `QTimer`, or signals to deliver results asynchronously:

```
void TestSearch::test_asyncSearch() {
    SearchModel model;
    QSignalSpy spy(&model, &SearchModel::searchComplete);

    model.search("qt quick");

    // Wait for the signal, up to 5 seconds
    QVERIFY(spy.wait(5000));
    QCOMPARE(spy.count(), 1);
    QVERIFY(model.rowCount() > 0);
}
```

`QSignalSpy::wait(ms)` runs the event loop until the signal fires or the timeout expires.

CMake Test Setup

```
# In tests/CMakeLists.txt
find_package(Qt6 REQUIRED COMPONENTS Test)

function(add_qt_test name)
    qt_add_executable(${name} ${name}.cpp)
    target_link_libraries(${name} PRIVATE Qt6::Test myapp_backend)
    add_test(NAME ${name} COMMAND ${name})
endfunction()

add_qt_test(tst_contactmodel)
add_qt_test(tst_searchservice)
add_qt_test(tst_filterproxy)
```

End-to-End UI Testing Strategies

Full end-to-end testing of Qt Quick UIs — simulating user interactions through the complete application stack — is the most difficult testing layer.

Approach 1: Squish (Commercial)

Froglogic's Squish is the industry-standard test automation tool for Qt applications. It supports:

- Object identification by name, type, or property
- Recording and playback of interactions
- Script-based tests in Python, JavaScript, Tcl, Ruby, or Perl
- Image recognition for custom-drawn items

```
# Squish test example (Python API)
startApplication("myapp")
clickButton(waitForObject(":mainWindow.saveButton_Button"))
type(waitForObject(":dialog.filenameField_TextField"), "output.pdf")
clickButton(waitForObject(":dialog.okButton_Button"))
waitForObjectExists(":statusBar.saveSuccess_Label")
```

Approach 2: `QAbstractItemModelTester`

For model testing specifically, `QAbstractItemModelTester` (from `Qt6::Test`) automatically validates that a model implementation follows the contract — correct begin/end bracket pairing, valid indices, consistent row counts:

```
void TestContactModel::test_modelIntegrity() {
    ContactModel model;
    auto *tester = new QAbstractItemModelTester(
        &model,
        QAbstractItemModelTester::FailureReportingMode::Fatal,
        this
    );

    // Perform model mutations — tester automatically validates each
change
    model.appendContact("Alice", "alice@example.com");
    model.appendContact("Bob", "bob@example.com");
    model.removeContact(0);
}
```

This catches common model implementation bugs (missing `beginInsertRows`, wrong row counts after mutation) that would otherwise appear as subtle view

glitches.

Approach 3: Headless Testing with `QOffscreenSurface`

For testing visual behavior without a display:

```
QT_QPA_PLATFORM=offscreen ./qmltestrunner -input tests/
```

The offscreen platform renders to memory buffers. Geometry is computed, animations run, and items respond to input — but nothing appears on screen. This is suitable for CI environments without a display.

Approach 4: Screenshot-Based Regression Testing

For visual regression detection, render items to images and compare:

```
void TestVisuals::test_buttonAppearance() {
    QQuickView view;
    view.setSource(QUrl("qrc:/qml/components/PrimaryButton.qml"));
    view.show();
    QTest::qWaitForWindowExposed(&view);

    // Grab the scene
    QImage rendered = view.grabWindow();
    QImage reference(":/test_references/primary_button_normal.png");

    QVERIFY(imagesMatch(rendered, reference, /* tolerance= */ 2));
}
```

Store reference images in version control. Run on CI with a fixed display DPI to avoid platform-dependent differences.

Summary

Testing a Qt Quick application spans three layers. QML's `TestCase` framework tests component behavior, user input simulation, and signal-driven async operations directly in QML. `QTest` handles C++ backend unit testing with typed assertions, `QSignalSpy`, data-driven test tables, and `QAbstractItemModelTester` for model contract validation. End-to-end testing relies on either commercial tools (Squish), headless rendering (offscreen platform), or screenshot regression tests. The highest-leverage tests are those for C++ models and service logic — they run fast, require no display, and cover the most error-prone code. QML component tests complement these for UI-specific behavior and accessibility.

Chapter 21: Architecture Patterns for Large Applications

MVVM in QML: Practical Boundaries Between Model, ViewModel, and View

Model-View-ViewModel (MVVM) maps cleanly onto Qt Quick's architecture because QML's binding engine naturally implements the ViewModel-to-View data flow that MVVM prescribes. The discipline is maintaining clean boundaries as the application grows.

Layer Responsibilities

Model (no Qt dependency, pure data and business logic):

- Domain entities as plain Python dataclasses or C++ structs
- Repository interfaces for data access (database, network, file system)
- Business rules and validation
- Independently unit-testable

ViewModel (Qt-aware, no visual opinion):

- `QObject` subclasses with `Q_PROPERTY` / `@Property` exposing view-ready data
- `QAbstractItemModel` subclasses for collections
- Transforms raw model data into display-ready form
- Exposes commands (`Q_INVOKABLE` / `@Slot`) for user actions
- Manages async operations (loading states, error states)
- Independently testable with `QTest` / `QSignalSpy`

View (QML only, no business logic):

- Binds to ViewModel properties
- Delegates user input to ViewModel commands
- Handles navigation, animation, and visual feedback
- Tested with `TestCase` / Squish

ViewModel Design Principles

1. Expose view-ready data, not raw domain data

```
# Bad: expose raw datetime — QML must format it
@Property(str, notify=timestampChanged)
def timestamp(self):
    return self._event.timestamp.isoformat()

# Good: expose formatted string — view just displays it
@Property(str, notify=timestampChanged)
def formattedTimestamp(self):
    return self._event.timestamp.strftime("%B %d, %Y at %I:%M %p")
```

2. Expose loading, empty, and error states explicitly

```
class ContactListViewModel(QObject):
    loadingChanged = Signal()
    errorChanged = Signal()
    emptyChanged = Signal()

    @Property(bool, notify=loadingChanged)
    def loading(self): return self._loading

    @Property(str, notify=errorChanged)
    def errorMessage(self): return self._error

    @Property(bool, notify=emptyChanged)
    def isEmpty(self): return len(self._contacts) == 0
```

In QML, these states drive which content to show:

```
StackLayout {
    currentIndex: {
        if (vm.loading) return 0
        if (vm.errorMessage) return 1
        if (vm.isEmpty) return 2
        return 3
    }

    LoadingSpinner { }
    ErrorView { message: vm.errorMessage }
    EmptyState { text: "No contacts yet" }
    ContactListView { model: vm.contacts }
}
```

3. ViewModels do not know about QML

A ViewModel should never import `QtQuick` or reference QML items. It is a pure data/logic adapter. The QML file imports and instantiates the ViewModel, not the reverse.

4. One ViewModel per screen

Each screen (or major feature area) has a dedicated ViewModel. Avoid monolithic application-level ViewModels that accumulate unrelated properties.

Plugin-Based Architectures with `QQmlExtensionPlugin`

For large applications that need to be divided into independently-built and independently-loadable modules, `QQmlExtensionPlugin` provides the mechanism.

What `QQmlExtensionPlugin` Solves

- **Separate compilation**: each plugin compiles independently; changes to one plugin do not require rebuilding others

- **Optional features**: plugins can be loaded conditionally at runtime
- **Third-party extension**: application developers can publish a plugin API; users (or other teams) extend the application by writing plugins
- **Team boundaries**: each team owns and ships a plugin; integration is through the published QML API

Creating a Plugin

```
// myplugin.h
#include <QQmlExtensionPlugin>

class MyPlugin : public QQmlExtensionPlugin
{
    Q_OBJECT
    Q_PLUGIN_METADATA(IID QQmlExtensionInterface_iid)

public:
    void registerTypes(const char *uri) override;
};


// myplugin.cpp
#include "myplugin.h"
#include "mytype.h"
#include <qqml.h>

void MyPlugin::registerTypes(const char *uri) {
    // Types registered here using the legacy API
    // OR use QML_ELEMENT macros — the plugin just initializes the
module
    qmlRegisterModule(uri, 1, 0);
}

// With modern CMake (Qt 6.6+), QML_ELEMENT macros handle
registration
// and the plugin class only needs to exist for dynamic loading
```

```
# CMakeLists.txt for the plugin
qt_add_library(myplugin SHARED)

qt_add_qml_module(myplugin
    URI "com.example.myplugin"
    VERSION 1.0
    PLUGIN_TARGET myplugin
    SOURCES
        myplugin.h myplugin.cpp
        mytype.h mytype.cpp
    QML_FILES
        MyView.qml
)
```

Plugin Discovery

Qt discovers plugins by searching `importPath` directories for `qmldir` files. The engine's default search paths include:

- The application's executable directory
- Paths added via `QQmlEngine::addImportPath()`
- `QT_QML_IMPORT_PATH` environment variable

```
// At application startup
engine.addImportPath("/opt/myapp/plugins");
```

QML then imports the plugin normally:

```
import com.example.myplugin

MyView { }
```

Runtime Plugin Loading

For truly dynamic plugin discovery (plugins not known at compile time):

```
QString pluginDir =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation)
                    + "/plugins";
engine.addImportPath(pluginDir);

// Scan the directory and list available plugins
QDir dir(pluginDir);
for (const QString &entry : dir.entryList(QDir::Dirs |
QDir::NoDotAndDotDot)) {
    QFileInfo qmldir(pluginDir + "/" + entry + "/qmldir");
    if (qmldir.exists()) {
        // Plugin is available — the engine will load it on first
import
        availablePlugins.append(entry);
    }
}
```

Structuring Mono-repos with Multiple QML Modules

Large-team Qt projects frequently use a mono-repo with multiple QML modules organized by domain or feature. The key challenge is managing dependencies between modules while maintaining buildability and testability in isolation.

Recommended Directory Structure

```
myapp/
├── CMakeLists.txt                (root build)
├── apps/
│   ├── desktop/                  (desktop application entry point)
│   └── mobile/                   (mobile application entry point)
├── modules/
│   ├── core/                     (shared utilities, base types)
│   │   ├── CMakeLists.txt
│   │   ├── qml/                  (QML: URI = com.example.core)
│   │   └── src/                  (C++/Python backend)
│   ├── contacts/                 (contacts feature)
│   │   ├── CMakeLists.txt
│   │   ├── qml/                  (QML: URI = com.example.contacts)
│   │   └── src/
│   ├── messaging/                (messaging feature)
│   │   ├── CMakeLists.txt
│   │   ├── qml/                  (QML: URI = com.example.messaging)
│   │   └── src/
│   └── settings/                 (settings feature)
│       ├── CMakeLists.txt
│       ├── qml/                  (QML: URI = com.example.settings)
│       └── src/
└── tests/
    ├── unit/
    └── integration/
```

Dependency Rules

Enforce a one-way dependency graph between modules:

```
apps/desktop   depends on   modules/*
modules/*      depend on    modules/core
modules/core   depends on   Qt only
```

Never have `core` depend on a feature module. Never have two feature modules depend on each other (if they share code, extract it to `core`).

Enforce this in CMake with explicit target_link_libraries dependencies and optional use of CMake's `FORBIDDEN_TARGETS` or custom lint checks.

Shared Component Library

The `core` module typically contains a shared component library of generic UI components (buttons, form fields, dialogs, cards) that all feature modules use:

```
modules/core/qml/
├── Button.qml              (custom styled button)
├── Card.qml                (surface card container)
├── Dialog.qml              (modal dialog base)
├── FormField.qml           (label + input group)
├── LoadingSpinner.qml
├── EmptyState.qml
└── ErrorView.qml
```

Feature modules import from `com.example.core` and combine core components with feature-specific logic:

```
// In com.example.contacts module
import com.example.core   // shared components

Card {
    FormField {
        label: "Name"
        textField.text: vm.name
        textField.onTextChanged: vm.name = textField.text
    }
}
```

Build-Time Module Isolation

Each module should build and test independently. Structure `CMakeLists.txt` so each module can be built with a `cmake --build . --target <module_name>` command:

```
# modules/contacts/CMakeLists.txt

# Backend library (C++ or Python bridge)
qt_add_library(contacts_backend STATIC
    src/contactmodel.cpp
    src/contactrepository.cpp
)
qt_add_qml_module(contacts_backend
    URI "com.example.contacts.backend"
    VERSION 1.0
    SOURCES src/contactmodel.h src/contactrepository.h
)
target_link_libraries(contacts_backend PUBLIC
    core_backend   # shared backend utilities
    Qt6::Quick Qt6::Sql
)

# QML module
qt_add_library(contacts_qml STATIC)
qt_add_qml_module(contacts_qml
    URI "com.example.contacts"
    VERSION 1.0
    QML_FILES
        qml/ContactListScreen.qml
        qml/ContactDetailScreen.qml
        qml/ContactEditDialog.qml
)
target_link_libraries(contacts_qml PUBLIC
    contacts_backend
    core_qml       # shared QML components
)
```

Versioning Across Modules

Each QML module has a version (`VERSION 1.0` in `qt_add_qml_module`). When a module's public API changes in a backward-incompatible way, increment the major version. Consumers import a specific version:

```
import com.example.contacts 2.0  // explicit version
```

For minor additions (new types, new properties), increment the minor version. Consumers that import `1.0` continue to work when `1.3` is installed, because

Qt's import system treats minor version increments as backward-compatible.

Inter-Module Communication

Feature modules should not call each other directly. They communicate through:

Shared singleton state in `core`:

```
// com.example.core — NavigationState singleton
pragma Singleton
QtObject {
    signal navigateRequested(string module, string screen, var args)
}


// In contacts module
Button {
    onClicked: NavigationState.navigateRequested("messaging",
"NewMessage",
                                                 { recipientId:
contact.id })
}
```

The application shell (in `apps/desktop`) connects to `NavigationState.navigateRequested` and performs the actual navigation.

Shared data models in `core`:

Types that cross feature boundaries live in `core`. The `UserSession` singleton (authentication state) belongs in `core`, not in the `settings` module.

Feature Flags and Module Activation

Large applications often need to enable/disable features at runtime (A/B testing, beta features, licensing tiers):

```
// FeatureFlags.qml — singleton in core
pragma Singleton
QtObject {
    property bool advancedSearch: false
    property bool darkMode: true
    property bool betaFeatures: false
}
```

Feature modules check flags before showing their content:

```
Loader {
    active: FeatureFlags.advancedSearch
    source: "AdvancedSearchPanel.qml"
}
```

On the C++ side, feature flags can be loaded from a remote configuration service, a license file, or a local settings file. Changing a flag property propagates through all QML bindings immediately.

Summary

MVVM in QML is not an aspirational pattern — it is the natural shape of a well-structured PySide6 or C++/QML application. The binding engine implements the ViewModel-to-View data flow; the discipline is keeping business logic out of QML and visual logic out of the backend. Plugin-based architecture via `QQmlExtensionPlugin` enables independently compiled and optionally loadable feature modules, suitable for platform extension and team isolation. Mono-repo structures with multiple QML modules — each with its own URI, version, and `qt_add_qml_module` target — scale to large teams by enforcing module boundaries, one-way dependency graphs, and independent buildability. Inter-module communication through shared singletons and navigation signals avoids the tight coupling that would otherwise accumulate as features grow.